

Universitatea Națională de Știință și Tehnologie POLITEHNICA București
Facultatea de Electronică, Telecomunicații și Tehnologia Informației

Sistem RAG pentru extragerea relațiilor între entități

Proiect de diplomă

prezentat ca cerință parțială pentru obținerea
titlului de *Inginer*
în domeniul *Inginerie Electronică, Telecomunicații și Tehnologii*
Informaționale
programul de studii *Electronică Aplicată*

Conducător științific
Ș.l. Dr. Ing. Liviu-Daniel ȘTEFAN
Drd. Fiz. Alexandru IONIȚĂ

Absolvent
Andreas BAȘCHIR

Anul 2026

Universitatea Națională de Știință și Tehnologie POLITEHNICA București
Facultatea de Electronică, Telecomunicații și Tehnologia Informației
Program de studiu **ELA**

Anexa 1

TEMA PROIECTULUI DE DIPLOMĂ
a studentului **BAȘCHIR V. Andreas, 442B**

1. Titlul temei: Sistem RAG pentru extragerea relațiilor între entități

2. Descrierea temei:

Retrieval-Augmented Generation (RAG) aparține ariei mai largi a procesării limbajului natural și urmărește integrarea dinamică a informațiilor externe în procesul de generare, prin combinarea unui modul de regăsire a informației cu un model generativ autoregresiv. Dată o interogare, un sistem RAG recuperează un set de documente candidate (documente text, imagini, înregistrări video, audio, etc.) dintr-o sursă externă și condiționează procesul de decodare pe aceste evidențe pentru a genera răspunsuri fundamentate factual. Prin această formulare, RAG abordează problema structurală a modelelor lingvistice mari, și anume dependența exclusivă de cunoștințele încorporate în corpusul de pre-antrenare, și investighează în ce măsură integrarea explicită a recuperării de informații poate crește robustețea, interpretabilitatea și rezistența la halucinații a sistemelor generative.

3. Contribuția originală:

Se vor studia și implementa metode de tip RAG, vizând optimizarea etapelor de regăsire, selecție și agregare a documentelor, precum și dezvoltarea unor mecanisme generative care integrează explicit evidențele recuperate. În completare, va fi dezvoltat un demonstrator care integrează aceste metode și ilustrează modul de regăsire a informațiilor relevante într-un cadru RAG.

Lucrarea va cuprinde un studiu al literaturii de specialitate, prezentarea metodelor utilizate, validarea acestora pe seturi de date publice și analiza cantitativă și calitativă a rezultatelor obținute. În final, vor fi formulate concluziile cercetării și vor fi indicate viitoarele perspective de cercetare.

4. Resurse și bibliografie:

- [1] Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., Dai, Y., Sun, J., Wang, H. and Wang, H., 2023. Retrieval-augmented generation for large language models: A survey. arXiv preprint arXiv:2312.10997, 2(1).
- [2] Mei, L., Mo, S., Yang, Z. and Chen, C., 2025. A survey of multimodal retrieval-augmented generation. arXiv preprint arXiv:2504.08748.
- [3] Chen, J., Xu, D., Fei, J., Feng, C.M. and Elhoseiny, M., 2025. Document haystacks: Vision-language reasoning over piles of 1000 documents. In Proceedings of the Computer Vision and Pattern Recognition Conference (pp. 24817-24826).
- [4] Xia, P., Zhu, K., Li, H., Wang, T., Shi, W., Wang, S., Zhang, L., Zou, J. and Yao, H., 2024. Mmed-rag: Versatile multimodal rag system for medical vision language models. arXiv preprint arXiv:2410.13085.
- [5] Hao, H., Han, J., Li, C., Li, Y.F. and Yue, X., 2025. RAP: Retrieval-Augmented Personalization for Multimodal Large Language Models. In Proceedings of the Computer Vision and Pattern Recognition Conference (pp. 14538-14548).
- [6] Jin, B., Yoon, J., Han, J., and Arik, S., 2025. Long-Context LLMs Meet RAG: Overcoming Challenges for Long Inputs in RAG. In Y. Yue, A. Garg, N. Peng, F. Sha, and R. Yu (Eds), International Conference on Representation Learning (Vol. 2025, pp. 37784-37822).
- [7] Asai, A., Wu, Z., Wang, Y., Sil, A., and Hajishirzi, H., 2024. Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection. International Conference on Representation Learning.

5. Data înregistrării temei: 2026-01-11 20:58:13

Conducător(i) lucrare,
Ș.L. dr. ing. Liviu-Daniel ȘTEFAN

Student,
BAȘCHIR V. Andreas

Drd. Fiz. Alexandru IONIȚĂ

Director departament,
Ș.L. dr. ing Bogdan FLOREA

Decan,
Prof. dr. ing. Mihnea UDREA

Cod Validare: **d96a201bb0**

Copyright © 2026, Andreas BAȘCHIR

Toate drepturile rezervate

Autorul acordă POLITEHNICA București dreptul de a reproduce și de a distribui public copii pe hârtie sau electronice ale acestei lucrări, în formă integrală sau parțială.

Declarație de onestitate academică

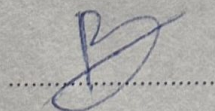
Prin prezenta declar că lucrarea cu titlul *Sistem RAG pentru extragerea relațiilor între entități*, prezentată în cadrul Facultății de Electronică, Telecomunicații și Tehnologia Informației a Universității Naționale de Știință și Tehnologie POLITEHNICA București ca cerință parțială pentru obținerea titlului de *Inginer* în domeniul *Electronică și Telecomunicații*, programul de studii *Electronică Aplicată* este scrisă de mine și nu a mai fost prezentată niciodată la o facultate sau instituție de învățământ superior din țară sau străinătate.

Declar că toate sursele utilizate, inclusiv cele de pe Internet, sunt indicate în lucrare, ca referințe bibliografice. Fragmentele de text din alte surse, reproduse exact, chiar și în traducere proprie din altă limbă, sunt scrise între ghilimele și fac referință la sursă. Reformularea în cuvinte proprii a textelor scrise de către alți autori face referință la sursă. Înțeleg că plagiatul constituie infracțiune și se sancționează conform legilor în vigoare.

Declar că toate rezultatele simulărilor, experimentelor și măsurărilor pe care le prezint ca fiind făcute de mine, precum și metodele prin care au fost obținute, sunt reale și provin din respectivele simulări, experimente și măsurători. Înțeleg că falsificarea datelor și rezultatelor constituie fraudă și se sancționează conform regulamentelor în vigoare.

București, Iunie 2026.

Absolvent: Andreas BAȘCHIR



Cuprins

Lista figurilor	11
Lista tabelelor	13
Lista acronimelor	15
Introducere	17
1. Studiu de literatură	21
1.1. Prezentarea domeniului lucrării de diplomă	21
1.1.1. De la rețele neuronale la modele lingvistice mari	21
1.1.2. Cunoștințe statice, halucinații și soluția RAG	21
1.1.3. RAG versus <i>fine-tuning</i>	23
1.2. Formularea problemei: Extragerea relațiilor între entități	23
1.2.1. Definiție și formalizare	23
1.2.2. Reprezentarea relațiilor: triplete și grafuri de cunoștințe	24
1.2.3. Metode clasice și limitările lor	24
1.2.4. De ce RAG pentru extragerea relațiilor	25
1.3. Taxonomie	26
1.3.1. Motivația definirii unei taxonomii	26
1.3.2. Criterii de clasificare	26
1.3.3. Categori principale	27
1.4. Arhitectura generală a unui sistem de tip RAG	28
1.4.1. Componentele sistemului	28
1.4.2. Fluxul de date	29
1.4.3. Indexarea: fragmentare, codificare și stocare vectorială	30
1.4.4. Construcția promptului și augmentarea contextului	31
1.4.5. Compromisuri în proiectarea unui sistem RAG	31
1.5. Metode și modele RAG	32
1.5.1. Naive RAG și Advanced RAG	32
1.5.2. Optimizări ale recuperării	32
1.5.3. Arhitecturi cu recuperare integrată	33
1.5.4. Recuperare adaptivă și auto-corecție	34
1.5.5. Structuri ierarhice, grafuri și contexte lungi	35
1.5.6. Sinteză comparativă	35
1.6. Metrice de evaluare	36
1.6.1. Metrice de recuperare	36
1.6.2. Metrice de generare	37
1.6.3. Metrice specifice sistemelor RAG	38
1.6.4. Dificultăți în evaluarea extragerii relațiilor	38
1.7. Baze de date	39

1.7.1.	Seturi de date pentru sisteme RAG	39
1.7.2.	Seturi de date pentru extragerea relațiilor	40
1.8.	Limitări și provocări	41
1.9.	Concluzii	42
2.	Arhitectura sistemului propus	43
2.1.	Privire de ansamblu	43
2.2.	Cerințe	43
2.3.	Tehnologii utilizate	44
2.4.	Configurația hardware	45
2.5.	Indexarea datelor	45
2.6.	Fluxul de procesare Naive RAG	45
2.7.	Fluxul de procesare Advanced RAG	46
3.	Rezultate experimentale	49
3.1.	Evaluarea pe SQuAD	49
3.1.1.	Configurarea experimentului	49
3.1.2.	Asigurarea unei comparații corecte	50
3.1.3.	Rezultate	50
3.1.4.	Semnificație statistică	51
3.1.5.	Stabilitatea rezultatelor pe seed-uri	51
3.1.6.	Unde ajută recuperarea hibridă	52
3.1.7.	Decalajul dintre recuperare și generare	53
3.1.8.	Observații calitative din interfața Streamlit	54
3.1.9.	Analiza erorilor	54
3.2.	Evaluarea pe extragerea relațiilor între entități	55
3.2.1.	Setul de date DocRED și formularea ca întrebări	55
3.2.2.	Rezultate pe recuperare și generare	56
3.2.3.	Rezistența la halucinații	57
3.2.4.	Fidelitatea răspunsurilor și efectul de selecție	58
3.3.	Discuție și limitări	59
3.3.1.	Compromisul dintre calitate și latență	59
3.3.2.	Constrângerea de ancorare și refuzurile de a răspunde	60
3.3.3.	Limitările analizei	61
3.4.	Demonstratorul	62
3.4.1.	Interfața și fluxul utilizatorului	62
3.4.2.	Modulul de evaluare	63
3.4.3.	Procesul de dezvoltare	64
	Concluzii	67
	Bibliografie	71
	Anexa 1. Cod sursă - ingest.py	75
	Anexa 2. Cod sursă - main.py	77
	Anexa 3. Cod sursă - benchmark.py	79

Lista figurilor

1.1. Creșterea numărului de lucrări despre modele lingvistice mari (LLMs) publicate pe arXiv, 2017–2025.	22
1.2. Harta conceptuală a taxonomiei adoptate, structurată pe cele patru criterii relevante pentru extragerea relațiilor între entități [5, 18].	28
1.3. Arhitectura generală a unui sistem RAG: indexare offline (sus) și inferență online (jos) [5, 20].	30
2.1. Faza offline de indexare. Unitățile unice ale setului de date (paragrafe pentru SQuAD, documente pentru DocRED) sunt codificate o singură dată și depuse în indexul dens (ChromaDB) și în indexul rar (BM25), folosite în comun de ambele fluxuri de procesare. Fiecare set de date are propriile indexuri.	44
2.2. Fluxul de procesare Naive RAG. Interogarea este codificată, cele mai apropiate cinci fragmente sunt căutate prin similaritate cosinus în ChromaDB și trimise direct modelului generativ, fără reformulare, recuperare hibridă sau reordonare.	46
2.3. Fluxul de procesare Advanced RAG. Interogarea este reformulată, fiecare variantă este căutată dens și prin BM25, rezultatele sunt fuzionate prin Reciprocal Rank Fusion, re-scorate cu un cross-encoder, iar primele cinci fragmente sunt trimise modelului generativ.	47
3.1. Metrici de recuperare: Context Recall@5 și MRR pentru Naive RAG și Advanced RAG. Barele de eroare reprezintă deviația standard calculată pe cele 4 seed-uri.	51
3.2. Calitatea răspunsurilor: Answer F1 și Exact Match pentru Naive RAG și Advanced RAG.	52
3.3. Latența medie de procesare a interogării în milisecunde. Latența Advanced RAG include apelul LLM pentru expansiunea interogării, recuperarea hibridă paralelă și re-scorarea cu cross-encoder.	53
3.4. Vedere de ansamblu comparativă a metricilor de calitate (Context Recall@5, MRR, Answer F1, Exact Match). Latența nu este inclusă deoarece se află pe o scară diferită, în milisecunde față de intervalul de la 0 la 1.	54
3.5. Metrici de recuperare pe DocRED: Context Recall@5 și MRR pentru Naive RAG și Advanced RAG. Avantajul arhitecturii avansate este vizibil mai mare decât pe SQuAD, deoarece întrebările despre relații depind de nume proprii de entități, pe care recuperarea hibridă le potrivește lexical. Barele de eroare reprezintă deviația standard pe cele 3 seed-uri.	57
3.6. Calitatea răspunsurilor pe DocRED: Answer F1 și Exact Match. Answer F1 este scăzut și plat deoarece compară token-uri cu entități de referință foarte scurte, în timp ce modelul răspunde în propoziții; Exact Match, care verifică prezența entității corecte, este metrica de generare relevantă aici.	58
3.7. Rata de halucinație pe cele 96 de întrebări fără răspuns din DocRED (valori mai mici sunt mai bune). Ambele sisteme refuză să răspundă în peste 90% din cazuri, ceea ce confirmă efectul constrângerii de ancorare. Diferența dintre cele două nu este semnificativă statistic la acest număr de întrebări.	59

3.8. Metrice RAGAS pe DocRED, calculate pe subsetul propriu fiecărui sistem (refuzurile excluse). Advanced RAG pare mai puțin fidel decât Naive, dar comparația este înșelătoare: cele două sisteme sunt notate pe seturi diferite de întrebări, deoarece refuză întrebări diferite.	60
3.9. Metrice RAGAS pe subsetul comun ($N = 70$), unde ambele sisteme răspund la aceleași întrebări. Decalajul de fidelitate din Figura 3.8 dispare: pe întrebări identice, cele două sisteme sunt la fel de fidele și la fel de relevante.	61
3.10. Vedere de ansamblu a metricilor de calitate pe DocRED. Fidelitatea și relevanța sunt calculate pe subsetul comun, pentru o comparație corectă. Advanced RAG conduce la recuperare și la Exact Match, iar la fidelitate și relevanță cele două sisteme sunt la egalitate. Latența nu este inclusă deoarece se află pe o scară diferită.	62
3.11. Interfața comparativă a demonstratorului. Cele două coloane corespund fluxurilor de procesare Naive și Advanced, care răspund simultan la aceeași întrebare, cu timpii de generare afișați deasupra fiecărui răspuns. În acest exemplu, Naive RAG nu găsește răspunsul, în timp ce Advanced RAG recuperează fragmentul corect.	63
3.12. Exemplu de interogare cu termeni numerici. Naive RAG produce un răspuns numeric halucinat, deoarece similaritatea cosinus ratează valoarea exactă, în timp ce Advanced RAG găsește valoarea corectă prin recuperarea BM25.	64

Lista tabelelor

1.1. Sinteză comparativă a metodelor RAG prezentate și a relevanței lor pentru extragerea relațiilor între entități.	36
1.2. Comparatie a seturilor de date pentru evaluarea RAG și extragerea relațiilor [3, 4, 11–14, 41, 42, 44, 45].	40
2.1. Tehnologiile utilizate în demonstrator	44
2.2. Parametrii fluxului de procesare Advanced RAG	46
3.1. Rezultate comparative Naive RAG vs. Advanced RAG pe SQuAD ($N = 600$ per sistem, medie $\pm$ deviație standard pe 4 seed-uri)	50
3.2. Teste de semnificație statistică (Advanced RAG vs. Naive RAG, $N = 600$ perechi)	51
3.3. Context Recall@5 și MRR pe fiecare dintre cele 4 seed-uri (150 de întrebări per seed)	52
3.4. Rezultate comparative pe DocRED ($N = 300$ per sistem: 204 cu răspuns plus 96 fără răspuns, medie $\pm$ deviație standard pe 3 seed-uri)	56
3.5. RAGAS pe subsetul comun de întrebări la care ambele sisteme au răspuns efectiv ($N = 70$, DocRED)	59

Lista acronimelor

API	Application Programming Interface
BERT	Bidirectional Encoder Representations from Transformers
BLEU	Bilingual Evaluation Understudy (metrică de evaluare pentru generare)
BM25	Best Matching 25 (funcție de scor pentru recuperare rară)
ColBERT	Contextualized Late Interaction over BERT
CRAG	Corrective Retrieval-Augmented Generation
DocRED	Document-level Relation Extraction Dataset
DPR	Dense Passage Retrieval
EM	Exact Match
FAISS	Facebook AI Similarity Search
FiD	Fusion-in-Decoder
FLARE	Forward-Looking Active REtrieval
GPT	Generative Pre-trained Transformer
HNSW	Hierarchical Navigable Small World (graf de căutare aproximativă)
HyDE	Hypothetical Document Embeddings
LLM	Large Language Model (model lingvistic mare)
MRR	Mean Reciprocal Rank
NLP	Natural Language Processing (procesarea limbajului natural)
OWL	Web Ontology Language (limbaj de ontologie web)
PID	Property Identifier (identificator de proprietate în Wikidata)
QA	Question Answering (răspuns la întrebări)
QID	Q-identifier (identificator de element canonic în Wikidata)
RAG	Retrieval-Augmented Generation (generare augmentată prin recuperare)
RAGAS	Retrieval-Augmented Generation Assessment
RAPTOR	Recursive Abstractive Processing for Tree-Organized Retrieval
RDF	Resource Description Framework

RDFS Resource Description Framework Schema

REALM Retrieval-Augmented Language Model Pre-Training

RETRO Retrieval-Enhanced Transformers

ROUGE Recall-Oriented Understudy for Gisting Evaluation (metrică de evaluare)

RRF Reciprocal Rank Fusion

SQuAD Stanford Question Answering Dataset

SVM Support Vector Machine (mașină cu vectori suport)

TACRED TAC Relation Extraction Dataset

Introducere

Scopul lucrării de diplomă

Sistemele RAG (Retrieval-Augmented Generation) reprezintă o arhitectură de inteligență artificială care combină recuperarea informațiilor relevante din surse externe cu generarea de text de către modele de limbaj mari (LLMs - Large Language Models). În loc să se bazeze exclusiv pe cunoștințele încorporate în parametrii modelului la antrenare, un sistem RAG caută mai întâi documente relevante pentru întrebarea utilizatorului, apoi le folosește ca bază pentru răspuns. RAG a fost propus ca soluție pentru natura statică a informațiilor reținute de modelele de limbaj mari, consecință a antrenării pe durate limitate, cu pauze între versiuni. Un model antrenat în 2023 nu cunoaște evenimente ulterioare și nu poate fi corectat fără reantrenare, un proces costisitor. Recuperarea externă rezolvă această problemă printr-un mecanism mult mai ieftin: actualizarea bazei de cunoștințe, nu a modelului.

Sistemele RAG au apărut oficial în 2020, prin lucrarea „Retrieval-Augmented Generation for Knowledge-Intensive NLP (*Natural Language Processing*) Tasks” publicată de Lewis et al. de la Facebook AI Research (astăzi Meta AI) [1], în contextul exploziei modelelor transformer precum BERT (*Bidirectional Encoder Representations from Transformers*) și GPT (*Generative Pre-trained Transformer*), care generau răspunsuri creative, dar adesea inexacte sau „halucinate” din cauza limitărilor cunoștințelor lor statice. De la varianta inițială „naive”, domeniul a evoluat rapid: au apărut arhitecturi precum Advanced RAG și Modular RAG, iar Microsoft a introdus GraphRAG [2] pentru relații complexe prin grafuri de cunoștințe. Recuperarea hibridă, care combină vectori denși și rari, s-a impus ca practică standard, iar astăzi RAG a trecut de mult de stadiul de prototip, fiind folosit în producție în aplicații de business, asistenți conversaționali și agenți.

Lucrarea de față se concentrează pe aplicarea RAG la extragerea relațiilor între entități, o sarcină de bază în procesarea limbajului natural care constă în identificarea legăturilor dintre entitățile menționate într-un text. Spre deosebire de clasificatoarele specializate, antrenate pentru un set fix de relații, un sistem RAG poate recupera contextul relevant pentru o pereche de entități și poate generaliza către relații noi fără reantrenare, ceea ce îl face o direcție atractivă pentru această sarcină. Extragerea relațiilor stă la baza construirii grafurilor de cunoștințe și a răspunsului la întrebări factuale, însă metodele clasice, bazate pe învățare supervizată, cer date adnotate pentru fiecare tip de relație și nu se transferă ușor către relații nevăzute. RAG promite să atenueze această limitare, dar rămâne neclar cât de mult ajută optimizările uzuale de recuperare atunci când întrebarea depinde de un nume propriu de entitate, nu de o similaritate tematică. Scopul lucrării este, pe de o parte, un studiu al literaturii de specialitate privind sistemele RAG și extragerea relațiilor între entități, iar pe de altă parte un studiu comparativ care măsoară, în condiții controlate, cât aduc optimizările de recuperare pe două sarcini: răspunsul la întrebări extractiv și extragerea relațiilor între entități.

Gradul de noutate

Lucrările care compară arhitecturi RAG sau introduc tehnici noi de recuperare se evaluează aproape întotdeauna pe sarcini de răspuns la întrebări de uz general, precum SQuAD (*Stanford*

Question Answering Dataset) sau Natural Questions. Pe aceste seturi, întrebarea și răspunsul se potrivesc adesea tematic cu un singur pasaj, iar recuperarea densă funcționează deja bine. Mai puțin clar este cât de mult ajută optimizările de tip Advanced RAG (recuperare hibridă densă și rară, fuziune prin Reciprocal Rank Fusion, reordonare cu cross-encoder și reformularea interogării) atunci când sarcina este extragerea relațiilor între entități, unde fiecare întrebare se sprijină pe un nume propriu specific, adesea rar în corpus.

Aici se situează contribuția lucrării, construită pe două comparații. Prima compară un sistem RAG de bază cu unul avansat, ținând constante corpusul, modelul generativ și promptul, pentru a măsura cât de semnificative sunt îmbunătățirile aduse de arhitectura de recuperare. A doua pune față în față două sarcini, răspunsul la întrebări de uz general, folosit ca referință, și extragerea relațiilor între entități, sarcina-țintă a lucrării, pentru a vedea dacă aceste îmbunătățiri se transferă diferit de la o sarcină la alta. Întrebarea centrală, în acord cu titlul lucrării, este dacă un flux de procesare RAG avansat aduce un câștig mai mare la extragerea relațiilor decât la răspunsul la întrebări și cu ce costuri, în latență, fidelitate și refuzuri. Gradul de noutate nu stă în componentele folosite, care sunt preluate din literatură, ci în întrebarea pusă și în condițiile comparației. Rezultatele nu reproduc pur și simplu cifre cunoscute: ele arată, pe date proprii, că avantajul recuperării avansate este sistematic mai mare la extragerea relațiilor între entități decât la răspunsul la întrebări extractive, un rezultat care nu se observă dacă evaluarea rămâne pe un singur set de date de uz general.

Obiectivele proiectului

Lucrarea are patru obiective concrete, subordonate întrebării de cercetare de mai sus:

1. Un studiu al literaturii care acoperă atât sistemele RAG (definiție, arhitectură, metode reprezentative și metrice de evaluare), cât și extragerea relațiilor între entități ca domeniu de sine stătător: ce înseamnă o relație, cum este reprezentată, ce metode existau înainte de modelele de limbaj și unde se încadrează RAG între ele;
2. Proiectarea unei comparații controlate între o arhitectură RAG de bază și una avansată, în care corpusul, modelul generativ și promptul de ancorare sunt identice, astfel încât diferențele de performanță să provină exclusiv din etapa de recuperare;
3. Evaluarea celor două sisteme pe cele două sarcini: răspunsul la întrebări extractiv, pe SQuAD [3], folosit pentru validarea arhitecturii, și extragerea relațiilor între entități, pe DocRED (*Document-level Relation Extraction Dataset*) [4], sarcina-țintă a lucrării. Evaluarea acoperă metrice de recuperare (Context Recall@5, MRR (*Mean Reciprocal Rank*)), de generare (Exact Match, Answer F1), latența, rezistența la halucinații pe întrebări fără răspuns și fidelitatea răspunsului prin RAGAS (*Retrieval-Augmented Generation Assessment*), toate însoțite de teste de semnificație statistică;
4. Analiza rezultatelor: unde și de ce ajută recuperarea avansată la extragerea relațiilor, ce limite are formularea aleasă și ce direcții rămân deschise, de la extragerea completă a tripletelor până la evaluarea pe seturi complementare precum T-REx.

Contribuții și rezultate

Contribuția principală este studiul comparativ realizat: o măsurare, în condiții controlate, a cât de mult ajută optimizările de recuperare dintr-un sistem Advanced RAG față de unul Naive RAG. Comparația ține constant corpusul, modelul generativ și promptul de ancorare și variază

doar etapa de recuperare, astfel încât diferențele observate să provină exclusiv din ea. Cele două sisteme sunt comparate din mai multe puncte de vedere: calitatea recuperării, calitatea răspunsului generat, latența, tendința de a halucina pe întrebări fără răspuns și fidelitatea răspunsului față de contextul recuperat.

Pe răspunsul la întrebări extractiv, sistemul a fost validat mai întâi pe SQuAD, ales pentru că răspunsul este un fragment de text cu sursă cunoscută, ceea ce permite o măsurare automată și reproductibilă a recuperării, fără a depinde de un model-judecător. Pe acest set, arhitectura avansată depășește varianta de bază pe toate metricile de calitate, cu diferențe semnificative statistic: Context Recall@5 crește de la 0,835 la 0,955, MRR de la 0,719 la 0,904, iar Exact Match de la 0,642 la 0,745. Costul este latența, de aproximativ 50 de ori mai mare, provenită mai ales din reformularea interogării cu un model de limbaj.

Pe extragerea relațiilor între entități, sistemul a fost evaluat pe DocRED, unde fiecare relație a fost reformulată ca întrebare după modelul extragerii prin citire. Aici avantajul recuperării avansate este chiar mai mare decât pe SQuAD: comparativ cu varianta de bază (Naive RAG), sistemul Advanced RAG crește scorul Context Recall de la 0,595 la 0,802 (un plus de 20,7 puncte procentuale, față de 12,0 pe SQuAD), iar MRR de la 0,525 la 0,752. Pe întrebări construite special fără răspuns în corpus, ambele sisteme refuză corect în peste 90% din cazuri, ceea ce confirmă rezistența la halucinații dată de constrângerea de ancorare. Evaluarea fidelității cu RAGAS, pe subsetul de întrebări la care ambele sisteme răspund, arată o fidelitate și o relevanță practic egale, deci recuperarea avansată răspunde la mai multe întrebări fără a pierde din fidelitatea răspunsului. Concluzia este că optimizările de recuperare ajută mai mult sarcina-țintă a lucrării decât răspunsul la întrebări de uz general.

Aceste rezultate au fost obținute cu ajutorul unui demonstrator *open-source*, care implementează ambele sisteme și le expune printr-o interfață Streamlit pentru testare interactivă, și al unui modul de evaluare automată care rulează comparația pe mai multe seed-uri, aplică testele de semnificație statistică și generează figurile.

Conținutul lucrării

Lucrarea este structurată în trei capitole. Primul construiește fundamentul teoretic, al doilea descrie arhitectura sistemului propus, iar al treilea o pune la încercare experimental.

Capitolul 1, studiul de literatură, parcurge domeniul dinspre general spre specific. Pornește de la contextul modelelor de limbaj și de la definiția generării augmentate prin recuperare, apoi definește problematica extragerii relațiilor între entități, sarcina-țintă pentru care RAG poate fi o soluție. Continuă cu o taxonomie a sistemelor RAG după tipul recuperării, al integrării și al sursei de cunoștințe, descrie arhitectura generală a unui flux de procesare RAG și metodele reprezentative din literatură, precum Self-RAG, CRAG (*Corrective Retrieval-Augmented Generation*), HyDE (*Hypothetical Document Embeddings*) și GraphRAG. Urmează metricile de evaluare pentru fiecare etapă a fluxului de procesare și seturile de date folosite în acest domeniu. Capitolul se încheie cu limitările și provocările actuale, iar concluzia care se desprinde, anume că nicio arhitectură nu domină pe toate criteriile, motivează abordarea experimentală: în loc să implementeze direct cea mai complexă variantă, lucrarea pornește de la una simplă și măsoară efectul fiecărei optimizări.

Capitolul 2 descrie arhitectura sistemului propus: cerințele, tehnologiile și proiectarea celor două sisteme, Naive RAG, cu recuperare densă și generare, și Advanced RAG, cu recuperare hibridă, fuziune, reordonare și reformularea interogării.

Capitolul 3 prezintă evaluarea experimentală. Arhitectura este validată întâi pe SQuAD, un set de date de citire extractivă care permite o evaluare automată și precisă a recuperării, apoi este testată pe DocRED, pe extragerea relațiilor între entități, sarcina-țintă a lucrării. Analiza

cantitativă acoperă metricile de recuperare și de generare, latența, rezistența la halucinații și fidelitatea răspunsului măsurată cu RAGAS, completată de o analiză calitativă a comportamentului celor două sisteme. La final, o secțiune dedicată descrie demonstratorul, care poate fi testat interactiv într-o interfață implementată în Streamlit.

Capitolul 1

Studiu de literatură

1.1 Prezentarea domeniului lucrării de diplomă

1.1.1 De la rețele neuronale la modele lingvistice mari

Apariția arhitecturii transformer în 2017 și a modelelor lingvistice mari (Large Language Models - LLMs) a dus la o schimbare paradigmatică a abordării problemelor de procesare a limbajului natural. Modele precum BERT și GPT-3 au demonstrat capacități remarcabile în înțelegerea și generarea de text coerent, favorizând o adoptare rapidă a LLM-urilor la nivel global, cercetarea despre LLM-uri devenind un punct central de interes. În Figura 1.1 se poate observa creșterea exponențială a numărului de lucrări despre LLM-uri publicate pe arXiv în perioada 2017–2025. Totuși, LLM-urile de sine stătătoare au o limitare fundamentală, cunoștințele lor sunt fixe, limitate la momentul antrenării, neputând fi actualizate fără a fi reantrenate [5].

Pentru a înțelege de ce a apărut RAG, este util un scurt parcurs al evoluției modelelor de procesare a limbajului. Învățarea profundă (eng. „*deep learning*”) a schimbat radical modul de abordare a sarcinilor de inteligență artificială, înlocuind trăsăturile construite manual cu reprezentări învățate automat din date [6, 7]. În procesarea limbajului natural, acest lucru a însemnat trecerea de la modele care se bazează pe reguli și pe trăsături lexicale la modele care învață singure reprezentări ale cuvintelor și ale propozițiilor.

Saltul decisiv a venit cu arhitectura transformer [8], care a introdus mecanismul de atenție ca element central. Spre deosebire de arhitecturile anterioare, transformerul procesează toate cuvintele unei secvențe în paralel și învață relațiile dintre ele indiferent de distanță, ceea ce l-a făcut potrivit pentru antrenarea pe corpusuri foarte mari. Pe această bază au fost construite modelele lingvistice mari, antrenate pe cantități uriașe de text pentru a prezice cuvântul următor. Pe măsură ce dimensiunea modelelor și a datelor de antrenare a crescut, capacitățile lor au crescut și ele, dar a devenit tot mai clară o limitare structurală: cunoștințele rămân înghețate la momentul antrenării. Acest lucru a motivat căutarea unor metode prin care un model să poată accesa informație externă, actualizată, la momentul interogării, și a deschis drumul către generarea augmentată prin recuperare.

1.1.2 Cunoștințe statice, halucinații și soluția RAG

Fenomenul de „halucinație” reprezintă un răspuns aparent bine formulat, dar factual incorect, al unui LLM. Modelele generează text pe baza distribuției de probabilități a datelor cu care au fost antrenate, nu pe baza unor cunoștințe verificate. În utilizările cu miză mică, halucinațiile pot fi tolerate, însă în domenii critice precum cel medical, juridic sau financiar ele sunt inacceptabile. În plus, informația încorporată în parametrii modelului este de natură „statică”, reflectă cunoștințele cunoscute la momentul antrenării. După antrenare, singura metodă de actualizare a cunoștințelor parametrice este procesul costisitor de *fine-tuning*.

Retrieval-Augmented Generation (RAG) este un tip de arhitectură de sistem de inteligență artificială care combină recuperarea informației relevante din surse externe cu capacitatea de generare a LLM-urilor. În contextul unui sistem RAG, documentele relevante sunt recuperate

Evoluția Exponențială a Cercetării în Domeniul LLM (2017 - 2025)
Sursă Date: API arXiv (Interogare Automată)

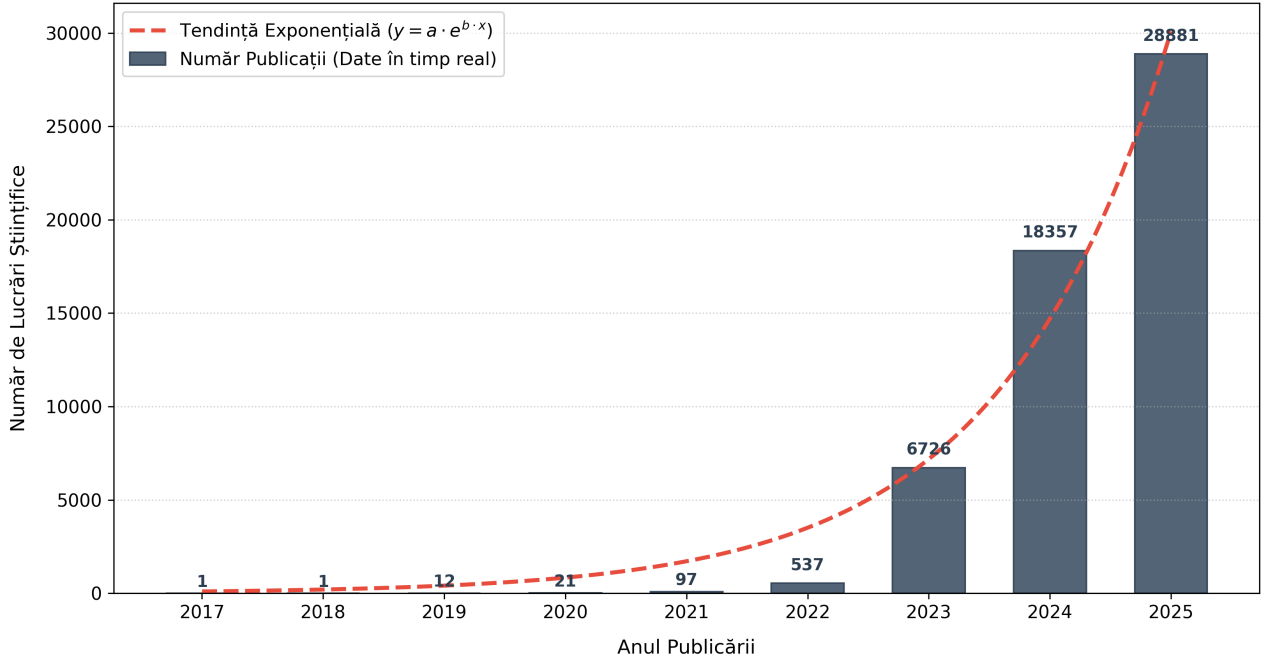


Figura 1.1: Creșterea numărului de lucrări despre modele lingvistice mari (LLMs) publicate pe arXiv, 2017–2025.

dintr-o sursă externă de date, încorporate în contextul LLM-ului, iar în final, se generează un răspuns. Aceasta adresează ambele probleme simultan: furnizează informația necesară unui context factual corect, reducând astfel halucinațiile, și permite modificarea cunoștințelor și extinderea contextului fără *fine-tuning* sau reantrenare. Termenul „RAG” a fost introdus în 2020 de către Patrick Lewis și colaboratorii săi de la Facebook AI Research (astăzi Meta AI) în lucrarea „*Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*” [1].

Ideea de bază a RAG este simplă, dar puternică: separă cunoștințele de capacitatea de raționament. Modelul lingvistic rămâne responsabil de înțelegerea limbajului și de formularea răspunsului, în timp ce cunoștințele factuale sunt mutate într-o sursă externă, ușor de inspectat și de actualizat. Această separare aduce și un beneficiu de transparență: fiecare răspuns poate fi însoțit de fragmentele pe care s-a bazat, ceea ce permite utilizatorului să verifice sursa. Spre deosebire de un model care răspunde dintr-o cunoaștere opacă, un sistem RAG face vizibil pe ce se sprijină răspunsul, un aspect important în aplicațiile unde corectitudinea trebuie justificată.

Formal, fie q o interogare și $\mathcal{D} = \{d_1, d_2, \dots, d_N\}$ o mulțime de documente. Un sistem RAG acționează în două etape. Prima etapă: un model de recuperare $p_\eta(\cdot | q)$ alege o submulțime $\mathcal{Z} \subseteq \mathcal{D}$ de documente-candidat. A doua etapă: un model lingvistic mare generativ p_θ generează răspunsul y condiționat de interogare, dar și de documentele recuperate:

$$p(y | q) = \sum_{z \in \mathcal{Z}} p_\eta(z | q) p_\theta(y | q, z). \quad (1.1)$$

Această formulare arată că răspunsul nu depinde doar de model, ci și de documentele recuperate: o recuperare mai bună schimbă distribuția peste $\mathcal{Z}$ și, implicit, calitatea răspunsului [1].

1.1.3 RAG versus *fine-tuning*

Pentru a adapta un model de limbaj la cunoștințe noi există două abordări principale: *fine-tuning*-ul și RAG. *Fine-tuning*-ul continuă antrenarea modelului pe date suplimentare, încorporând noile cunoștințe direct în parametri. RAG lasă parametrii neschimbați și furnizează cunoștințele la momentul interogării, prin recuperare externă.

Cele două abordări rezolvă probleme diferite. *Fine-tuning*-ul este potrivit pentru a învăța modelul un stil, un format de răspuns sau un domeniu de specialitate, deoarece modifică modul în care modelul generează text. RAG este potrivit pentru a furniza fapte care se schimbă în timp sau care sunt prea numeroase pentru a fi memorate, deoarece baza de cunoștințe poate fi actualizată fără a atinge modelul [5]. Adăugarea unui document nou face imediat disponibile informațiile pe care le conține, fără reantrenare, un avantaj important pentru domeniile în care cunoștințele se schimbă des.

Cele două abordări nu se exclud. Un sistem poate folosi un model cu *fine-tuning* pe domeniul țintă, alimentat în același timp cu dovezi recuperate prin RAG. În practică, RAG este adesea preferat ca primă soluție, deoarece costul actualizării bazei de cunoștințe este mult mai mic decât costul reantrenării modelului.

1.2 Formularea problemei: Extragerea relațiilor între entități

Dacă RAG este mijlocul tehnic al acestei lucrări, extragerea relațiilor între entități este scopul ei. Cele două sunt domenii distincte: extragerea relațiilor este o problemă clasică a procesării limbajului natural, studiată cu mult înainte de modelele lingvistice mari, în timp ce RAG este o arhitectură recentă care poate fi aplicată acestei probleme. Această secțiune definește sarcina, arată cum sunt reprezentate relațiile, trece în revistă metodele clasice și limitările lor, apoi explică în ce situații RAG devine o opțiune atractivă.

1.2.1 Definiție și formalizare

Extragerea relațiilor între entități (eng. „*relation extraction*”) este sarcina de a identifica, într-un text, legăturile semantice dintre entitățile menționate. Este o componentă clasică a extragerii informației (eng. „*information extraction*”), studiată pe linia sistemelor din anii 1990, cu mult înainte de modelele lingvistice mari [9]. Într-un flux tipic de extragere a informației, ea urmează după recunoașterea entităților numite (eng. „*named entity recognition*”), care localizează și etichetează entitățile, și după rezolvarea coreferinței, care leagă mențiunile aceleiași entități; abia apoi extragerea relațiilor stabilește cum se leagă entitățile între ele [4].

Formal, dat un text și un set de entități menționate, o instanță de relație este un triplet (e_h, r, e_t) , unde e_h este entitatea-cap (subiect), e_t este entitatea-coadă (obiect), iar r aparține unui set de tipuri de relații $\mathcal{R}$ (de exemplu „loc de naștere” sau „țară de cetățenie”). Relațiile sunt de regulă binare și direcționate: tripletul (e_h, r, e_t) nu este echivalent cu (e_t, r, e_h) . Din propoziția „Albert Einstein s-a născut în Ulm în 1879” se extrag, de exemplu, tripletele (Einstein, loc de naștere, Ulm) și (Einstein, an de naștere, 1879).

În practică, sarcina apare în mai multe formulări. Clasificarea relației (eng. „*relation classification*”) primește o pereche de entități și cere atribuirea unui tip din $\mathcal{R}$, inclusiv o clasă specială „fără relație” pentru perechile nelegate. Extragerea tripletelor (eng. „*relation triple extraction*”) cere identificarea simultană a entităților și a relațiilor dintr-un text, fără perechi date dinainte. Completarea unui câmp lipsă (eng. „*slot filling*”) pornește de la o entitate-cap și un tip de relație și cere găsirea entității-coadă [10]. Sarcina se împarte și după granularitate: la nivel de propoziție, unde ambele entități apar în aceeași frază, și la nivel de document, unde relația trebuie dedusă din dovezi împrăștiate în mai multe propoziții, ceea ce cere raționament

inter-propoziție [4]. O distincție suplimentară separă extragerea cu schemă închisă, limitată la un set fix $\mathcal{R}$, de extragerea deschisă, în care relația este exprimată direct prin cuvintele textului.

1.2.2 Reprezentarea relațiilor: triplete și grafuri de cunoștințe

Rezultatul extragerii este aproape întotdeauna reprezentat ca triplete, iar mulțimea lor formează un graf de cunoștințe (eng. „*knowledge graph*”). Formal, un graf de cunoștințe este o submulțime $\mathcal{G} \subseteq \mathcal{E} \times \mathcal{R} \times \mathcal{E}$ de triplete peste o mulțime de entități $\mathcal{E}$ și un set de relații $\mathcal{R}$, adică un graf orientat în care nodurile sunt entități, iar muchiile etichetate sunt relațiile dintre ele. Baze de cunoștințe mari precum Wikidata, DBpedia și Freebase sunt exemple de astfel de grafuri, iar extragerea automată a relațiilor din text este una dintre metodele prin care asemenea baze pot fi populate și extinse [11].

Această reprezentare explică ce anume se extrage: nu text liber, ci o muchie tipizată între două noduri, care poate fi stocată, interogată și verificată. Standardul RDF (Resource Description Framework) formalizează tripletele ca afirmații (subiect, predicat, obiect), iar seturi precum WebNLG leagă tripletele RDF de exprimările lor în limbaj natural [12]. Peste acest nivel, o ontologie, exprimată de exemplu prin standardele RDFS (Resource Description Framework Schema) sau OWL (Web Ontology Language), impune o schemă: fiecare tip de relație are un domeniu și un codomeniu, adică entitatea-cap și cea-coadă trebuie să fie de anumite tipuri, iar entitățile sunt organizate în clase. În Wikidata, de pildă, entitățile au identificatori canonici unici (cunoscuți sub denumirea de QID, de la Q-identifier, reprezentând elementele din baza de cunoștințe), iar relațiile sunt descrise prin proprietăți specifice (identificatori de tip PID, de la Property Identifier), ceea ce face tripletele lipsite de ambiguitate, independent de forma de suprafață din text.

Două aspecte complică reprezentarea. Primul este normalizarea entităților: aceeași entitate poate apărea sub forme de suprafață diferite („Einstein”, „Albert Einstein”, „A. Einstein”). Din acest motiv, legarea mențiunilor la un singur identificator canonic (eng. „*entity linking*”) devine un pas necesar pentru a evita nodurile duplicate în graf. Al doilea aspect este reprezentarea relațiilor cu n argumente sau a celor cu calificatori. De exemplu, o relație de tipul „a ocupat funcția” poate fi însoțită de un interval de timp. Deoarece tripletele binare nu pot exprima direct aceste detalii, grafurile de cunoștințe le tratează prin reificare, adică prin introducerea unui nod intermediar care leagă individual toate argumentele.

1.2.3 Metode clasice și limitările lor

Înainte de modelele lingvistice mari, extragerea relațiilor a fost abordată prin câteva familii de metode, fiecare cu compromisuri proprii. Metodele bazate pe reguli și tipare lexico-sintactice identifică relații prin șabloane scrise manual peste text sau peste arborele de dependențe, de tipul tiparelor Hearst [9]. Au precizie mare, dar acoperire mică: prind doar formulările prevăzute explicit, sunt fragile la variațiile de exprimare și cer efort manual pentru fiecare relație și domeniu.

A doua familie tratează extragerea ca o clasificare supervizată: pentru o pereche de entități dintr-o propoziție, un clasificator alege tipul relației dintr-un set fix, plus o clasă „fără relație”. Abordările timpurii combinau trăsături lexicale, sintactice și de dependențe cu clasificatoare precum SVM (*Support Vector Machine*) sau cu metode cu nuclee [13], iar cele neuronale ulterioare au înlocuit trăsăturile construite manual cu reprezentări învățate automat. Pe seturi precum TACRED (*TAC Relation Extraction Dataset*), cu 41 de tipuri de relații, aceste modele ating performanțe bune, dar limitarea comună rămâne dependența de date adnotate pentru fiecare tip de relație și de o schemă fixă: un model antrenat pe 41 de relații nu poate extrage o relație nouă fără reantrenare [14].

Costul adnotării a motivat supervizarea la distanță (eng. „*distant supervision*”), care alinază automat tripletele dintr-o bază de cunoștințe existentă cu propozițiile ce conțin aceleași două entități, generând date de antrenare fără adnotare manuală [15]. Ipoteza este însă prea tare: nu orice propoziție care conține două entități exprimă și relația dintre ele, ceea ce introduce etichete zgomotoase.

O direcție diferită este extragerea deschisă a informației (eng. „*Open Information Extraction*”), care nu pornește de la o schemă fixă, ci extrage triplete cu relații exprimate prin chiar cuvintele din text [16]. Avantajul este acoperirea relațiilor arbitrare, fără adnotare; dezavantajul este că relațiile rezultate sunt fraze de suprafață nenormalizate, greu de aliniat la o bază de cunoștințe.

Dincolo de specificul fiecărei familii, metodele clasice împărtășesc câteva limitări structurale. Funcționează cel mai bine la nivel de propoziție și se descurcă slab când relația cere agregarea dovezilor din mai multe propoziții sau documente [4]. Sunt organizate ca un flux în care erorile de la recunoașterea entităților se propagă la extragerea relațiilor. Metodele clasice depind de o schemă fixă și de date adnotate, ceea ce le face costisitoare de extins către relații noi sau domenii noi, și se comportă slab pe relațiile rare, din coada distribuției.

1.2.4 De ce RAG pentru extragerea relațiilor

Numitorul comun al limitărilor de mai sus este rigiditatea: metodele clasice fie cer efort manual, fie depind de o schemă fixă și de date adnotate, fie produc rezultate zgomotoase sau nenormalizate. RAG oferă o cale de a ocoli o parte din ele, mutând cunoștințele într-o sursă externă și lăsând modelul să raționeze peste dovezile recuperate.

Cheia este reformularea sarcinii. În loc să antreneze un clasificator pentru o schemă fixă, un sistem RAG poate trata extragerea unei relații ca pe o întrebare: un tip de relație este transformat într-un șablon („Care este relația entității cap?”), pentru care se recuperează pasajele relevante, iar un model generativ produce entitatea-coadă din contextul recuperat [10]. Formulată ca citire (eng. „*reading comprehension*”), o relație nouă devine pur și simplu o întrebare nouă, ceea ce permite extragerea în regim zero-shot, fără reantrenare pentru fiecare relație [10]. Aceeași logică permite abordarea extragerii de relații ca o sarcină intensivă în cunoștințe (knowledge-intensive), alături de răspunsul la întrebări, unde componenta de recuperare aduce dovezile necesare din afara parametrilor modelului [1].

Avantajele se văd în câteva situații concrete. Când schema de relații este deschisă sau se schimbă des, RAG adaugă o relație prin actualizarea bazei de cunoștințe, nu prin reantrenare. Când dovada unei relații este împrăștiată în mai multe propoziții sau documente, recuperarea poate aduna pasajele relevante înainte de extragere, ceea ce ajută direct la relațiile la nivel de document [4]. Și, fiindcă fiecare relație extrasă poate fi urmărită până la fragmentul care o susține, sistemul oferă proveniență (eng. „*provenance*”) și devine auditabil, un avantaj important în domenii precum medicina sau dreptul.

RAG nu înlocuiește însă metodele clasice peste tot, iar o comparație onestă cere precizarea limitelor. Pentru o schemă fixă, cu multe date adnotate, un clasificator supervizat dedicat poate fi mai precis și mult mai ieftin la inferență, fără latența și costul unui model generativ. Calitatea extragerii depinde critic de calitatea recuperării: dacă pasajul potrivit nu este recuperat, modelul fie refuză, fie inventează o relație, iar generarea introduce un risc de halucinație pe care clasificatoarele nu îl au. Când relațiile sunt dense și explicit structurate, abordările care indexează direct legăturile dintre entități, precum GraphRAG și HippoRAG, pot fi mai potrivite decât recuperarea pe fragmente de text, deoarece exploatează structura de graf în loc să o reconstruiască din pasaje [2, 17]. RAG este, prin urmare, o opțiune atractivă mai ales pentru scheme deschise, relații rare și raționament la nivel de document, și mai puțin pentru sarcini fixe, bine delimitate, deja rezolvate eficient de metodele clasice.

Extragerea relațiilor între entități, fie prin metode clasice, fie prin RAG, susține o gamă largă de aplicații în care informația structurată este mai utilă decât textul liber [5]. Cea mai directă este construirea grafurilor de cunoștințe: baze precum Wikidata, DBpedia sau Freebase pot fi populate și extinse prin relații extrase automat din text [11]. În domenii specializate, precum bioinformatica (relații proteină-proteină), medicina (relații medicament-boală) sau dreptul (legături între spețe și precedente), extragerea relațiilor din literatura de specialitate accelerează accesul la cunoaștere. Asistenții conversaționali și agenții au nevoie de fapte actualizate și structurate, nu doar de text recuperat, iar motoarele de căutare moderne livrează răspunsuri directe sprijinite pe relații și fapte extrase, nu doar liste de linkuri.

Firul comun al acestor aplicații este transformarea textului nestructurat în fapte structurate, ușor de stocat, interogat și verificat. În toate aceste contexte, calitatea extragerii relațiilor determină direct utilitatea cunoștințelor obținute.

1.3 Taxonomie

1.3.1 Motivația definirii unei taxonomii

Domeniul RAG s-a dezvoltat rapid, producând numeroase variante și îmbunătățiri ale arhitecturii originale. Identificarea similarităților și diferențelor între abordări este dificilă fără o clasificare sistematică. Lucrările Gao et al. (2024) [5] și Peng et al. (2025) [18] au fost folosite ca referință pentru această secțiune, care introduce o taxonomie bazată pe aspectele tehnologice cheie. Trebuie reținut că taxonomia este limitată la specificul acestei lucrări (RAG pentru extragerea relațiilor) și nu acoperă întreg domeniul.

O taxonomie are două roluri practice. Primul este de a oferi un vocabular comun: atunci când o metodă este descrisă ca „recuperare hibridă cu reordonare post-recuperare”, criteriile taxonomiei spun imediat unde se plasează în spațiul soluțiilor. Al doilea este de a ghida deciziile de proiectare: alegerea unei arhitecturi pentru o aplicație concretă revine la alegerea unei opțiuni pe fiecare criteriu, în funcție de tipul datelor, de latența acceptabilă și de natura sarcinii. Pentru această lucrare, taxonomia justifică direct cele două arhitecturi comparate în această lucrare, Naive și Advanced RAG.

1.3.2 Criterii de clasificare

Tipul mecanismului de recuperare

Primul criteriu privește modul în care sistemul găsește documentele relevante, iar distincția fundamentală este între potrivirea lexicală, pe cuvinte, și cea semantică, pe sens. Recuperarea rară aplică o potrivire ponderată de termeni, cu BM25 (*Best Matching 25*) ca standard al domeniului [19], și captează potrivirea lexicală exactă: este rapidă și robustă pe termeni rari, numere și denumiri proprii, dar sensibilă la sinonime și la reformulări, fiindcă nu modelează sensul. Recuperarea densă codifică interogarea și fragmentele în vectori într-un spațiu continuu și le compară prin similaritate cosinus [20], captând asemănarea de sens; prinde sinonimia și parafraza, dar poate dilua termenii rari și are un cost de calcul mai mare. Recuperarea hibridă combină cele două scoruri, de regulă prin Reciprocal Rank Fusion [21], ca să beneficieze de ambele tipuri de potrivire. La aceste mecanisme se poate adăuga un model de reordonare de tip *cross-encoder*, care revaluează fiecare pereche interogare-fragment cu o precizie mai mare decât similaritatea vectorială, un pas deosebit de util pentru extragerea relațiilor, unde fragmentul corect trebuie să conțină simultan ambele entități și formularea relației dintre ele.

Modul de integrare între recuperare și generare

Al doilea criteriu privește cât de strâns sunt legate cele două componente. Într-un sistem

modular, recuperatorul și generatorul sunt decuplate: funcționează independent și pot fi optimizate sau înlocuite separat, ceea ce oferă flexibilitate și ușurează depanarea. Într-un sistem end-to-end, recuperarea și generarea sunt cuplate și optimizate împreună, de regulă prin antrenare cu coborâre de gradient, ceea ce poate aduce performanțe mai bune pe o sarcină dată, dar face sistemul mai greu de modificat pe bucăți. Sistemele comparate în această lucrare sunt modulare tocmai pentru că această decuplare permite izolarea și măsurarea efectului fiecărei etape de recuperare.

Tipul sursei de cunoștințe

Al treilea criteriu privește forma datelor din care se recuperează. Sursele nestructurate sunt documente în limbaj natural, precum articole Wikipedia, PDF-uri sau pagini web; sunt abundente și ușor de adunat, dar trebuie indexate semantic, iar relațiile dintre entități rămân implicite în text. Sursele structurate sunt grafuri de cunoștințe cu entități și relații modelate explicit, precum Wikidata, DBpedia sau Freebase, care permit navigarea directă a legăturilor dintre entități; această proprietate le face deosebit de potrivite pentru extragerea relațiilor, deoarece relația căutată este reprezentată explicit, nu dedusă din formulări de suprafață. Sursele hibride combină textul liber cu un graf de cunoștințe, exploatând complementaritatea celor două: textul oferă acoperire, iar graful oferă structură.

Etapa augmentării

Al patrulea criteriu privește momentul în care se intervine asupra procesului pentru a-i îmbunătăți calitatea. Optimizările pre-recuperare acționează asupra interogării înainte de căutare, prin reformularea sau extinderea ei și prin descompunerea întrebărilor complexe, ca să crească șansa de a regăsi fragmentul potrivit. Optimizările la momentul recuperării reglează însuși procesul de căutare, prin alegerea numărului de fragmente returnate sau prin strategii adaptive care decid cât și de unde să se recupereze. Optimizările post-recuperare prelucrează fragmentele deja găsite, prin reordonare cu *cross-encoder*, compactarea contextului și filtrarea celor irelevante, înainte ca acestea să ajungă la modelul generativ. Un sistem avansat le combină de obicei pe toate trei, iar varianta Advanced RAG comparată în această lucrare folosește optimizări pre-recuperare, prin reformularea interogării, și post-recuperare, prin reordonare.

1.3.3 Categoriile principale

Pe baza criteriilor de mai sus, literatura distinge patru categorii principale de arhitecturi RAG, ordonate aici de la cea mai simplă la cea mai complexă. Ele nu sunt etape obligatorii, ci puncte pe un spectru: un sistem real poate combina elemente din mai multe categorii.

Naive RAG

Paradigma originală din lucrarea Lewis et al. 2020 [1]: indexare, urmată de recuperare densă și generare. Recuperatorul și generatorul sunt elemente separate, și nu se folosesc optimizări. Este arhitectura de bază, folosită ca referință în majoritatea studiilor, incluzând această teză.

Advanced RAG

Adaugă optimizări pre- și post-recuperare, precum: rescriere a interogărilor, recuperare hibridă, și reordonare cu model special antrenat. Dovedește îmbunătățiri semnificative față de varianta de bază pe teste de referință standard, cu costuri computaționale moderate. Spre deosebire de Modular RAG, păstrează structura liniară a fluxului Naive, dar inserează etape de optimizare înainte și după recuperare. Acest echilibru între simplitate și performanță îl face o alegere practică pentru multe aplicații: aduce o parte importantă din beneficiile arhitecturilor complexe, fără complexitatea lor de implementare. Aceasta este arhitectura comparată în această lucrare cu varianta de bază.

Modular RAG

Diferă de metodele Naive și Advanced în structură, fiecare element din arhitectura Modular

RAG este un modul separat (ex. recuperare, memorare, rutare, predicție, căutare). Modulele pot fi rearanjate, înlocuite sau apelate în bucle, ceea ce permite fluxuri de lucru complexe, precum recuperarea iterativă sau rutarea interogării către surse diferite în funcție de tip. Este cea mai adaptabilă arhitectură, însă introduce și cel mai înalt nivel de complexitate, atât la implementare, cât și la depanare.

Graph RAG

Specifică pentru surse structurate de date: indexare pe grafuri de cunoștințe, recuperare utilizând relații semantice, generare pe bază de subgrafuri. În loc să caute fragmente de text izolate, Graph RAG navighează legăturile dintre entități, ceea ce îi permite să răspundă la întrebări care presupun coroborarea mai multor relații. Este ideal pentru extragerea relațiilor din grafuri de cunoștințe, dar necesită structurarea prealabilă a datelor sub formă de graf, un pas costisitor. Această categorie este descrisă în detaliu de Peng et al. (2025) [18].

Figura 1.2 sintetizează taxonomia adoptată sub forma unei hărți conceptuale, organizată pe cele patru criterii de clasificare descrise mai sus. Cele patru categorii principale (Naive, Advanced, Modular și Graph RAG) se diferențiază prin combinațiile de opțiuni alese pe fiecare criteriu. Această lucrare compară direct două dintre aceste categorii, Naive RAG (recuperare densă, fără optimizări) și Advanced RAG (recuperare hibridă, optimizări pre- și post-recuperare), pentru a măsura efectul optimizărilor de recuperare.

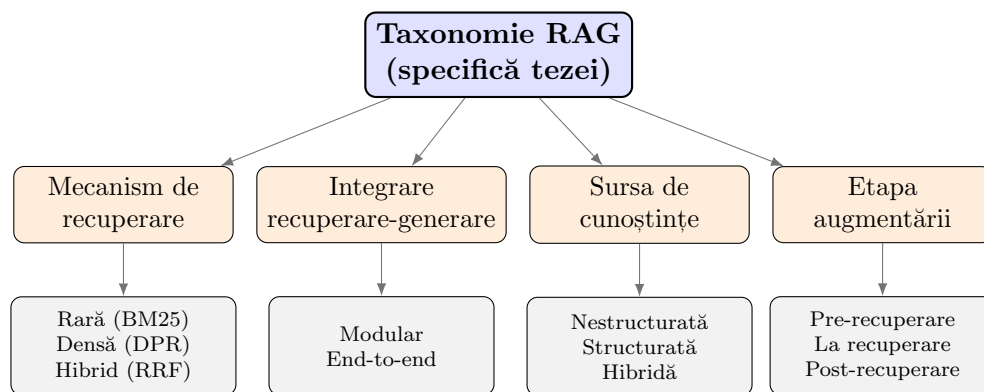


Figura 1.2: Harta conceptuală a taxonomiei adoptate, structurată pe cele patru criterii relevante pentru extragerea relațiilor între entități [5, 18].

1.4 Arhitectura generală a unui sistem de tip RAG

Un sistem RAG funcționează în două faze: o fază offline de indexare și o fază online de inferență. Componentele principale sunt interogarea utilizatorului, baza de cunoștințe, mecanismul de recuperare și modelul generativ [5].

1.4.1 Componentele sistemului

Interogarea reprezintă nevoia de informație a utilizatorului și este punctul de pornire al fluxului online. Rolul ei este dublu: pe de o parte declanșează recuperarea, pe de altă parte stabilește criteriul față de care se măsoară relevanța fragmentelor. În formele de bază, textul interogării este transmis direct mecanismului de recuperare, fără prelucrare. Sistemele avansate aplică mai întâi transformări ale interogării: reformulare semantică, extindere cu termeni sinonimi sau descompunere în sub-interogări, pentru a crește acoperirea și a reduce sensibilitatea față de formulări ambigue [5]. Pentru extragerea relațiilor, formularea interogării contează mult,

deoarece aceeași relație poate fi exprimată în multe feluri, iar o singură formulare poate rata documentele care folosesc alți termeni.

Baza de cunoștințe este sursa externă din care se extrage informația și determină, prin conținutul ei, ce poate și ce nu poate ști sistemul. Spre deosebire de cunoștințele parametrice ale modelului, baza de cunoștințe poate fi actualizată oricând, fără reantrenare. Documentele trec printr-o etapă de indexare offline: sunt împărțite în fragmente (eng. „*chunks*”), codificate cu un model de reprezentare vectorială bazat pe arhitectura transformer [8] și stocate într-o bază de date vectorială [20]. Strategia de fragmentare influențează direct calitatea recuperării: fragmentele prea scurte pierd contextul necesar pentru identificarea relațiilor între entități, iar cele prea lungi diluează semnalul relevant. Calitatea bazei de cunoștințe limitează performanța întregului sistem, deoarece o relație care nu apare în niciun document nu poate fi recuperată, oricât de bun ar fi mecanismul de recuperare.

Mecanismul de recuperare punctează fragmentele din baza vectorială în raport cu interogarea și returnează primele k rezultate. Rolul lui este de a aduce în fața modelului generativ exact dovezile necesare, nici prea puține, încât să rateze informația, nici prea multe, încât să o îngroape în zgomot. Tipul de recuperare folosit, densă, rară sau hibridă, împreună cu eventuala reordonare cu *cross-encoder*, a fost prezentat în taxonomia din Secțiunea 1.3; alegerea dintre aceste mecanisme este una dintre deciziile centrale de proiectare, cu efect direct asupra calității dovezilor recuperate.

Modelul generativ primește contextul augmentat, format din interogarea originală și fragmentele recuperate, și produce răspunsul final [1]. Rolul lui nu este de a ști răspunsul, ci de a-l sintetiza din dovezile furnizate: identifică în fragmente informația relevantă, o leagă de întrebare și o exprimă coerent. În cazul extragerii relațiilor, modelul localizează în context legătura dintre entități și o formulează ca răspuns. Arhitecturile orientate spre fidelitate constrâng explicit modelul să fundamenteze răspunsurile pe dovezile recuperate, reducând astfel rata de halucinații [22]. Constrângerea se impune de obicei prin promptul de sistem, care instruieste modelul să răspundă doar din context și să semnaleze explicit absența informației, în loc să completeze din cunoștințele sale interne.

1.4.2 Fluxul de date

Figura 1.3 ilustrează cele două trasee ale fluxului de date. Offline, documentele sunt procesate o singură dată: fragmentate, codificate și stocate în baza vectorială. Online, fiecare interogare parcurge procesarea interogării, recuperarea fragmentelor, augmentarea contextului și generarea răspunsului.

Separarea în două faze are o motivație practică. Indexarea este o operație costisitoare, deoarece presupune codificarea întregului corpus, dar trebuie făcută o singură dată și poate fi refolosită pentru orice număr de interogări. Inferența, în schimb, trebuie să fie rapidă, deoarece se execută la fiecare cerere a utilizatorului. De aceea merită să optimizăm indexarea offline, chiar dacă durează mult, pentru ca recuperarea online să fie cât mai rapidă. Atunci când corpusul se schimbă, doar documentele noi trebuie codificate și adăugate, fără a reface întregul index.

Un exemplu concret clarifică rolul fiecărei componente. Să presupunem interogarea „Unde s-a născut Marie Curie?”, pentru care răspunsul corect este tripletul (Marie Curie, loc de naștere, Varșovia). În faza online, interogarea este mai întâi codificată într-un vector. Mecanismul de recuperare caută în baza vectorială fragmentele cele mai apropiate semantic și returnează, de exemplu, un pasaj din articolul Wikipedia despre Marie Curie care conține propoziția „Maria Skłodowska s-a născut la Varșovia în 1867”. Acest fragment este introdus în contextul modelului generativ împreună cu interogarea originală. Modelul generativ identifică în fragment legătura dintre entitatea-subiect și locul de naștere și produce răspunsul fundamentat pe dovada

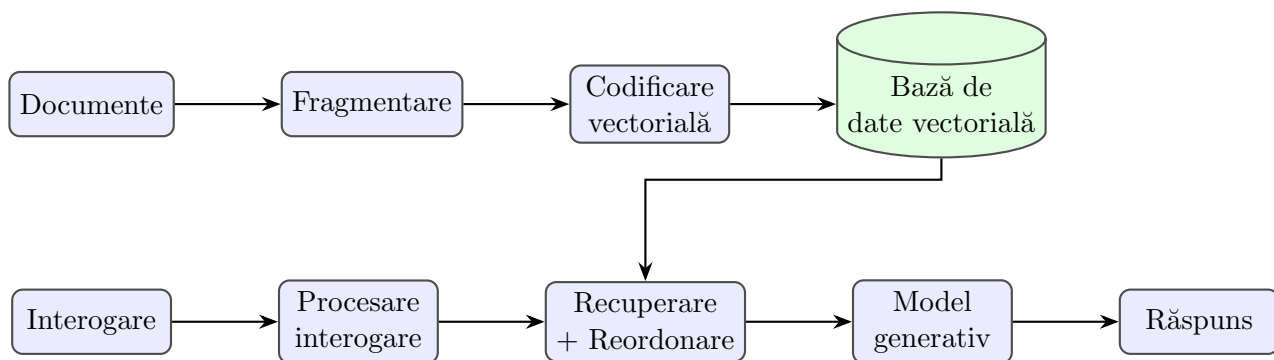


Figura 1.3: Arhitectura generală a unui sistem RAG: indexare offline (sus) și inferență online (jos) [5, 20].

recuperată, nu pe cunoștințele sale interne.

Acest exemplu evidențiază două aspecte importante pentru extragerea relațiilor. Primul: recuperarea trebuie să aducă un fragment care conține simultan entitatea și indicele lexical al relației, altfel modelul nu are pe ce să se bazeze. Al doilea: variația lexicală (interogarea spune „Marie Curie”, iar textul „Maria Skłodowska”) poate face recuperarea densă să rateze fragmentul corect, problemă pe care optimizările din Advanced RAG, precum reformularea interogării și recuperarea hibridă, o atenuează.

1.4.3 Indexarea: fragmentare, codificare și stocare vectorială

Fragmentarea (eng. „*chunking*”) este pasul prin care documentele sunt împărțite în unități indexabile, iar alegerea strategiei influențează direct calitatea recuperării. Există câteva abordări uzuale. Fragmentarea de lungime fixă împarte textul în bucăți de un număr fix de tokenuri sau caractere, eventual cu o suprapunere între fragmente consecutive pentru a nu pierde contextul de la graniță. Este simplă și rapidă, dar poate tăia o propoziție sau o relație în două. Fragmentarea pe granițe naturale, la nivel de propoziție sau paragraf, respectă structura textului și păstrează relațiile intacte în interiorul unui fragment, dar produce fragmente de lungimi inegale. Fragmentarea semantică grupează propozițiile înrudite pe baza similarității de reprezentare, încercând să mențină împreună informația coerentă tematic, dar cu un cost de calcul mai mare la indexare.

Compromisul fundamental este între specificitate și context [5]. Fragmentele scurte sunt specifice, dar pot pierde contextul necesar pentru a identifica o relație care se întinde pe mai multe propoziții. Fragmentele lungi păstrează contextul, dar diluează semnalul relevant și introduc zgomot în promptul modelului generativ. Pentru extragerea relațiilor, fragmentul ideal conține ambele entități și formularea relației dintre ele, ceea ce favorizează fragmentarea pe granițe naturale, la nivel de paragraf.

Odată ce textul este fragmentat, pasul următor al indexării este codificarea. Calitatea recuperării dense depinde de modelul de codificare, care transformă textul într-un vector numeric. Modelele de codificare moderne sunt encodere bazate pe arhitectura transformer [8], antrenate astfel încât textele apropiate ca sens să aibă vectori apropiați în spațiul de reprezentare. Alegerea modelului implică un compromis între calitate și viteză: modelele mari produc reprezentări mai bune, dar sunt lente și costisitoare, în timp ce modelele compacte precum familia MiniLM oferă un raport bun între acuratețe și cost, motiv pentru care sunt des folosite în sistemele RAG fără constrângeri hardware mari.

Vectorii rezultați sunt stocați într-o bază de date vectorială, care permite căutarea rapidă a celor mai apropiați vecini. Căutarea exactă a vecinilor într-un corpus mare ar fi prea lentă, așa că aceste baze de date folosesc algoritmi de căutare aproximativă a vecinilor (eng. „*approximate*

nearest neighbor”), de tipul grafurilor HNSW (*Hierarchical Navigable Small World*) [23], care găsesc rapid vectori apropiați cu o pierdere mică de acuratețe. Soluții uzuale sunt FAISS (*Facebook AI Similarity Search*) [24], ChromaDB și Milvus [25].

1.4.4 Construcția promptului și augmentarea contextului

După recuperare, fragmentele selectate trebuie integrate în promptul modelului generativ. Acest pas, deși adesea trecut cu vederea, influențează direct calitatea răspunsului. Un prompt RAG tipic are trei părți: o instrucțiune de sistem care stabilește comportamentul modelului, contextul format din fragmentele recuperate și interogarea utilizatorului.

Instrucțiunea de sistem este locul în care se impune constrângerea de fundamentare (eng. „*grounding*”): modelul este instruit să răspundă exclusiv pe baza contextului furnizat și să semnaleze explicit când informația lipsește, în loc să completeze din cunoștințele sale interne. Această constrângere reduce halucinațiile și este caracteristică sistemelor orientate spre fidelitate [22]. Ordinea fragmentelor în context contează și ea: modelele tind să acorde mai multă atenție începutului și sfârșitului unui context lung, fenomen care motivează plasarea fragmentelor cele mai relevante pe pozițiile de capăt [26].

Pentru extragerea relațiilor, modul de construire a promptului determină dacă modelul are simultan în față ambele entități și formularea relației. Un context bine construit, în care fragmentele relevante nu sunt diluate de pasaje de umplură, crește șansa ca relația corectă să fie identificată și exprimată în răspuns.

1.4.5 Compromisuri în proiectarea unui sistem RAG

Proiectarea unui sistem RAG presupune o serie de decizii între care există compromisuri inevitabile. Niciuna nu are un răspuns universal corect; alegerea depinde de aplicație, de date și de resursele disponibile. Înțelegerea acestor compromisuri ajută la interpretarea rezultatelor din partea experimentală a lucrării.

Primul compromis privește dimensiunea fragmentelor. Fragmentele mici sunt specifice și aduc puțin zgomot, dar riscă să piardă contextul necesar pentru identificarea unei relații care se întinde pe mai multe propoziții. Fragmentele mari păstrează contextul, dar diluează semnalul și consumă mai mult din spațiul limitat al promptului. Nu există o dimensiune optimă universală; ea depinde de structura textului și de tipul întrebărilor.

Al doilea compromis privește numărul de fragmente recuperate, parametrul k . Un k mic riscă să rateze informația relevantă, în timp ce un k mare introduce pasaje irelevante care pot induce în eroare modelul generativ [26]. Creșterea lui k nu îmbunătățește monoton performanța, ci atinge un punct dincolo de care zgomotul depășește beneficiul acoperirii.

Al treilea compromis privește alegerea între recuperarea densă și cea rară. Recuperarea densă captează sensul, dar ratează termenii rari; recuperarea rară captează potrivirile lexicale exacte, dar nu înțelege sinonimele. Recuperarea hibridă încearcă să le combine, dar cu o complexitate și un cost de calcul mai mari.

Al patrulea compromis, poate cel mai vizibil în practică, este între calitate și latență. Fiecare etapă de optimizare, de la expansiunea interogărilor la reordonare, îmbunătățește calitatea recuperării, dar adaugă timp de procesare. În aplicațiile interactive, unde utilizatorul așteaptă un răspuns, acest compromis devine o constrângere de proiectare, nu doar o opțiune. Partea experimentală a lucrării măsoară direct acest compromis pe cele două arhitecturi comparate.

1.5 Metode și modele RAG

1.5.1 Naive RAG și Advanced RAG

Formularea originală a RAG, propusă de Lewis et al. (2020) [1], combină un recuperator bazat pe Dense Passage Retrieval (DPR) cu un model generativ de tip sequence-to-sequence, optimizate parțial în comun. Fluxul de lucru este liniar: indexare, recuperare densă, generare. Această variantă servește ca referință în majoritatea studiilor din literatura de specialitate [5], inclusiv în cadrul acestei teze. Avantajul principal al Naive RAG este simplitatea: are puține componente, este ușor de implementat și de înțeles, iar costul computațional este redus, deoarece o singură interogare densă este suficientă.

Dezavantajele sunt bine documentate. Sistemul nu gestionează variațiile lexicale (sinonime, parafrazări), deoarece se bazează exclusiv pe similaritatea semantică, nu evaluează calitatea documentelor recuperate și nu filtrează fragmentele irelevante înainte de generare [5]. Pe interogări cu termeni rari sau numerici, recuperarea densă ratează frecvent fragmentul corect, ceea ce limitează direct calitatea răspunsului final.

Advanced RAG adaugă optimizări pre-recuperare și post-recuperare față de varianta de bază [5]. Pre-recuperare: fragmentare semantică, îmbogățirea fragmentelor cu metadate, generare de interogări multiple. Post-recuperare: reordonare cu model *cross-encoder*, compactarea contextului pentru a reduce zgomotul din prompt. Scorurile dense și rare sunt combinate prin RRF (*Reciprocal Rank Fusion*) [21]. Studiile raportează îmbunătățiri consistente față de Naive RAG pe teste de referință standard. Avantajul Advanced RAG este creșterea calității recuperării: recuperarea hibridă prinde atât potrivirile semantice, cât și pe cele lexicale, iar reordonarea așază fragmentele cu adevărat relevante pe primele poziții. Dezavantajul este costul: fiecare optimizare adaugă latență, iar expansiunea interogărilor presupune un apel suplimentar la un model de limbaj, ceea ce poate crește timpul de răspuns de câteva ori față de varianta de bază, după cum arată și partea experimentală a acestei lucrări.

Pentru extragerea relațiilor, optimizările post-recuperare sunt direct relevante: identificarea corectă a unei relații necesită fragmente care conțin simultan ambele mențiuni de entitate și indiciile lexicale ale relației. Reordonarea crește șansa ca aceste fragmente să ajungă în contextul modelului generativ.

1.5.2 Optimizări ale recuperării

Recuperarea densă stă la baza majorității sistemelor RAG moderne, inclusiv a celor comparate în această lucrare. Dense Passage Retrieval (DPR) [20] folosește două encodere separate, unul pentru interogare și unul pentru pasaje, antrenate astfel încât perechile relevante să aibă vectori apropiați în spațiul de reprezentare. La recuperare, interogarea este codificată o singură dată, iar pasajul cu cel mai mare scor de similaritate, calculat ca produs scalar între vectori, este returnat. Avantajul față de metodele lexicale precum BM25 este captarea sinonimiei și a parafrazării: DPR poate lega „loc de naștere” de „s-a născut la” chiar dacă termenii nu coincid. Limitarea este că un singur vector pentru întregul pasaj poate dilua termenii rari sau specifici.

ColBERT (*Contextualized Late Interaction over BERT*) [27] rafinează această idee prin interacțiune târzie (eng. „late interaction”). În loc de un singur vector per pasaj, ColBERT păstrează câte un vector pentru fiecare token și calculează relevanța prin suma similarităților maxime dintre tokenurile interogării și cele ale pasajului. Acest nivel fin de detaliu îmbunătățește precizia, mai ales pe interogări cu termeni specifici, dar crește costul de stocare și de calcul, deoarece indexul trebuie să rețină reprezentări la nivel de token. Pentru extragerea relațiilor, interacțiunea târzie ajută la potrivirea precisă a mențiunilor de entitate, care sunt adesea termeni rari.

O categorie de tehnici pre-recuperare vizează direct interogarea, înainte ca aceasta să ajungă la mecanismul de recuperare. Motivul este că o singură formulare a întrebării poate rata documentele relevante exprimate cu alte cuvinte. Reformularea (eng. „*query rewriting*”) transformă interogarea într-o formă mai potrivită pentru recuperare, de exemplu clarificând un termen ambiguu sau eliminând cuvinte de umplură. Expansiunea cu interogări multiple (eng. „*multi-query*”) generează mai multe variante ale interogării originale, fiecare căutată independent, iar rezultatele sunt apoi combinate. Ideea din spatele expansiunii cu interogări multiple este creșterea acoperirii: dacă una dintre reformulări folosește termenii care apar efectiv în document, fragmentul corect este recuperat chiar dacă formularea originală îl rata. Costul este un număr mai mare de recuperări și, de obicei, un apel suplimentar la un model de limbaj pentru a genera reformulările. Pentru extragerea relațiilor, această tehnică este utilă deoarece aceeași relație poate fi exprimată în multe feluri („s-a născut la”, „locul nașterii”, „originar din”), iar mai multe reformulări cresc șansa de a potrivi formularea din text.

Mai elaborat, în loc de simple reformulări, HyDE [28] generează cu un model de limbaj un pseudo-document care ar conține răspunsul la interogare. Acest pseudo-document este codificat și se caută documente reale similare prin recuperare densă. Encoderele contrastive elimină erorile factuale din pseudo-document, păstrând tiparele de relevanță semantică [28]. Avantajul principal al HyDE este că îmbunătățește recuperarea fără a necesita date de antrenare etichetate pentru interogări. Dezavantajul este dependența de calitatea generării inițiale: un pseudo-document slab generat poate conduce recuperarea în direcția greșită. Pentru extragerea relațiilor, HyDE poate ajuta atunci când interogarea este formulată abstract: pseudo-documentul generat conține adesea formulări concrete ale relației căutate, apropiind recuperarea de fragmentele care exprimă explicit acea relație.

1.5.3 Arhitecturi cu recuperare integrată

Spre deosebire de optimizările modulare de mai sus, unele arhitecturi integrează recuperarea direct în model. REALM (*Retrieval-Augmented Language Model Pre-Training*) [29] integrează recuperarea chiar în etapa de pre-antrenare, tratând recuperatorul ca o variabilă latentă optimizată împreună cu modelul de limbaj. Abordarea arată că un recuperator co-antrenat cu generatorul înțelege mai bine ce tipuri de documente sunt utile pentru diferite sarcini.

Fusion-in-Decoder (FiD) [30] codifică fiecare pasaj recuperat independent prin encoder, dar realizează fuziunea la nivelul decoderului. Astfel, modelul poate agrega dovezi din surse multiple fără a sufoca encoderele cu un context prea lung. RETRO (*Retrieval-Enhanced Transformers*) [31] extinde această idee la scara unei baze de recuperare de ordinul trilioanelor de tokenuri, fără a crește proporțional numărul de parametri ai modelului, obținând eficiență parametrică pe seama unei infrastructuri mai complexe.

Pentru extragerea relațiilor, mecanismul de fuziune din FiD contează mult: o relație poate fi susținută de dovezi distribuite în mai multe pasaje, iar fuziunea la nivelul decoderului permite modelului să combine aceste dovezi fără a fi limitat de lungimea unui singur context. Spre deosebire de concatenarea simplă a fragmentelor, această abordare scalează mai bine cu numărul de pasaje recuperate.

Avantajul comun al acestor trei abordări este integrarea strânsă dintre recuperare și model: recuperatorul nu mai este o componentă separată, ci este antrenat împreună cu generatorul, ceea ce îmbunătățește relevanța documentelor aduse. Dezavantajul comun este costul: REALM și RETRO presupun infrastructură de antrenare complexă și corpusuri foarte mari, iar integrarea profundă face aceste sisteme mai greu de modificat sau de înlocuit pe componente decât arhitecturile modulare.

1.5.4 Recuperare adaptivă și auto-corecție

O altă direcție urmărește ca sistemul însuși să evalueze și să corecteze calitatea recuperării.

CRAG [32] adaugă un evaluator al calității documentelor recuperate. Când relevanța fragmentelor este insuficientă, sistemul declanșează acțiuni corective: recuperare suplimentară sau căutare pe web, urmată de rafinarea cunoștințelor prin descompunere și recompunere a informației. Dacă documentele recuperate sunt de bună calitate, fluxul rămâne cel standard. Mecanismul de auto-corecție face sistemul mai robust față de recuperările greșite și reduce riscul extragerii unei relații dintr-un context irelevant. CRAG este una dintre primele abordări care tratează explicit eșecul recuperării ca un caz de gestionat, nu ca o excepție rară.

Avantajul principal al CRAG este robustețea: sistemul nu se prăbușește atunci când recuperarea inițială eșuează, ci caută alternative. Limitarea este costul suplimentar al evaluatorului și al recuperărilor corective, care pot dubla timpul de răspuns pe interogările dificile. Pentru extragerea relațiilor, capacitatea de a respinge un context irelevant înainte de generare este foarte valoroasă, deoarece o relație extrasă dintr-un fragment greșit este o eroare greu de detectat ulterior.

Self-RAG [22] duce ideea mai departe și antrenează un singur model să decidă adaptiv când să recupereze, când să genereze și când să critice propriile răspunsuri, folosind tokenuri speciale de reflecție introduse în vocabularul modelului. Modelul evaluează relevanța fragmentului recuperat și gradul de fundamentare factuală a răspunsului generat înainte de a-l returna utilizatorului. Față de CRAG, care utilizează un evaluator extern, Self-RAG internalizează întregul proces de evaluare și corecție. Fidelitatea și controlabilitatea cresc [22]. Pentru extragerea relațiilor, posibilitatea de a verifica dacă tipul de relație prezis este susținut de dovezile recuperate este direct utilă, deoarece reduce numărul de relații extrase din fragmente slab relevante.

Avantajul Self-RAG este că evaluarea și generarea folosesc același model, fără componente externe de întreținut. Limitarea este că modelul trebuie antrenat special pentru a produce tokenurile de reflecție, ceea ce presupune date de antrenare dedicate și un cost de pregătire mai mare decât în cazul unui flux de procesare modular. Acest compromis între simplitatea de operare și costul de antrenare este reprezentativ pentru tensiunea dintre arhitecturile integrate și cele modulare discutată în taxonomie.

Dacă CRAG și Self-RAG corectează recuperarea, alte metode reglează decizia de a recupera. Adaptive-RAG [33] alege dinamic strategia de recuperare potrivită pentru fiecare interogare, fără recuperare, cu recuperare unică sau cu recuperare iterativă, pe baza complexității estimate cu un clasificator mic antrenat separat. Interogările simple nu necesită recuperare; cele complexe, care implică mai mulți pași de raționament, beneficiază de recuperare iterativă.

FLARE (*Forward-Looking Active REtrieval*) [34] abordează problema diferit, prin recuperare activă: modelul generează propoziția următoare și, atunci când detectează tokenuri cu probabilitate scăzută (indiciu de incertitudine), folosește predicția respectivă ca interogare și recuperează documente suplimentare înainte de a regenera propoziția. Recuperarea iterativă din Adaptive-RAG ajută la identificarea relațiilor distribuite pe mai multe documente, iar FLARE reduce halucinațiile în extragerea pe texte lungi.

Avantajul ambelor metode este eficiența adaptivă: resursele de recuperare sunt cheltuite doar atunci când este nevoie, evitând recuperările inutile pe interogări simple. Dezavantajul este complexitatea suplimentară și, în cazul FLARE, costul recuperării repetate pe parcursul generării, care poate încetini semnificativ producerea unui răspuns lung. În plus, Adaptive-RAG depinde de calitatea clasificatorului de complexitate: o clasificare greșită trimite o interogare dificilă pe ruta fără recuperare, ratând documentele necesare.

1.5.5 Structuri ierarhice, grafuri și contexte lungi

Un al treilea grup de metode schimbă structura cunoștințelor recuperate, nu doar fluxul de recuperare. RAPTOR (*Recursive Abstractive Processing for Tree-Organized Retrieval*) [35] construiește un arbore de rezumate de jos în sus: fragmentele de text sunt grupate în clustere, rezumate, iar procesul se repetă recursiv, generând niveluri de abstractizare din ce în ce mai înalte. La inferență, recuperarea operează pe acest arbore și poate agrega informații de la mai multe niveluri de abstractizare simultan.

Metoda obține rezultate bune pe sarcini care necesită raționament distribuit pe mai multe documente [35]. Avantajul față de recuperarea standard este că furnizează modelului generativ atât detalii fine, cât și context global. Dezavantajul constă în costul de construire și actualizare a arborelui. Pentru extragerea relațiilor, nivelurile de rezumat ale arborelui pot ajuta la identificarea relațiilor implicite, care nu apar formulat într-o singură propoziție, ci reies din coroborarea mai multor pasaje rezumate la un nivel superior de abstractizare.

Dacă RAPTOR organizează informația ierarhic, abordările pe grafuri o structurează ca rețea de entități. GraphRAG [2] construiește un graf de cunoștințe din corpus prin extragerea entităților și relațiilor, structurând informația în comunități ierarhice rezumate. Recuperarea operează pe acest graf prin traversarea relațiilor semantice, permițând raționamentul global, nu doar căutarea locală de fragmente similare.

HippoRAG [17] integrează un model de limbaj, un graf de cunoștințe și Personalised PageRank: corpusul este transformat într-un graf fără schemă predefinită, iar conceptele din interogare servesc ca noduri de pornire pentru propagarea pe graf. Ambele abordări se înscriu în direcția de cercetare Graph RAG descrisă de Peng et al. (2025) [18]. Avantajul comun este descoperirea relațiilor implicite distribuite în mai multe pasaje, ceea ce le face direct relevante pentru sarcina de extragere a relațiilor. Limitarea comună este costul de construire și actualizare a grafului.

Indiferent de structura aleasă, integrarea contextelor lungi rămâne o provocare comună. Creșterea numărului de pasaje recuperate nu garantează îmbunătățirea performanței. Pasajele irelevante din contexte lungi degradează calitatea generării, un fenomen documentat de Jin et al. (2024) [26]. Problema selectivității devine la fel de importantă ca problema acoperirii. Problema integrării contextelor lungi este foarte relevantă pentru extragerea relațiilor, unde o relație poate fi susținută de dovezi distribuite în mai multe documente. Pe de o parte, sunt necesare mai multe pasaje pentru a acoperi toate dovezile; pe de altă parte, prea multe pasaje irelevante diluează semnalul. Echilibrul dintre acoperire și selectivitate devine astfel o decizie de proiectare centrală, nu un detaliu de reglaj.

Alte extensii documentate în literatura recentă includ RAG multimodal, care integrează documente text, imagini, audio și video [36], MMed-RAG pentru modele vizuale din domeniul medical [37], RAP pentru personalizarea răspunsurilor în funcție de preferințele utilizatorului [38] și abordări de recuperare din colecții de mii de documente [39]. Aceste direcții arată că RAG nu este o singură arhitectură, ci o familie de tehnici care se adaptează la tipul sursei de date și la cerințele aplicației. Pentru extragerea relațiilor din text, varianta relevantă rămâne RAG pe documente text, cu accent pe recuperarea precisă a fragmentelor care conțin entitățile și relația dintre ele.

1.5.6 Sinteză comparativă

Tabelul 1.1 rezumă metodele prezentate, grupându-le după ideea principală și relevanța pentru extragerea relațiilor între entități. Se observă o tendință clară în literatură: de la fluxuri de procesare liniare fixe (Naive RAG) spre sisteme care evaluează și corectează adaptiv calitatea recuperării (CRAG, Self-RAG) sau care structurează explicit cunoștințele sub formă de graf

(GraphRAG, HippoRAG). Pentru extragerea relațiilor, metodele cele mai promițătoare sunt cele care fie recuperează precis termenii rari (recuperare hibridă, ColBERT), fie modelează explicit legăturile dintre entități (abordările pe grafuri).

Metodă	Idee principală	Relevanță pentru extragerea relațiilor
Naive RAG [1]	Recuperare densă + generare	Referință de bază; ratează termenii rari
Advanced RAG [5]	Optimizări pre/post-recuperare	Reordonarea aduce fragmentele cu ambele entități
ColBERT [27]	Interacțiune târzie la nivel de token	Potrivire precisă a mențiunilor de entitate
HyDE [28]	Pseudo-document generat	Îmbunătățește recall fără date etichetate
RAPTOR [35]	Arbore de rezumate ierarhice	Agregare de dovezi multi-document
CRAG [32]	Evaluator al calității recuperării	Reduce extragerea din context irrelevant
Self-RAG [22]	Auto-reflecție prin tokenuri speciale	Verifică susținerea relației de către dovezi
Adaptive-RAG [33]	Strategie de recuperare adaptivă	Recuperare iterativă pentru relații multi-hop
FLARE [34]	Recuperare activă pe parcursul generării	Reduce halucinațiile pe texte lungi
GraphRAG [2]	Graf de cunoștințe din corpus	Modelează explicit relațiile între entități
HippoRAG [17]	Graf + Personalised PageRank	Descoperă relații implicite distribuite

Tabel 1.1: Sintează comparativă a metodelor RAG prezentate și a relevanței lor pentru extragerea relațiilor între entități.

1.6 Metrici de evaluare

Evaluarea unui sistem RAG este mai complexă decât evaluarea unui model generativ izolat, deoarece performanța globală depinde atât de componenta de recuperare, cât și de cea de generare [5]. O recuperare slabă limitează calitatea răspunsului indiferent de capacitățile modelului generativ. Din acest motiv, evaluarea se face de obicei pe două niveluri. Evaluarea pe componente măsoară separat calitatea recuperării (cât de bune sunt fragmentele aduse) și calitatea generării (cât de bun este răspunsul pentru un context dat). Evaluarea end-to-end măsoară calitatea răspunsului final, așa cum îl primește utilizatorul. Cele două perspective sunt complementare: un scor end-to-end slab nu spune unde este problema, în timp ce metricile pe componente localizează cauza, dar nu garantează că un sistem cu ambele componente bune produce și un răspuns final bun.

1.6.1 Metrici de recuperare

Precizia la rang k (Precision@ k) este proporția documentelor relevante printre primele k rezultate recuperate:

$$\text{Precision@}k = \frac{|\{\text{relevante}\} \cap \{\text{top-}k\}|}{k}. \quad (1.2)$$

Recuperarea la rang k (Recall@ k) reprezintă proporția tuturor documentelor relevante care au fost efectiv recuperate:

$$\text{Recall@}k = \frac{|\{\text{relevante}\} \cap \{\text{top-}k\}|}{|\{\text{relevante}\}|}. \quad (1.3)$$

Mean Reciprocal Rank (MRR) măsoară rangul primului document relevant, mediat peste $|Q|$ interogări:

$$\text{MRR} = \frac{1}{|Q|} \sum_{i=1}^{|Q|} \frac{1}{\text{rang}_i}. \quad (1.4)$$

Hits@ k este o metrică binară care indică dacă cel puțin un document relevant apare printre primele k rezultate. Spre deosebire de Recall@ k , care măsoară proporția documentelor relevante recuperate, Hits@ k răspunde doar la întrebarea dacă recuperarea a reușit sau nu. Este utilă atunci când un singur document relevant este suficient pentru a răspunde, situație frecventă în răspunsul la întrebări factuale.

Normalized Discounted Cumulative Gain (nDCG) ține cont de poziția exactă a fiecărui document relevant, aplicând o reducere logaritmică pentru pozițiile inferioare. Spre deosebire de MRR, care privește doar primul document relevant, nDCG evaluează întreaga listă de rezultate, fiind potrivită atunci când mai multe documente relevante contribuie la răspuns. Valoarea este normalizată între 0 și 1, unde 1 corespunde ordinii ideale.

Aceste metrici sunt calculate față de un set de documente relevante adnotate manual. În practică, Recall@ k este adesea mai informativă decât Precision@ k pentru sistemele RAG, deoarece un document relevant ratat nu poate fi recuperat ulterior de modelul generativ [20].

Un exemplu numeric clarifică aceste definiții. Să presupunem o interogare cu trei documente relevante în corpus, pentru care sistemul returnează cinci rezultate, dintre care relevante sunt cele de pe pozițiile 1, 2 și 4. Atunci Precision@5 = 3/5 = 0,6, deoarece trei din cele cinci rezultate sunt relevante, iar Recall@5 = 3/3 = 1,0, deoarece toate documentele relevante au fost recuperate. Primul rezultat relevant apare pe poziția 1, deci contribuția la MRR pentru această interogare este 1/1 = 1,0. Dacă, în schimb, primul rezultat relevant ar fi apărut pe poziția 3, contribuția ar fi fost 1/3 ≈ 0,33, ceea ce arată cum MRR penalizează plasarea fragmentului corect pe poziții inferioare.

1.6.2 Metrici de generare

Exact Match (EM) este o metrică binară care indică dacă răspunsul de referință apare ca subșir în răspunsul generat, după normalizarea majusculor și a semnelor de punctuație. Spre deosebire de potrivirea strictă, token cu token, această variantă nu penalizează cuvintele suplimentare din jurul referinței, fiind potrivită pentru răspunsuri scurte sau factuale.

Scorul F_1 la nivel de token echilibrează precizia P și recuperarea R față de răspunsul de referință:

$$F_1 = \frac{2 \cdot P \cdot R}{P + R}. \quad (1.5)$$

Pentru răspunsuri mai lungi, unde potrivirea exactă este irealistă, se utilizează suprapunerea de n -grame calculată cu metricile ROUGE (*Recall-Oriented Understudy for Gisting Evaluation*) și BLEU (*Bilingual Evaluation Understudy*) [5]. BLEU măsoară precizia n -gramelor, adică ce proporție din n -gramele răspunsului generat apar în referință, fiind orientată spre precizie și folosită inițial pentru traducerea automată. ROUGE măsoară în schimb acoperirea, adică ce proporție din n -gramele referinței apar în răspunsul generat, fiind orientată spre recall și folosită frecvent pentru rezumare. Varianta ROUGE-L măsoară lungimea celui mai lung subșir comun,

captând ordinea parțială a tokenurilor fără a cere potrivire pe poziții consecutive.

Aceste metrice au o limitare comună importantă: penalizează reformulările corecte. Un răspuns corect din punct de vedere semantic, dar exprimat cu alte cuvinte decât referința, primește un scor scăzut, deși este perfect valid. Acest aspect este relevant pentru sistemele RAG cu modele generative moderne, care tind să parafrazeze răspunsul în loc să copieze textual fragmentul recuperat.

1.6.3 Metrice specifice sistemelor RAG

Metricile clasice de generare nu surprind fenomenul halucinației în contextul RAG, deoarece nu verifică dacă răspunsul este fundamentat pe contextul recuperat. Cadrul RAGAS [40] introduce metrice dedicate:

Fidelitatea (eng. „*faithfulness*”) măsoară proporția afirmațiilor din răspuns care pot fi deduse din contextul recuperat. Răspunsul este descompus în afirmații individuale, iar fiecare afirmație este verificată față de context. Formal, fidelitatea este raportul

$$\text{Fidelitate} = \frac{\text{numărul de afirmații susținute de context}}{\text{numărul total de afirmații din răspuns}}. \quad (1.6)$$

O fidelitate de 1,0 indică un răspuns complet susținut de dovezile furnizate, în timp ce o valoare scăzută semnalează prezența halucinațiilor.

Relevanța răspunsului (eng. „*answer relevancy*”) măsoară în ce grad răspunsul adresează interogarea originală, penalizând răspunsurile complete factual, dar tangențiale față de întrebare. Se calculează generând mai multe întrebări posibile pornind de la răspuns și măsurând similaritatea lor medie cu întrebarea originală: un răspuns relevant produce întrebări apropiate de cea reală.

Precizia contextului și recuperarea contextului evaluează calitatea recuperării din perspectiva generării: câte dintre fragmentele recuperate sunt cu adevărat necesare și câte dintre fragmentele necesare au fost recuperate [40]. Aceste metrice sunt centrale pentru sistemele orientate spre fidelitate [22].

Aceste metrice au în comun faptul că folosesc un model de limbaj ca evaluator (eng. „*LLM-as-judge*”). În loc să compare răspunsul cu o referință fixă, un model de limbaj apreciază dacă afirmațiile din răspuns sunt susținute de context sau dacă răspunsul adresează întrebarea. Avantajul este flexibilitatea: metrica tolerează reformulările și surprinde nuanțe pe care potrivirea de n -grame le ratează. Dezavantajul este costul și dependența de calitatea modelului-judecător, care poate la rândul lui greși. Pentru extragerea relațiilor, evaluarea cu model-judecător poate verifica dacă relația exprimată în răspuns este efectiv susținută de fragmentul recuperat, ceea ce o face complementară metricilor clasice bazate pe potrivire exactă.

Pe lângă aceste metrice, comportamentul unui sistem RAG se evaluează și prin rata de halucinație și prin latența de recuperare. Rata de halucinație este fracția de întrebări fără răspuns în corpus la care sistemul produce totuși un răspuns în loc să refuze, deci măsoară rezistența la fabricarea de informație. Latența de recuperare este timpul necesar procesării unei interogări și arată costul practic al optimizărilor de recuperare. Ambele sunt folosite în partea experimentală a lucrării.

1.6.4 Dificultăți în evaluarea extragerii relațiilor

Evaluarea extragerii relațiilor se realizează cu scorurile micro și macro F_1 calculate pe tipurile de relații. Micro- F_1 ponderează fiecare instanță egal, favorizând tipurile de relații frecvente. Macro- F_1 tratează fiecare tip de relație egal, indiferent de frecvența sa.

Există două dificultăți structurale. Prima este distribuția dezechilibrată a tipurilor de relații în seturile de date existente, care face ca micro- F_1 să fie adesea înșelătoare [41]. A doua este problema falselor negative: relații corecte neanotate în setul de referință duc la subestimarea performanței reale. Re-DocRED [41] a demonstrat că DocRED [4] conținea un număr semnificativ de astfel de false negative.

Metricile de fidelitate nu reflectă direct corectitudinea tipului de relație extras: un sistem poate fi fidel față de contextul recuperat, dar contextul însuși poate conține relații ambigue sau greșit formulate. Prin urmare, evaluarea completă a unui sistem RAG pentru extragerea relațiilor necesită atât metrici de fidelitate, cât și metrici specifice extragerii.

O dificultate suplimentară ține de nivelul de detaliu al evaluării. O relație poate fi corectă ca tip, dar greșită ca argumente, sau corectă ca argumente, dar greșită ca direcție (de exemplu, „X este autorul lui Y” față de „Y este autorul lui X”). O metrică binară care marchează relația ca fiind corectă sau greșită pierde aceste nuanțe. De aceea, evaluările riguroase raportează separat performanța pe identificarea entităților, pe clasificarea tipului de relație și pe direcția relației, ceea ce oferă o imagine mai detaliată a punctelor slabe ale sistemului.

1.7 Baze de date

Evaluarea unui sistem RAG pentru extragerea relațiilor se sprijină pe două tipuri de seturi de date. Primul tip provine din regăsirea informației și răspunsul la întrebări (QA), unde se măsoară dacă sistemul recuperează pasajul corect și produce răspunsul potrivit. Al doilea tip este specific extragerii relațiilor, unde adnotările sunt triplete de forma (subiect, relație, obiect) și se măsoară dacă sistemul identifică legăturile corecte dintre entități. Această secțiune prezintă seturile reprezentative din ambele categorii, iar Tabelul 1.2 le sintetizează. În această lucrare, validarea folosește SQuAD, iar evaluarea pe extragerea relațiilor folosește DocRED.

1.7.1 Seturi de date pentru sisteme RAG

SQuAD [3] (Stanford Question Answering Dataset) leagă întrebări de pasaje din Wikipedia, răspunsul reprezentând un fragment de text contiguu extras din pasaj. Setul conține 100.000 de întrebări construite de adnotatori umani, ceea ce îl face un standard pentru evaluarea recuperării și extracției.

Natural Questions [42] conține întrebări reale adresate motorului de căutare Google, perechi cu pagini Wikipedia. Spre deosebire de SQuAD, întrebările nu sunt construite cu pasaje în față, ceea ce le face mai reprezentative pentru scenariile reale de utilizare. Setul de date include atât răspunsuri scurte, cât și lungi.

TriviaQA [43] este un set de perechi întrebare-răspuns colectate din resurse trivia, însoțite de documente justificative găsite independent prin căutare web. Dificultatea suplimentară față de SQuAD este că documentele justificative nu sunt garantat să conțină răspunsul.

HotpotQA [44] este un set de date care necesită raționament pe mai multe documente simultan. Fiecare întrebare este adnotată cu faptele justificative din mai multe articole Wikipedia, forțând sistemele să realizeze recuperare și raționament multi-hop. Este relevant pentru extragerea relațiilor care implică mai multe entități intermediare.

CRAG-Bench [45] (Comprehensive RAG Benchmark) conține 4.409 perechi QA generate cu API (*Application Programming Interface*)-uri de căutare web și de grafuri de cunoștințe. Acoperă cinci domenii și opt categorii de întrebări cu niveluri variabile de dinamism temporal, ceea ce îl face util pentru evaluarea robusteții față de informații care se schimbă în timp.

1.7.2 Seturi de date pentru extragerea relațiilor

T-REx [11] este un set de aliniament la scară largă între rezumatele Wikipedia și tripletele Wikidata: 11 milioane de triplete aliniate cu 3,09 milioane de rezumate. Structura sa face T-REx potrivit pentru evaluarea sistemelor care combină recuperarea din text liber cu validarea față de grafuri de cunoștințe.

WebNLG [12] furnizează perechi date-text pornind de la triplete RDF din DBpedia. Setul de date are aplicații atât în extragerea relațiilor, cât și în verbalizarea datelor structurate, deoarece fiecare triplet este însoțit de fraze naturale care îl exprimă.

SemEval-2010 Task 8 [13] este un test de referință pentru clasificarea relațiilor semantice între perechi nominale la nivel de propoziție, cu nouă tipuri de relații direcționale plus o clasă negativă. Este utilizat frecvent ca referință pentru compararea metodelor de extragere a relațiilor la nivel de propoziție.

TACRED [14] este un set de date de mari dimensiuni construit din articole de știri și rapoarte ale agenției Associated Press, cu 41 de tipuri de relații și o clasă negativă predominantă. Dezechilibrul accentuat între clase face evaluarea macro- F_1 mai informativă decât micro- F_1 .

DocRED [4] extinde extragerea relațiilor la nivel de document, necesitând raționament inter-propoziții. Un număr semnificativ de relații implică entități menționate în propoziții diferite, ceea ce face DocRED mai dificil decât seturile sentence-level.

Re-DocRED [41] corectează problema falselor negative din DocRED prin reannotare manuală sistematică. Studiul original a arătat că o parte considerabilă din relațiile corecte din DocRED nu fuseseră adnotate, ceea ce subestima performanța modelelor evaluate pe setul original.

Set de date	Sarcină	Nivel	Caracteristici principale
SQuAD	QA extractiv	Propoziție	Fragment de text ca răspuns
Natural Questions	QA	Document	Întrebări reale, răspuns scurt/lung
HotpotQA	QA multi-hop	Multi-document	Fapte justificative adnotate
CRAG-Bench	QA factual	Web / graf	4.409 perechi, dinamism temporal
T-REx	Extragere relații	Document	Aliniament text-triplete Wikidata
WebNLG	Extragere/verbalizare	Triplete RDF	Perechi date-text
SemEval-2010 T8	Extragere relații	Propoziție	Relații nominale, 9 tipuri
TACRED	Extragere relații	Propoziție	41 tipuri de relații
DocRED	Extragere relații	Document	Raționament inter-propoziții
Re-DocRED	Extragere relații	Document	Corectare false negative

Tabel 1.2: Comparație a seturilor de date pentru evaluarea RAG și extragerea relațiilor [3, 4, 11–14, 41, 42, 44, 45].

Alegerea setului de date pentru evaluare depinde de ceea ce se dorește măsurat. Pentru a valida recuperarea și răspunsul la întrebări, seturile de tip QA precum SQuAD sunt potrivite, deoarece răspunsul este un fragment cu sursă cunoscută, ceea ce permite o verificare automată și obiectivă. Pentru a măsura direct extragerea relațiilor, sunt necesare seturi cu triplete adnotate, precum T-REx sau DocRED, unde se poate verifica dacă sistemul identifică legătura corectă dintre două entități.

În această lucrare, validarea folosește SQuAD, pentru că permite o măsurare precisă și reproductibilă a recuperării fără a depinde de un model-judecător, iar evaluarea pe extragerea relațiilor folosește DocRED, ale cărui adnotări sub formă de triplete permit o evaluare directă a relațiilor extrase.

1.8 Limitări și provocări

Pe baza metodelor, metricilor și seturilor de date discutate în acest capitol se conturează câteva limitări și provocări recurente ale sistemelor RAG aplicate extragerii relațiilor între entități. Ele privesc deopotrivă recuperarea, generarea, datele și evaluarea, și delimitează spațiul în care se plasează contribuția experimentală a lucrării.

Calitatea recuperării este factorul cel mai important. Documentele irelevante sau parțial relevante induc în eroare modelul generativ, ducând la halucinații chiar și atunci când există o sursă externă de informație [5, 32]. Recuperarea fixă, neadaptivă, nu poate evalua sau corecta automat rezultatele slabe [22], iar fragmentele care conțin una dintre entitățile căutate, dar nu și relația relevantă, sunt o sursă frecventă de erori în extragerea relațiilor.

Integrarea contextelor lungi ridică o a doua problemă, denumită uneori *lost in the middle*: modelele generative tind să ignore informațiile aflate la mijlocul unui context lung, chiar dacă sunt relevante, iar pasajele irelevante dintr-un context prea lung degradează calitatea generării [26].

Limitările datelor și ale metricilor afectează direct evaluarea. Seturile de date pentru extragerea relațiilor au distribuții dezechilibrate și conțin false negative [41], metricile de fidelitate nu reflectă direct corectitudinea tipului de relație extras [40], iar construirea de seturi de calitate la nivel de document este costisitoare și lentă.

Extragerea relațiilor între entități ridică dificultăți specifice. Identificarea unei relații necesită adesea agregarea dovezilor din mai multe propoziții sau documente, un raționament inter-propoziție greu de realizat într-un singur pas de recuperare [2, 4], iar relațiile implicite, care nu apar formulat explicit în text, adaugă o provocare suplimentară.

Costul computațional este o constrângere practică. Indexarea densă, recuperarea hibridă, reordonarea cu *cross-encoder* și generarea cu modele mari consumă resurse semnificative [20, 31]. Abordările adaptive și active implică recuperare iterativă, cu mai multe apeluri la model pe interogare [33, 34], iar metodele bazate pe grafuri adaugă costul construirii și actualizării grafului de cunoștințe [17, 35]. În absența unor resurse computaționale ample, aceste cerințe limitează scala la care sistemele pot fi evaluate sau folosite în producție.

Evaluarea automată introduce o dependență circulară. Multe metrici specifice RAG, precum cele din cadrul RAGAS [40], folosesc un model de limbaj ca judecător pentru a aprecia fidelitatea sau relevanța răspunsului, iar calitatea evaluării ajunge să depindă de calitatea unui alt model de limbaj, care la rândul lui poate halucina sau aprecia greșit. În plus, evaluarea cu un model-judecător este lentă și costisitoare, deoarece presupune câte un apel suplimentar pentru fiecare pereche întrebare-răspuns evaluată, ceea ce face dificilă rularea pe seturi mari de date.

Fiabilitatea sursei contează la fel de mult ca arhitectura. Atunci când baza de cunoștințe conține informații greșite, învechite sau intenționat manipulate, sistemul RAG le poate prelua și prezenta ca fapte, cu aparența de autoritate dată de fundamentarea pe o sursă. Robustețea unui sistem RAG nu depinde doar de arhitectură, ci și de calitatea și de fiabilitatea sursei de cunoștințe, un aspect adesea trecut cu vederea în evaluările de laborator.

O provocare specifică extragerii relațiilor ține de echilibrul dintre generalizare și fiabilitate. Metodele clasice supervizate sunt precise pe o schemă fixă de relații, dar nu se transferă la relații nevăzute fără reantrenare, în timp ce abordările deschise sau bazate pe RAG generalizează la relații noi, dar cu o fiabilitate mai greu de garantat [10]. Un sistem RAG care formulează extragerea ca întrebare poate trata orice tip de relație, însă corectitudinea răspunsului depinde critic de recuperarea fragmentului potrivit și de capacitatea modelului de a nu inventa o relație plauzibilă, dar nesuștinută de context [22].

Nu în ultimul rând, două limitări mai puțin evidente reprezintă decalajul dintre recuperare și generare, și acoperirea lingvistică. Pe de o parte, o recuperare bună nu garantează o relație corect extrasă, deoarece modelul generativ poate parafraza greșit, poate inversa direcția relației

sau poate combina dovezi din pasaje diferite într-o afirmație eronată [4]. Pe de altă parte, majoritatea seturilor de date și a modelelor folosite în literatură sunt centrate pe limba engleză și pe texte enciclopedice, astfel încât comportamentul sistemelor pe alte limbi sau pe domenii specializate, precum cel medical sau juridic, rămâne insuficient studiat [5].

1.9 Concluzii

Capitolul de față a prezentat contextul teoretic al sistemelor RAG aplicate extragerii relațiilor între entități. Au fost discutate limitările cunoștințelor parametrice ale modelelor lingvistice mari [1, 5] și motivul pentru care augmentarea cu recuperare externă este o soluție practică, urmată de o taxonomie specifică acestei teze, construită pe criteriile tipului de mecanism de recuperare, al modului de integrare recuperare-generare și al sursei de cunoștințe.

Analiza literaturii arată că robustețea sistemelor RAG depinde în primul rând de calitatea recuperării și de capacitatea de a evalua adaptiv dovezile recuperate [22, 32]. Extragerea relațiilor beneficiază cel mai mult de structurile de grafuri de cunoștințe și de abordările cu raționament la nivel de document [2, 4, 17]. Provocările deschise rămân integrarea contextelor lungi [26], limitările metricilor și datelor [40, 41] și costul computațional [31].

Sinteza metodelor relevă un compromis recurent: pe măsură ce arhitecturile devin mai capabile, de la recuperarea densă simplă la reordonare, recuperare adaptivă și grafuri de cunoștințe, cresc și complexitatea de implementare, și costul de calcul. Pentru extragerea relațiilor, abordările care modelează explicit legăturile dintre entități oferă cea mai bună potrivire structurală, dar presupun o etapă costisitoare de construire a grafului, în timp ce recuperarea hibridă cu reordonare pe text nestructurat aduce o parte din beneficii la un cost mult mai mic.

Tocmai din această sinteză rezultă întrebarea care motivează partea experimentală. Optimizările de recuperare de tip Advanced RAG sunt evaluate în literatură aproape exclusiv pe sarcini de răspuns la întrebări de uz general, iar abordările cele mai potrivite structural pentru relații, cele bazate pe grafuri, rămân costisitoare. Cât de mult ajută aceste optimizări aplicate pe text nestructurat atunci când sarcina este extragerea relațiilor între entități rămâne o întrebare insuficient măsurată, pe care lucrarea de față o adresează direct.

Lucrarea se plasează la intersecția optimizării recuperării cu dezvoltarea de mecanisme generative care integrează explicit dovezile recuperate. Studiul comparativ pune față în față Naive RAG și Advanced RAG, pornind de la observația că optimizările pre- și post-recuperare conduc la îmbunătățiri măsurabile ale fidelității și relevanței răspunsurilor [5, 40], aspecte cu impact direct asupra corectitudinii relațiilor extrase.

Direcția adoptată este motivată de un compromis practic. Arhitecturile bazate pe grafuri sunt cele mai potrivite structural pentru extragerea relațiilor, dar presupun un cost ridicat de construire a grafului și o complexitate de implementare semnificativă. Advanced RAG, prin recuperarea hibridă și reordonarea aplicate pe text nestructurat, oferă o parte importantă din beneficii la un cost mult mai mic, ceea ce îl face un punct de plecare rezonabil pentru studiul comparativ. Capitolele următoare descriu arhitectura acestui sistem și o compară cantitativ și calitativ cu varianta de bază.

Capitolul 2

Arhitectura sistemului propus

Studiul comparativ din această lucrare pune față în față două arhitecturi RAG: una de bază (Naive RAG) și una cu optimizări de recuperare (Advanced RAG). Pentru a măsura efectul optimizărilor de recuperare descrise în Capitolul 1, comparația este proiectată astfel încât toate componentele să fie menținute constante între cele două sisteme, cu o singură excepție: complexitatea etapei de recuperare. Ambele folosesc același model generativ, același set de date și aceeași bază de cunoștințe, diferind doar prin modul în care sunt selectate fragmentele trimise modelului.

Sistemul rulează integral local, pe hardware propriu, fără apeluri către servicii externe. Această alegere a fost importantă din două motive. Primul ține de reproducibilitate: un model rulat local produce aceleași rezultate de fiecare dată, fără a depinde de versiunea unui API care se poate schimba peste noapte. Al doilea ține de control: rularea locală oferă acces complet la promptul de sistem, la parametrii de recuperare și la fragmentele intermediare, ceea ce face posibilă analiza pas cu pas a comportamentului.

Capitolul descrie proiectarea celor două fluxuri de procesare, de la cerințe și tehnologii până la algoritmi de recuperare. Demonstratorul care le implementează, cu interfața sa și cu modulul de evaluare folosit pentru a produce rezultatele, este prezentat separat, la finalul capitolului de rezultate.

2.1 Privire de ansamblu

Sistemul are două faze. Faza offline, executată câte o dată pentru fiecare set de date (SQuAD și DocRED), construiește baza de cunoștințe. Faza online, executată la fiecare interogare, recuperează fragmentele relevante și generează răspunsul. Codul este împărțit în module care corespund acestor faze: `ingest.py` pentru indexare (Anexa 1), pachetele `naive` și `advanced` pentru cele două fluxuri de procesare, `benchmark.py` pentru evaluare (Anexa 3) și `main.py` pentru interfață (Anexa 2).

Figura 2.1 prezintă faza offline. Setul de date este încărcat, unitățile sunt deduplicate, apoi codificate și depuse în două indexuri: unul dens în ChromaDB și unul rar, BM25. Ambele fluxuri de procesare citesc din aceleași indexuri, ceea ce garantează că diferențele de performanță provin din strategia de recuperare, nu din date diferite. Cele două fluxuri de procesare sunt descrise în detaliu mai jos, după prezentarea cerințelor și a tehnologiilor.

2.2 Cerințe

Înainte de implementare au fost stabilite câteva cerințe care au ghidat deciziile ulterioare.

Pe partea funcțională, sistemul trebuie să implementeze două fluxuri de procesare RAG complete, de la interogare la răspuns, peste aceeași bază de cunoștințe. Trebuie să existe o interfață care permite interogarea simultană a ambelor sisteme și afișarea răspunsurilor una lângă alta, astfel încât diferențele să fie vizibile direct. În plus, este nevoie de un modul de

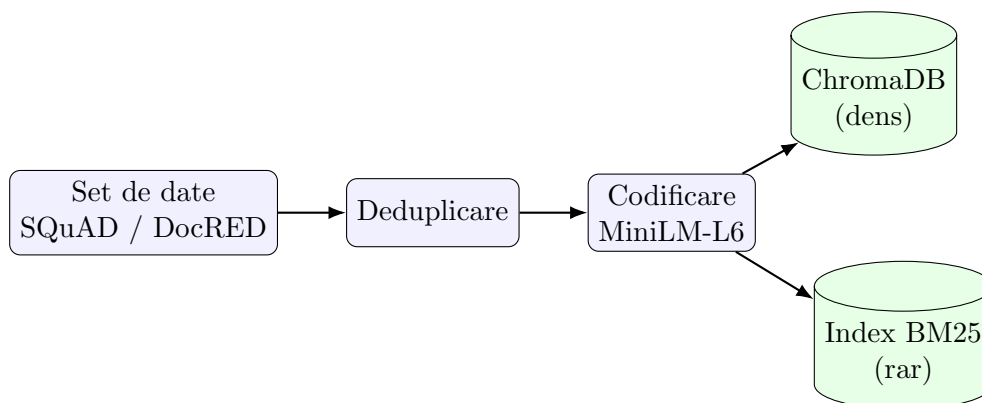


Figura 2.1: Faza offline de indexare. Unitățile unice ale setului de date (paragrafe pentru SQuAD, documente pentru DocRED) sunt codificate o singură dată și depuse în indexul dens (ChromaDB) și în indexul rar (BM25), folosite în comun de ambele fluxuri de procesare. Fiecare set de date are propriile indexuri.

evaluare automată care rulează cele două sisteme pe un set de întrebări și calculează metrici comparative.

Pe partea nefuncțională, comparația trebuie să fie corectă, ceea ce înseamnă că singura variabilă între cele două sisteme este arhitectura de recuperare. Rezultatele trebuie să fie reproducibile, motiv pentru care evaluarea rulează local, cu seed-uri fixate. Sistemul trebuie să ruleze pe o singură placă grafică de consum, fără infrastructură distribuită. Codul trebuie organizat modular, cu fluxurile de procesare separate de interfață și de modulul de evaluare, pentru ca fiecare componentă să poată fi modificată independent.

2.3 Tehnologii utilizate

Tabelul 2.1 rezumă tehnologiile folosite și rolul fiecăreia. Alegerile au urmărit instrumente mature, larg folosite în sistemele RAG, care pot rula local.

Componentă	Tehnologie	Rol
Model generativ	Ollama + Qwen3-8B	Generarea răspunsului
Expansiune interogări	Qwen3-1.7B	Reformularea interogării
Model de codificare	all-MiniLM-L6-v2	Codificarea semantică a textului
Reordonare	ms-marco-MiniLM-L-6-v2	Re-scorarea fragmentelor
Bază vectorială	ChromaDB	Indexare și căutare densă
Indexare rară	bm25s	Căutare BM25 pe cuvinte cheie
Interfață	Streamlit	Interacțiune și vizualizare
Evaluare	RAGAS, SciPy	Metrici și teste statistice
Set de date	SQuAD, DocRED (HuggingFace)	Corpus și întrebări de test

Tabel 2.1: Tehnologiile utilizate în demonstrator

Modelul generativ este Qwen3-8B, rulat prin Ollama. Qwen3 are un mod de raționament (eng. „*thinking mode*”) care poate fi dezactivat cu directiva `/no_think` în prompt. Această opțiune este folosită la expansiunea interogărilor, unde un raționament extins ar încetini procesul fără să aducă un beneficiu real.

Modelul de codificare este `all-MiniLM-L6-v2`, un model compact de aproximativ 22 de milioane de parametri care produce vectori de dimensiune 384, din familia encoderelor pentru recuperare densă a căror abordare seminală este DPR [20]. Raportul bun între calitate și viteză

îl face o alegere des întâlnită în sistemele RAG fără constrângeri hardware mari. Pentru reordonare se folosește un *cross-encoder* din aceeași familie, **ms-marco-MiniLM-L-6-v2**, antrenat pentru a estima relevanța unei perechi (interogare, pasaj).

2.4 Configurația hardware

Întreg sistemul a fost dezvoltat și evaluat pe o singură stație de lucru, fără infrastructură distribuită. Configurația cuprinde o placă grafică NVIDIA RTX A5000 cu 24 GB de memorie video, un procesor Intel Xeon Silver 4309Y la 2,80 GHz cu 8 nuclee și 16 fire de execuție, 64 GB de memorie RAM și un disc M.2 de 1 TB. Placa grafică rulează cele două modele Qwen3 prin Ollama. Modelele de codificare și *cross-encoder*, mult mai mici, rulează tot pe placa grafică la evaluarea pe SQuAD, dar sunt mutate pe procesor la cea pe DocRED, pentru a lăsa întreaga memorie video modelelor lingvistice; din acest motiv latențele absolute pe DocRED sunt mai mari. Cei 24 GB de memorie video sunt suficienți pentru a ține simultan modelele Qwen3 și modelele auxiliare, fără a le descărca și reîncărca între interogări. Toate valorile de latență raportate în capitolul de rezultate depind de această configurație.

2.5 Indexarea datelor

Indexarea este realizată de scriptul `ingest.py` (codul complet în Anexa 1) și rulează o singură dată, la instalare. Setul de date SQuAD este încărcat din depozitul HuggingFace, iar paragrafele se repetă, pentru că fiecare apare în mai multe rânduri, câte unul per întrebare. Primul pas elimină aceste duplicate, păstrând fiecare paragraf o singură dată împreună cu titlul articolului din care provine.

Unitatea de bază a corpusului este paragraful SQuAD, indexat ca atare. Nu se aplică o fragmentare suplimentară, deoarece paragrafele SQuAD au deja o lungime potrivită pentru recuperare, suficient de scurte pentru a fi specifice și suficient de lungi pentru a păstra contextul răspunsului. Paragrafele unice sunt codificate cu modelul MiniLM și depuse în ChromaDB, configurat cu spațiu cosinus pentru similaritate. Modelul de codificare trunchiază textul la maximum 256 de token-uri, limita ferestrei sale de context, deci vectorii denși reprezintă doar prima porțiune a fiecărui text. În paralel, textul brut integral al acelorași paragrafe este tokenizat și indexat cu BM25, fără nicio trunchiere: BM25 operează pe frecvențe de termeni, nu pe vectori, și nu este limitat de fereastra de context a modelului de codificare. Indexul BM25 se construiește la prima utilizare și se salvează pe disc, astfel încât pornirile ulterioare să nu îl reconstruiască.

Aceeași procedură se aplică și setului DocRED, sarcina-țintă a lucrării, pe care se măsoară extragerea relațiilor între entități, în timp ce SQuAD rămâne setul de referință pentru validarea arhitecturii. Documentele DocRED sunt indexate ca unități, fără fragmentare suplimentară. Fiecare set de date are propria colecție ChromaDB și propriul index BM25, astfel încât recuperarea să nu amestece documente din surse diferite.

2.6 Fluxul de procesare Naive RAG

Naive RAG urmează cel mai simplu traseu posibil, cel din formularea originală a RAG [1]: codifică interogarea, caută cele mai apropiate fragmente prin similaritate cosinus și le trimite modelului generativ. Nu există reformulare, recuperare hibridă sau reordonare. Pașii, ilustrați în Figura 2.2, sunt următorii.

1. Interogarea este codificată cu **all-MiniLM-L6-v2** într-un vector de dimensiune 384.

2. ChromaDB returnează primele $K = 5$ fragmente după similaritate cosinus.
3. Fragmentele sunt concatenate și incluse într-un prompt împreună cu interogarea.
4. Qwen3-8B generează răspunsul, instruit prin promptul de sistem să folosească exclusiv contextul furnizat.

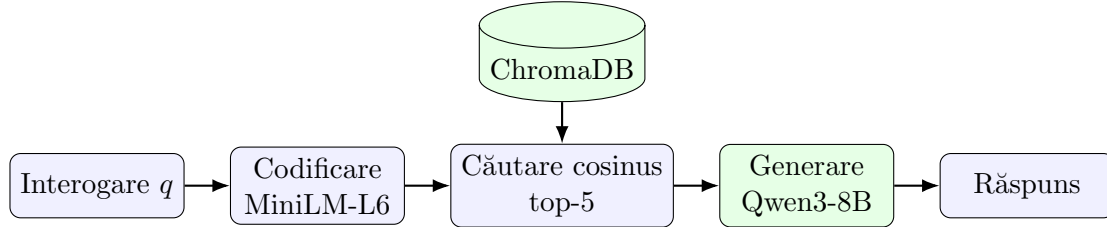


Figura 2.2: Fluxul de procesare Naive RAG. Interogarea este codificată, cele mai apropiate cinci fragmente sunt căutate prin similaritate cosinus în ChromaDB și trimise direct modelului generativ, fără reformulare, recuperare hibridă sau reordonare.

Promptul de sistem cere modelului să răspundă doar pe baza contextului și să refuze explicit dacă informația lipsește. Această variantă de bază are limitările cunoscute din literatură [5]: ratează termenii rari pe care similaritatea semantică îi diluează și nu filtrează fragmentele irelevante recuperate din greșeală.

2.7 Fluxul de procesare Advanced RAG

Advanced RAG adaugă patru etape peste varianta de bază: expansiunea interogării, recuperarea hibridă, fuziunea rezultatelor și reordonarea. Figura 2.3 prezintă fluxul complet, iar Tabelul 2.2 listează parametrii folosiți.

Parametru	Valoare	Semnificație
Reformulări ale interogării	2	Variante generate pe lângă interogarea originală
Candidați denși per variantă	20	Fragmente returnate de ChromaDB
Candidați BM25 per variantă	20	Fragmente returnate de BM25
Constanta RRF	60	Parametrul k din fuziune
Dimensiunea setului de candidați	40	Candidați trimiși la reordonare
Fragmente finale	5	Fragmente trimise modelului generativ

Tabel 2.2: Parametrii fluxului de procesare Advanced RAG

Prima etapă este expansiunea interogării. Un model mic, Qwen3-1.7B, generează două reformulări ale interogării originale. Scopul este reducerea dependenței de formularea exactă a întrebării: dacă utilizatorul folosește alți termeni decât cei din document, o reformulare bună poate recupera totuși fragmentul corect. Modelul mic a fost ales deliberat, pentru că reformularea nu cere raționament complex, iar acest apel se află pe calea critică a fiecărei interogări.

A doua etapă este recuperarea hibridă. Pentru fiecare dintre cele trei interogări (originala și cele două reformulări) se execută în paralel o căutare densă în ChromaDB și o căutare BM25, fiecare returnând câte 20 de candidați. Recuperarea densă prinde asemănările de sens, iar BM25 [19] prinde potrivirile lexicale exacte, utile pentru termeni rari, numere și denumiri

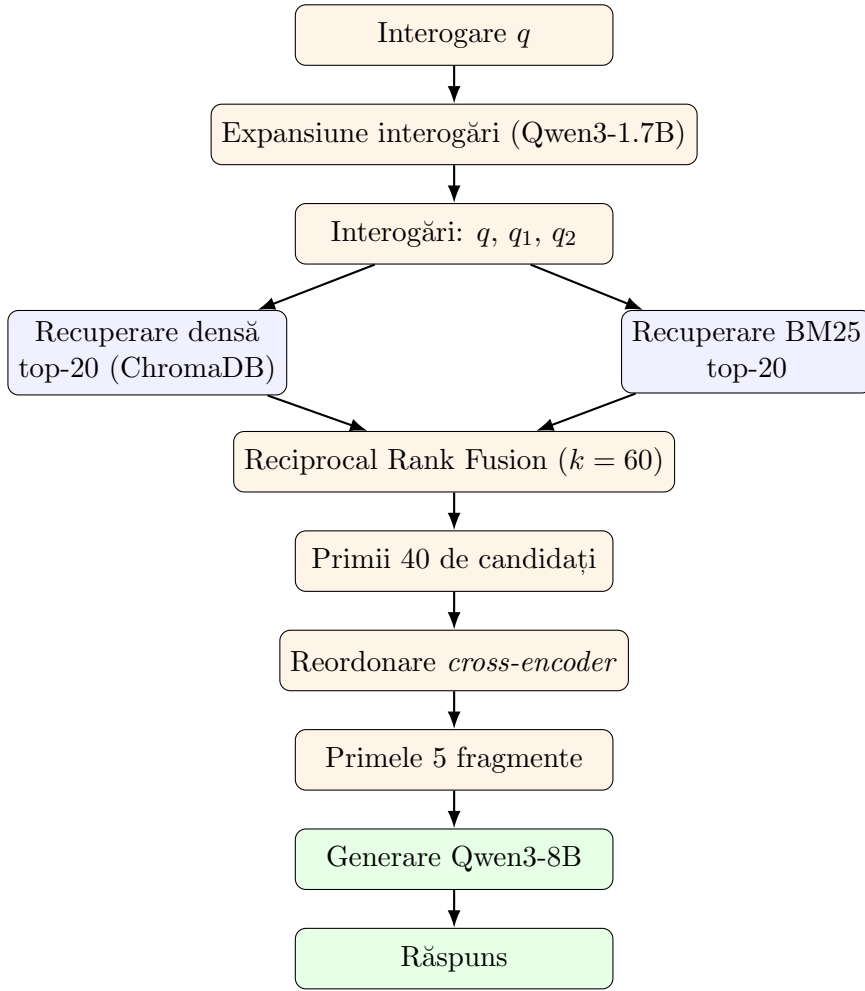


Figura 2.3: Fluxul de procesare Advanced RAG. Interogarea este reformulată, fiecare variantă este căutată dens și prin BM25, rezultatele sunt fuzionate prin Reciprocal Rank Fusion, re-scorate cu un cross-encoder, iar primele cinci fragmente sunt trimise modelului generativ.

proprii. Cele șase căutări rulează concurrent, pe fire de execuție separate, pentru a limita timpul total.

A treia etapă fuzionează cele șase liste de rezultate prin Reciprocal Rank Fusion [21]. Fiecare fragment primește un scor care însumează contribuțiile din toate listele în care apare, după formula

$$\text{RRF}(d) = \sum_l \frac{1}{k + r_l(d)}, \quad (2.1)$$

unde $r_l(d)$ este rangul fragmentului d în lista l , iar $k = 60$. Constanta k atenuează influența pozițiilor de top, astfel încât un fragment aflat constant pe locuri bune în mai multe liste să fie favorizat față de unul aflat pe primul loc într-o singură listă. După fuziune se păstrează primii 40 de candidați.

A patra etapă este reordonarea. *Cross-encoder*-ul `ms-marco-MiniLM-L-6-v2` primește perechea (interogare, fragment) și produce un scor de relevanță. Spre deosebire de modelul de codificare, care codifică separat interogarea și fragmentul, *cross-encoder*-ul le procesează împreună, ceea ce dă o estimare mai precisă a relevanței, dar cu un cost de calcul mai mare. Din acest motiv el se aplică doar celor 40 de candidați rămași, nu întregului corpus. Primele cinci fragmente după reordonarea cu *cross-encoder* ajung la modelul generativ, care produce răspunsul cu același prompt de ancorare ca în varianta Naive.

Capitolul 3

Rezultate experimentale

Acest capitol prezintă evaluarea experimentală a celor două arhitecturi descrise în capitolul anterior. Comparația măsoară efectul optimizărilor de recuperare pe două sarcini: răspunsul la întrebări extractiv, pe SQuAD, folosit pentru validarea arhitecturii, și extragerea relațiilor între entități, pe DocRED, sarcina-țintă a lucrării. Sunt raportate metrice de recuperare și de generare, teste de semnificație statistică și o analiză calitativă a comportamentului celor două sisteme. La final, o secțiune dedicată descrie demonstratorul folosit, cu interfața sa și cu modulul de evaluare.

3.1 Evaluarea pe SQuAD

SQuAD este setul de date de referință al lucrării, folosit pentru a valida arhitectura înainte de sarcina-țintă, extragerea relațiilor între entități, prezentată în Secțiunea 3.2. Această secțiune prezintă mai întâi rezultatele cantitative, apoi interpretarea lor calitativă.

3.1.1 Configurarea experimentului

Evaluarea a rulat pe câte 150 de întrebări selectate aleator din split-ul de antrenare SQuAD [3], pe 4 seed-uri distincte (42, 1042, 2042, 3042), totalizând 600 de evaluări per sistem. Rularea pe mai multe seed-uri permite estimarea variabilității și raportarea mediei împreună cu deviația standard, în loc de o singură cifră care ar putea depinde de eșantionul ales.

Metricile calculate acoperă atât recuperarea, cât și generarea. Context Recall@5 este fracția de întrebări pentru care cel puțin unul dintre cei $K=5$ candidați recuperați conține răspunsul corect și provine din articolul corect. Întrucât fiecare întrebare are un singur document-sursă, această măsură binară coincide cu Recall@5 din Capitolul 1, deoarece regăsirea documentului relevant înseamnă recall complet. Condiția privind sursa este necesară pentru a preveni potrivirile false pe fragmente scurte și frecvente, precum „mai” sau „SUA”, care pot apărea în contexte complet diferite [3]. Mean Reciprocal Rank (MRR) este inversul rangului primului fragment relevant, mediat pe toate întrebările [20]. MRR penalizează sistemele care plasează fragmentul corect pe poziții inferioare în lista de rezultate.

Answer F1 este scorul F_1 la nivel de token între răspunsul generat și răspunsurile de referință SQuAD, calculat ca maxim peste toate variantele de referință disponibile. Exact Match este proporția de răspunsuri generate care conțin textul de referință ca subsir exact, după normalizarea majusculilor și a semnelor de punctuație. Latența medie de recuperare măsoară timpul de procesare a interogării în milisecunde. Pentru Advanced RAG, aceasta include apelul LLM pentru expansiunea interogării, recuperarea hibridă paralelă și re-scorarea cu cross-encoder.

Pentru testarea semnificației statistice s-au aplicat două teste pereche pe cele 600 de înregistrări combinate din toate seed-urile. Un test pereche compară cele două sisteme pe aceeași întrebare, nu două grupuri independente, ceea ce elimină variația dată de dificultatea diferită a întrebărilor și izolează efectul arhitecturii de recuperare. Alegerea testului depinde de natura metricii.

Testul Wilcoxon signed-rank a fost aplicat pe metricile continue (MRR și Answer F1). Pentru fiecare întrebare se calculează diferența dintre scorul Advanced RAG și cel al Naive

RAG. Dacă cele două sisteme ar fi la fel de bune, aceste diferențe ar fi împărțite aproape egal între pozitive și negative, iar mărimea lor s-ar anula reciproc. Testul măsoară tocmai acest dezechilibru: dacă un sistem obține scoruri mai mari în mod constant, și încă cu marje mai însemnate, diferențele se acumulează într-o singură direcție, iar testul o semnalează. Spre deosebire de testul t pereche, Wilcoxon nu presupune că diferențele urmează o distribuție normală, ipoteză care nu se verifică pentru aceste metrice [3].

Testul McNemar a fost aplicat pe metricile binare (Context Recall și Exact Match), unde pentru fiecare întrebare fiecare sistem fie reușește, fie nu. McNemar se uită doar la perechile discordante, adică la întrebările unde un sistem reușește și celălalt eșuează, și verifică dacă cele două tipuri de discordanță sunt la fel de frecvente. Dacă un sistem reușește mult mai des acolo unde celălalt dă greș, diferența este semnificativă; cazurile în care ambele sisteme se comportă la fel nu aduc informație despre care este mai bun. O valoare p mică indică, în ambele teste, că diferența observată este prea sistematică pentru a fi pusă pe seama hazardului eșantionării.

3.1.2 Asigurarea unei comparații corecte

Validitatea concluziilor depinde de corectitudinea comparației, adică de garanția că singura diferență dintre cele două sisteme este arhitectura de recuperare. Mai multe decizii de proiectare servesc acest scop.

În primul rând, ambele fluxuri de procesare folosesc același model generativ (Qwen3-8B), același model de codificare (all-MiniLM-L6-v2) și aceeași bază de cunoștințe, indexată o singură dată. Diferă doar etapa de recuperare. În al doilea rând, ambele sisteme primesc exact același prompt de sistem, cu aceeași constrângere de ancorare, astfel încât diferențele de comportament la generare să nu provină din instrucțiuni diferite. În al treilea rând, ambele sisteme sunt evaluate pe exact aceleași întrebări, în aceeași ordine, pentru fiecare seed, ceea ce permite aplicarea testelor statistice pereche. De asemenea, înainte de fiecare rulare de evaluare, modelele și indexul BM25 sunt încărcate printr-o etapă de încălzire, astfel încât timpii de latență măsurați să nu includă costul unic de inițializare. Aceste măsuri reduc riscul ca diferențele observate să fie atribuite unor factori colaterali, nu arhitecturii de recuperare.

3.1.3 Rezultate

Rezultatele agregate pe cele 4 seed-uri sunt prezentate în Tabelul 3.1. Advanced RAG depășește Naive RAG pe toate metricile de calitate, cu o creștere de 12,0 puncte procentuale la Context Recall (de la 83,5% la 95,5%), 0,185 la MRR (de la 0,719 la 0,904) și 10,3 puncte procentuale la Exact Match (de la 64,2% la 74,5%). Answer F1 crește mai modest, cu 7,3% relativ (de la 0,191 la 0,205).

Metrică	Naive RAG	Advanced RAG
Context Recall@5	0,835 $\pm$ 0,006	0,955 $\pm$ 0,015
MRR	0,719 $\pm$ 0,013	0,904 $\pm$ 0,023
Exact Match	0,642 $\pm$ 0,027	0,745 $\pm$ 0,045
Answer F1	0,191 $\pm$ 0,008	0,205 $\pm$ 0,015
Latență medie (ms)	15,6 $\pm$ 0,1	769,3 $\pm$ 20,8

Tabel 3.1: Rezultate comparative Naive RAG vs. Advanced RAG pe SQuAD ($N = 600$ per sistem, medie $\pm$ deviație standard pe 4 seed-uri)

Figurile 3.1 și 3.2 ilustrează grafic diferențele pentru metricile de recuperare și de calitate a răspunsurilor. Figura 3.3 prezintă comparativ latența medie de procesare, iar Figura 3.4 oferă

o vedere de ansamblu a tuturor metricilor de calitate.

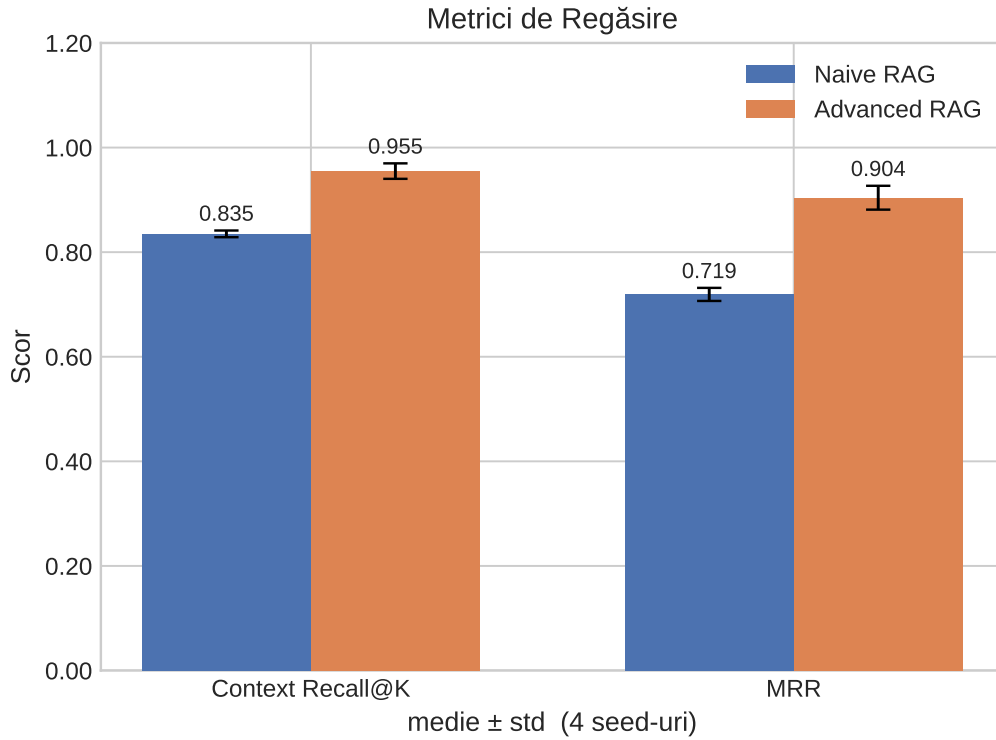


Figura 3.1: Metrici de recuperare: Context Recall@5 și MRR pentru Naive RAG și Advanced RAG. Barele de eroare reprezintă deviația standard calculată pe cele 4 seed-uri.

3.1.4 Semnificație statistică

Toate diferențele observate sunt semnificative statistic la $p < 0,05$, cu marje considerabile față de prag (Tabelul 3.2).

Metrică	Test	Valoarea p	Observații
MRR	Wilcoxon	$6,5 \times 10^{-27}$	
Answer F1	Wilcoxon	$1,8 \times 10^{-3}$	
Context Recall	McNemar	$7,7 \times 10^{-20}$	Adv. mai bun: 74; Naive mai bun: 2
Exact Match	McNemar	$4,3 \times 10^{-10}$	Adv. mai bun: 82; Naive mai bun: 20

Tabel 3.2: Teste de semnificație statistică (Advanced RAG vs. Naive RAG, $N = 600$ perechi)

Valoarea p extrem de mică pentru MRR ($6,5 \times 10^{-27}$) indică o diferență sistematică, nu un artefact al eșantionării. Testul McNemar pentru Context Recall arată că există 74 de întrebări unde Advanced RAG recuperează fragmentul corect, iar Naive RAG nu reușește, față de doar 2 în situația inversă. Raportul de 37:1 în favoarea sistemului avansat confirmă că îmbunătățirile în recuperare nu provin din cazuri marginale, ci reprezintă un avantaj consistent.

3.1.5 Stabilitatea rezultatelor pe seed-uri

Pentru a verifica dacă diferențele nu depind de un eșantion particular de întrebări, evaluarea a fost repetată pe patru seed-uri, fiecare selectând alte 150 de întrebări. Tabelul 3.3 prezintă valorile pentru Context Recall și MRR pe fiecare seed.

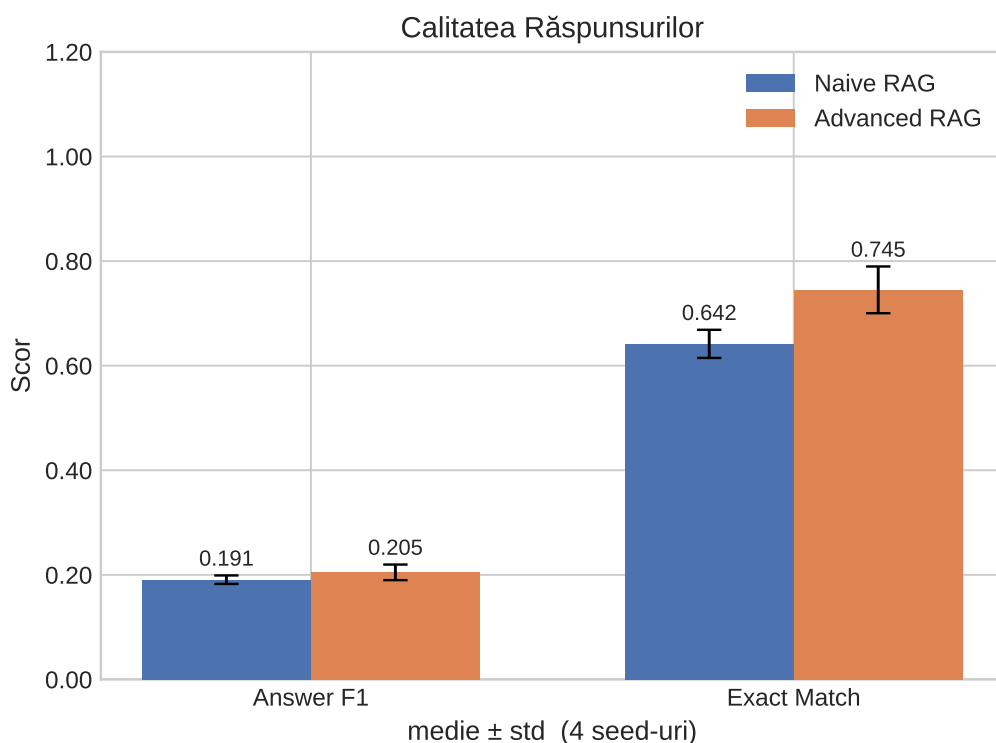


Figura 3.2: Calitatea răspunsurilor: Answer F1 și Exact Match pentru Naive RAG și Advanced RAG.

Seed	Context Recall@5		MRR	
	Naive	Advanced	Naive	Advanced
42	0,833	0,960	0,721	0,915
1042	0,840	0,947	0,728	0,898
2042	0,827	0,940	0,701	0,875
3042	0,840	0,973	0,727	0,928

Tabel 3.3: Context Recall@5 și MRR pe fiecare dintre cele 4 seed-uri (150 de întrebări per seed)

Se observă că Advanced RAG depășește Naive RAG pe fiecare seed în parte, nu doar în medie. Variația între seed-uri este mică pentru ambele sisteme (sub 2 puncte procentuale la Context Recall pentru Naive și sub 3,5 pentru Advanced), ceea ce arată că rezultatele sunt stabile și nu depind de un eșantion norocos. Această consistență întărește concluzia testelor de semnificație: avantajul arhitecturii avansate este real și reproductibil.

3.1.6 Unde ajută recuperarea hibridă

Diferența cea mai pronunțată dintre cele două sisteme apare la metricile de recuperare, nu la cele de generare. Pe SQuAD, creșterea cu 12,0 puncte la Context Recall înseamnă că, în aproximativ 12 din 100 de cazuri, Advanced RAG găsește fragmentul corect acolo unde Naive RAG îl ratează. MRR crescut de la 0,719 la 0,904 arată că, pe lângă faptul că găsește mai des fragmentul corect, îl și plasează pe poziții mai înalte.

Recuperarea hibridă (densă + BM25) are un avantaj clar pe interogările cu termeni specifici: date, numere, denumiri proprii, acronime. Modelul de codificare codifică sensul textului,

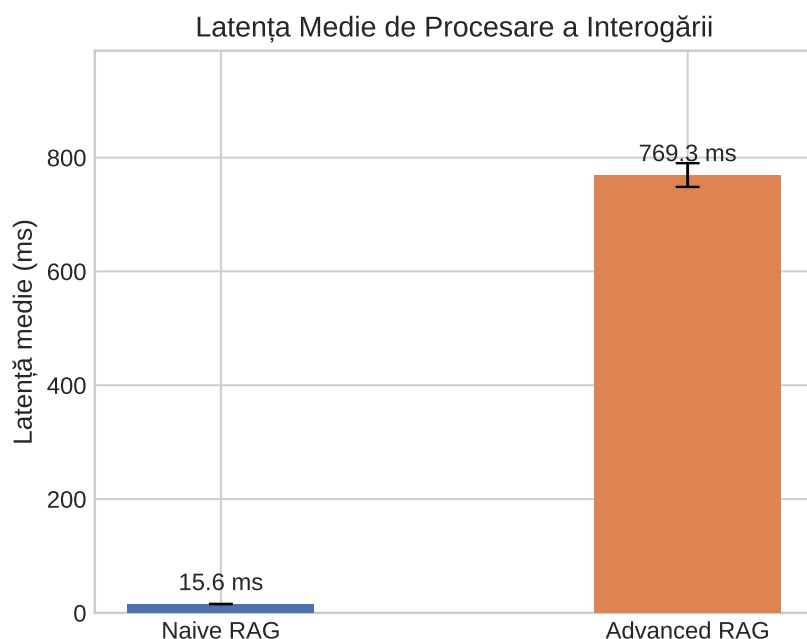


Figura 3.3: Latența medie de procesare a interogării în milisecunde. Latența Advanced RAG include apelul LLM pentru expansiunea interogării, recuperarea hibridă paralelă și re-scorarea cu cross-encoder.

dar poate dilua termenii rari într-un spațiu vectorial continuu unde similaritatea cosinus favorizează documentele apropiate tematic, nu neapărat identice lexical. BM25 [19] nu are această problemă: un termen care apare exact o dată în corpus va fi regăsit cu precizie dacă apare și în interogare. Expansiunea interogărilor contribuie la recall prin acoperirea variantelor de formulare, reducând dependența de formularea exactă a întrebării [5].

3.1.7 Decalajul dintre recuperare și generare

Îmbunătățirile la recuperare nu se traduc proporțional în îmbunătățiri la generare. Pe SQuAD, Context Recall crește cu 12 puncte procentuale, dar Answer F1 crește cu doar 7% relativ, iar Exact Match cu 10%.

Valoarea absolută scăzută a lui Answer F1, în jur de 0,2, vine din modul în care este calculată metrica, nu dintr-o calitate slabă a răspunsurilor. Answer F1 măsoară suprapunerea de token-uri dintre răspunsul generat și fragmentul de referință din SQuAD, care este de obicei foarte scurt, de unul sau două cuvinte. Modelul generativ răspunde însă în propoziții complete și tinde să parafrazeze, nu să reproducă textual fragmentul: la întrebarea „Unde s-a născut Einstein?”, referința este „Ulm”, dar modelul produce „Albert Einstein s-a născut la Ulm”. Răspunsul este corect, dar conține multe token-uri în plus față de referință, ceea ce coboară F1 chiar și atunci când răspunsul este corect semantic. Exact Match, care verifică doar dacă referința apare ca subșir în răspuns, nu este afectat de aceste token-uri suplimentare și de aceea rămâne mult mai mare (0,745 față de 0,205).

Îmbunătățirea modestă a lui Answer F1 are o explicație separată: Naive RAG recuperează corect fragmentul în 83,5% din cazuri, deci modelul generativ primea oricum contextul potrivit pentru cea mai mare parte a întrebărilor. Câștigul din Advanced RAG vine tocmai din cazurile dificile, adică interogările cu termeni rari sau cu răspunsuri scurte greu de localizat prin similaritate semantică pură, unde generarea oricum eșuează mai des.

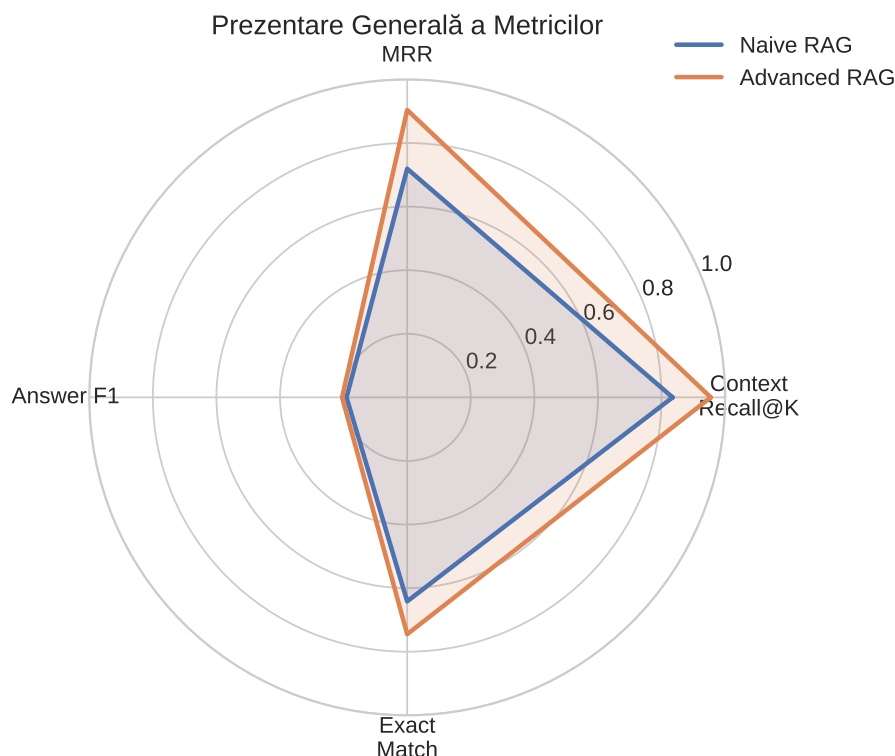


Figura 3.4: Vedere de ansamblu comparativă a metricilor de calitate (Context Recall@5, MRR, Answer F1, Exact Match). Latența nu este inclusă deoarece se află pe o scară diferită, în milisecunde față de intervalul de la 0 la 1.

3.1.8 Observații calitative din interfața Streamlit

Interfața Streamlit permite observarea directă a diferențelor de comportament fără a parcurge metricile agregate.

Pe întrebări cu termeni numerici sau coduri specifice („Câte specii...”, „În ce an...”, „Care este codul...”), Naive RAG returnează adesea fragmente tematic relevante care nu conțin numărul exact. Advanced RAG găsește fragmentul corect prin BM25, care tratează numerele ca termeni distincți și nu le diluează în spațiu vectorial.

Când răspunsul nu există în corpus, ambele sisteme refuză explicit. Diferența este că Advanced RAG produce mai rar răspunsuri parțiale construite din context adiacent irelevant, deoarece cross-encoder-ul elimină candidații slab relevanți înainte de generare.

Pe întrebări tematice generale („Despre ce este...”, „Care este rolul...”), ambele sisteme produc rezultate similare. Recuperarea densă funcționează bine pe similitudine semantică, iar avantajul hibridizării este mai puțin pronunțat în aceste cazuri.

Aceste observații sunt consistente cu rezultatele cantitative: câștigurile Advanced RAG sunt mai mari pe cazurile dificile (termeni rari, răspunsuri numerice) și mai mici pe interogările tematice generale, unde recuperarea densă funcționează deja bine [5, 19].

3.1.9 Analiza erorilor

Deși Advanced RAG depășește Naive RAG în ansamblu, testul McNemar arată că există și cazuri în care varianta de bază răspunde corect, iar cea avansată nu. La Exact Match, 20 de întrebări sunt rezolvate de Naive RAG, dar ratate de Advanced RAG, față de 82 în situația inversă. La Context Recall, raportul este de doar 2 la 74. Aceste cazuri minoritare merită analizate, deoarece arată limitele optimizărilor.

O primă sursă de erori ține de expansiunea interogărilor. Atunci când modelul de reformulare produce variante care se îndepărtează de sensul original, recuperarea hibridă poate aduce fragmente relevante pentru reformulare, dar nu pentru întrebarea reală. În aceste situații, simplitatea recuperării dense a Naive RAG, care lucrează direct cu interogarea originală, devine un avantaj. Acest fenomen explică o parte din cele 20 de cazuri unde Naive RAG obține Exact Match, iar Advanced RAG nu.

O a doua sursă ține de reordonare. *Cross-encoder*-ul reordonează fragmentele după relevanța estimată, dar estimarea nu este perfectă: ocazional, un fragment care conține răspunsul exact este plasat sub pragul de cinci fragmente, în timp ce recuperarea densă simplă l-ar fi păstrat. Aceste erori sunt rare, dar reale, și ilustrează că o etapă suplimentară de procesare nu garantează întotdeauna un rezultat mai bun pe fiecare interogare în parte.

A treia categorie ține de natura metricii Exact Match. O parte din diferențele observate nu reflectă răspunsuri greșite, ci reformulări: ambele sisteme pot produce un răspuns corect semantic, dar care nu se potrivește exact cu referința. În aceste cazuri, diferența la Exact Match este un artefact al metricii, nu o eroare reală de extragere. Această observație întărește argumentul că o evaluare completă necesită și metrici care tolerează reformularea, precum cele bazate pe fidelitate.

3.2 Evaluarea pe extragerea relațiilor între entități

Evaluarea pe SQuAD validează arhitectura pe o sarcină de tip întrebare-răspuns extractiv, dar nu măsoară direct scopul lucrării, extragerea relațiilor între entități. Pentru a testa exact această capacitate, demonstratorul a fost extins cu o a doua evaluare, pe setul de date DocRED [4], un corpus de extragere a relațiilor la nivel de document.

3.2.1 Setul de date DocRED și formularea ca întrebări

DocRED (Document-Level Relation Extraction Dataset) [4] conține documente Wikipedia adnotate manual, fiecare cu un set de entități și o listă de relații sub formă de triplete (subiect, relație, obiect), împreună cu propozițiile care susțin fiecare relație. Spre deosebire de seturile de extragere a relațiilor la nivel de propoziție, DocRED cere identificarea unor relații care se întind uneori pe mai multe propoziții, ceea ce îl apropie de scenariile reale de extragere a cunoștințelor [4].

Pentru a folosi aceeași infrastructură de evaluare ca în cazul SQuAD, fiecare triplet a fost reformulat ca întrebare, după modelul extragerii relațiilor prin citire (reading comprehension) [10]. Dintr-un triplet (entitate-cap, tip de relație, entitate-coadă) se construiește o întrebare care conține explicit atât tipul de relație, cât și entitatea-cap, după șablonul „Care este tipul-de-relație al entității-cap?”; răspunsul așteptat este entitatea-coadă, cu toate formele ei de suprafață acceptate ca variante corecte. Tipul de relație apare așadar explicit în întrebare, nu este omis: de exemplu, pentru tipul de relație „loc de naștere” și entitatea-cap „Marie Curie”, întrebarea generată este „Care este locul de naștere al Marie Curie?”, cu răspunsul așteptat „Varșovia”. În testul propriu-zis, întrebările au fost însă formulate în limba engleză, pentru a corespunde limbii documentelor și a etichetelor de relații din DocRED; pentru exemplul anterior, forma efectivă a întrebării este „What is the place of birth of Marie Curie?”. Documentul sursă devine contextul de referință, iar titlul lui servește la verificarea sursei, la fel ca în SQuAD. Această formulare corespunde sarcinii de completare a unui câmp lipsă (slot filling), în care modelul primește capul și tipul relației și trebuie să producă coada [10, 14]. Pentru fiecare document se păstrează un singur triplet, ales determinist, astfel încât unitatea de evaluare să rămână identică cu cea din SQuAD, o întrebare per document. Corpusul indexat în ChromaDB este format din documentele DocRED, deduplicate după titlu.

Pe plan tehnic, adăugarea celui de-al doilea set de date a presupus separarea logicii de date de cele două fluxuri de procesare, care erau legate inițial de o singură colecție. Fiecare set de date are acum propria colecție ChromaDB și propriul index BM25, astfel încât recuperarea să nu amestece niciodată documente din surse diferite. Ambele fluxuri de procesare rezolvă colecția activă în momentul interogării, în funcție de setul de date selectat la pornire.

Evaluarea pe DocRED a folosit câte 100 de întrebări pe fiecare dintre cele 3 seed-uri (42, 1042, 2042), totalizând 300 de întrebări, dintre care 204 cu răspuns în corpus și 96 fără răspuns. Întrebările fără răspuns au fost construite prin asocierea unei entități reale cu o relație pe care documentul ei nu o conține, aleasă determinist din vocabularul celor 96 de tipuri de relații din DocRED [4]. Aceste cazuri testează direct comportamentul de refuz al sistemului: răspunsul corect este abținerea. Trebuie remarcat că adnotările DocRED nu sunt exhaustive, deci o relație considerată inexistentă poate fi uneori adevărată în realitate, ceea ce adaugă zgomot acestei metrice [41]. Spre deosebire de evaluarea pe SQuAD, modelele de codificare și de reordonare au rulat pe procesor, lăsând întreaga memorie a plăcii grafice modelelor lingvistice; din acest motiv latențele absolute raportate pe DocRED sunt mai mari decât cele de pe SQuAD, măsurate cu modelele auxiliare pe placa grafică descrisă în Secțiunea 2.4.

3.2.2 Rezultate pe recuperare și generare

Tabelul 3.4 prezintă rezultatele pe DocRED. La fel ca pe SQuAD, Advanced RAG depășește net Naive RAG pe metricile de recuperare, dar de această dată marja este și mai mare. Context Recall crește de la 0,595 la 0,802 (+20,7 puncte procentuale), iar MRR de la 0,525 la 0,752 (+0,227). Pe SQuAD, aceleași diferențe erau de aproximativ 12 puncte la recall și 0,18 la MRR. Cu alte cuvinte, optimizările de recuperare ajută mai mult la extragerea relațiilor decât la întrebările extractive simple [5].

Metrică	Naive RAG	Advanced RAG	Semnificație
Context Recall@5	0,595 ± 0,137	0,802 ± 0,051	McNemar $p = 4,5 \times 10^{-10}$
MRR	0,525 ± 0,118	0,752 ± 0,069	Wilcoxon $p = 6,7 \times 10^{-11}$
Exact Match	0,375 ± 0,072	0,450 ± 0,020	McNemar $p = 0,036$
Answer F1	0,074 ± 0,010	0,073 ± 0,008	Wilcoxon $p = 0,97$ (n.s.)
Rata de halucinație	0,090 ± 0,048	0,071 ± 0,031	McNemar $p = 0,63$ (n.s.)
RAGAS Faithfulness	0,950 ± 0,019	0,874 ± 0,031	efect de selecție
RAGAS Answer Relevancy	0,876 ± 0,005	0,868 ± 0,007	—
Latență medie (ms)	34,3 ± 1,8	2092,8 ± 56,8	—

Tabel 3.4: Rezultate comparative pe DocRED ($N = 300$ per sistem: 204 cu răspuns plus 96 fără răspuns, medie ± deviație standard pe 3 seed-uri)

Explicația ține de natura interogărilor. O întrebare despre o relație conține aproape întotdeauna un nume propriu de entitate, capul relației, iar răspunsul corect este tot un nume propriu, coada. Recuperarea densă tinde să dilueze aceste nume rare într-un spațiu semantic unde domină documentele tematic apropiate, în timp ce BM25 [19] le potrivește exact. Pe DocRED, unde fiecare întrebare depinde de un nume specific, avantajul recuperării hibride devine mai pronunțat decât pe SQuAD (Figura 3.5).

Trebuie totuși remarcat că linia de bază Naive pornește mai jos pe DocRED (0,595) decât pe SQuAD (0,835), așa că o parte din câștigul absolut mai mare se datorează și spațiului mai amplu de îmbunătățire disponibil, nu doar naturii interogărilor. Mecanismul lexical rămâne însă explicația de fond: analiza pe interogări de pe SQuAD a arătat că avantajul recuperării hibride se concentrează tocmai pe întrebările cu termeni rari și numerici, exact categoria în

care intră numele proprii de entități, dominante pe DocRED. Efectul de plafon și mecanismul lexical acționează deci în aceeași direcție, fără ca primul să îl anuleze pe al doilea.

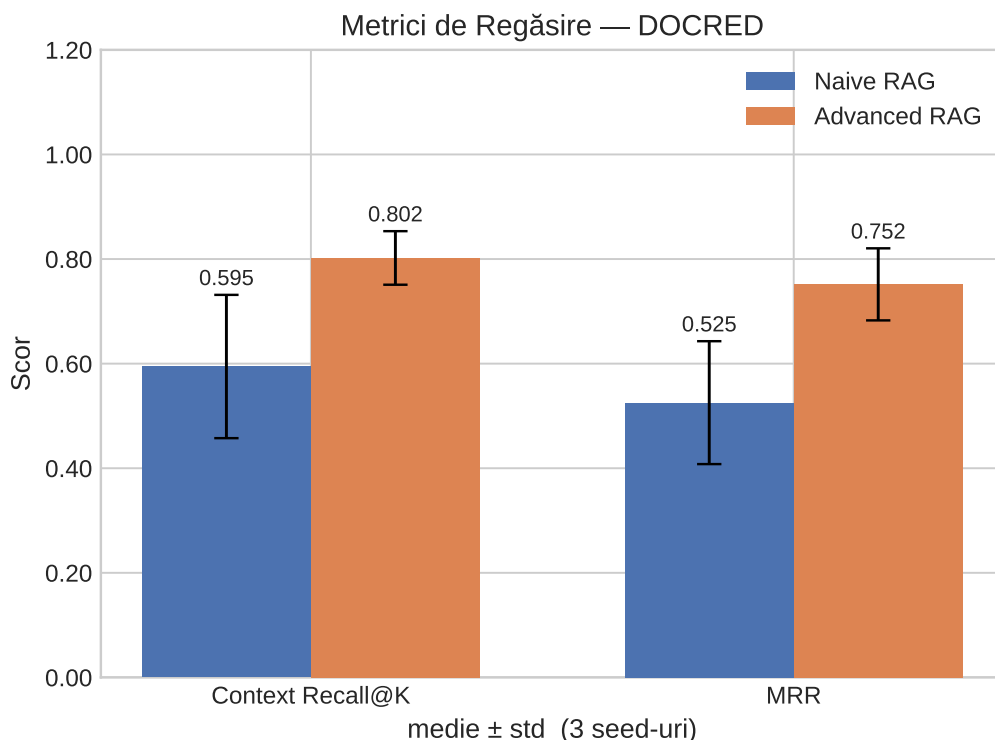


Figura 3.5: Metrici de recuperare pe DocRED: Context Recall@5 și MRR pentru Naive RAG și Advanced RAG. Avantajul arhitecturii avansate este vizibil mai mare decât pe SQuAD, deoarece întrebările despre relații depind de nume proprii de entități, pe care recuperarea hibridă le potrivește lexical. Barele de eroare reprezintă deviația standard pe cele 3 seed-uri.

La generare, imaginea este mai nuanțată. Exact Match crește de la 0,375 la 0,450, o diferență semnificativă statistic (McNemar, $p = 0,036$). Answer F1 rămâne însă scăzut și aproape neschimbat, în jur de 0,07 pentru ambele sisteme, fără diferență semnificativă. Motivul este că Answer F1 măsoară suprapunerea de token-uri între răspunsul generat și entitatea de referință, care este de obicei foarte scurtă, de unul sau două cuvinte, în timp ce modelul răspunde în propoziții complete. Pe DocRED, metrica relevantă pentru generare este așadar Exact Match, care verifică dacă entitatea corectă apare în răspuns, nu suprapunerea exactă de cuvinte (Figura 3.6) [3].

3.2.3 Rezistența la halucinații

Cele 96 de întrebări fără răspuns testează una dintre proprietățile centrale promise de RAG: refuzul de a inventa atunci când informația lipsește [22]. Pentru fiecare astfel de întrebare, comportamentul corect este abținerea, iar rata de halucinație măsoară fracția de cazuri în care sistemul a produs totuși un răspuns în loc să refuze. Valori mai mici sunt mai bune.

Ambele sisteme se descurcă bine, cu rate de halucinație scăzute: 0,090 pentru Naive și 0,071 pentru Advanced (Figura 3.7). Advanced RAG halucinează ceva mai rar, dar diferența nu este semnificativă statistic (McNemar, $p = 0,63$), pentru că numărul de întrebări negative este mic, 96, cu doar câteva perechi discordante. Rezultatul confirmă totuși că promptul de ancorare funcționează: chiar și pe întrebări construite special ca să nu aibă răspuns, ambele sisteme refuză în peste 90% din cazuri, în loc să fabrice o relație inexistentă [22].

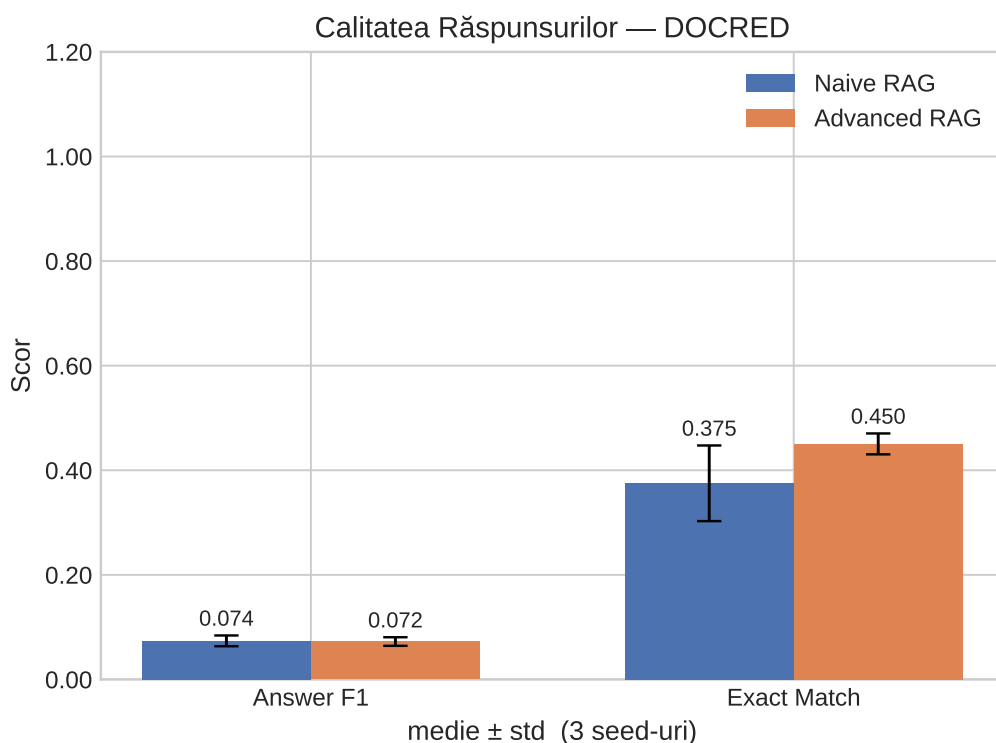


Figura 3.6: Calitatea răspunsurilor pe DocRED: Answer F1 și Exact Match. Answer F1 este scăzut și plat deoarece compară token-uri cu entități de referință foarte scurte, în timp ce modelul răspunde în propoziții; Exact Match, care verifică prezența entității corecte, este metrica de generare relevantă aici.

3.2.4 Fidelitatea răspunsurilor și efectul de selecție

Pe lângă potrivirea cu referința, calitatea răspunsurilor a fost evaluată și cu RAGAS [40], un cadru care folosește un model lingvistic ca evaluator pentru a estima fidelitatea (faithfulness) răspunsului față de contextul recuperat și relevanța lui față de întrebare. Refuzurile sunt excluse din această metrică, deoarece un refuz corect nu conține afirmații de verificat [40].

La prima vedere, rezultatele par să contrazică restul analizei: pe DocRED, Advanced RAG obține o fidelitate mai mică decât Naive (0,874 față de 0,950), deși recuperează vădit mai bine (Figura 3.8). Explicația nu ține de calitatea răspunsurilor, ci de faptul că cele două sisteme sunt evaluate pe seturi diferite de întrebări. Pentru că refuzurile sunt excluse, fiecare sistem este notat doar pe întrebările la care a ales să răspundă. Naive RAG refuză mai des, 116 din 204 întrebări, față de 102 pentru Advanced, deci este evaluat pe un subset mai mic și mai ușor, format din întrebările la care chiar și recuperarea lui mai slabă a găsit clar răspunsul. Advanced RAG încearcă în plus întrebări mai grele, inclusiv relații care se întind pe mai multe propoziții, unde afirmațiile sunt mai greu de fundamentat [4].

Pentru o comparație corectă, fidelitatea a fost recalculată pe subsetul comun, adică pe cele 70 de întrebări la care ambele sisteme au dat un răspuns efectiv, nu un refuz (Tabelul 3.5). Pe acest set identic, decalajul dispare aproape complet: 0,940 pentru Naive și 0,935 pentru Advanced, o diferență mai mică decât deviația standard între seed-uri, deci o egalitate statistică (Figura 3.9). Relevanța răspunsului este la fel de apropiată (0,877 față de 0,878). Pe întrebări identice, cele două sisteme sunt deci la fel de fidele contextului și la fel de relevante; deficitul aparent de fidelitate al sistemului avansat era un artefact de selecție, nu o slăbiciune reală de fundamentare [40].

Figura 3.10 sintetizează metricile de calitate pe DocRED, cu fidelitatea și relevanța calculate

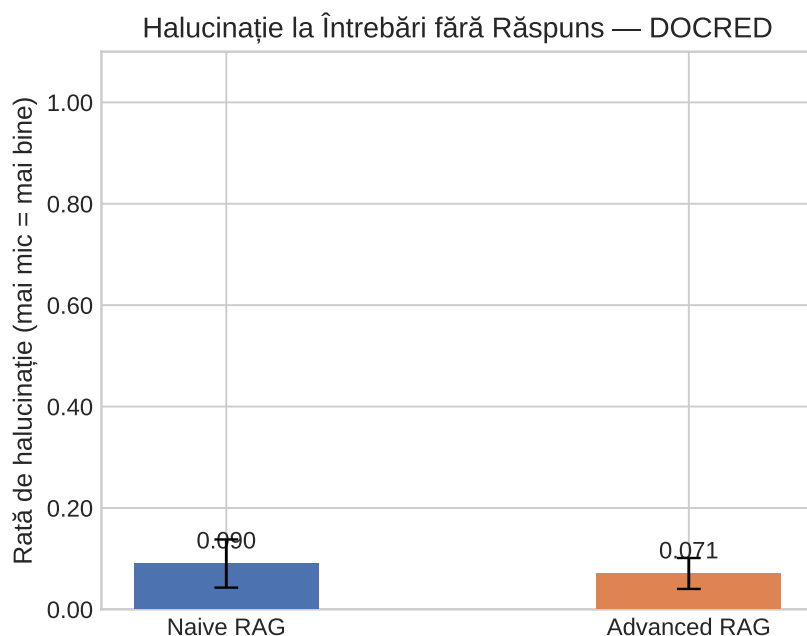


Figura 3.7: Rata de halucinație pe cele 96 de întrebări fără răspuns din DocRED (valori mai mici sunt mai bune). Ambele sisteme refuză să răspundă în peste 90% din cazuri, ceea ce confirmă efectul constrângerii de ancorare. Diferența dintre cele două nu este semnificativă statistic la acest număr de întrebări.

Metrică	Naive RAG	Advanced RAG
Faithfulness (fidelitate)	0,940 ± 0,012	0,935 ± 0,009
Answer Relevancy (relevanță)	0,877 ± 0,015	0,878 ± 0,022

Tabel 3.5: RAGAS pe subsetul comun de întrebări la care ambele sisteme au răspuns efectiv ($N = 70$, DocRED)

pe subsetul comun. Imaginea de ansamblu este consistentă: Advanced RAG conduce clar la recuperare (Context Recall, MRR) și la Exact Match, fără să piardă nimic la fidelitate sau relevanță. Concluzia evaluării pe extragerea relațiilor este că arhitectura avansată recuperează mai bine și răspunde la mai multe întrebări, fără un cost în fidelitatea răspunsului [4, 40].

3.3 Discuție și limitări

3.3.1 Compromisul dintre calitate și latență

Cel mai vizibil cost al arhitecturii avansate este latența de recuperare: pe SQuAD, 769 ms față de 16 ms pentru varianta de bază, o creștere de aproximativ 50 de ori, măsurată pe configurația hardware din Secțiunea 2.4. Pe DocRED, latențele absolute sunt mai mari, în jur de 2090 ms pentru Advanced, deoarece modelele auxiliare au rulat pe procesor, însă raportul dintre cele două sisteme se păstrează.

Defalcarea acestei latențe este instructivă. Componenta dominantă este apelul la modelul de expansiune a interogărilor, care, deși folosește un model mic, presupune o generare completă și consumă cea mai mare parte din cele 770 ms. Recuperarea densă și BM25 rulează în paralel pe cele trei formulări și contribuie relativ puțin, fiind operații de căutare optimizate. Reordonarea cu *cross-encoder* adaugă un cost intermediar, proporțional cu numărul de candidați evaluați,

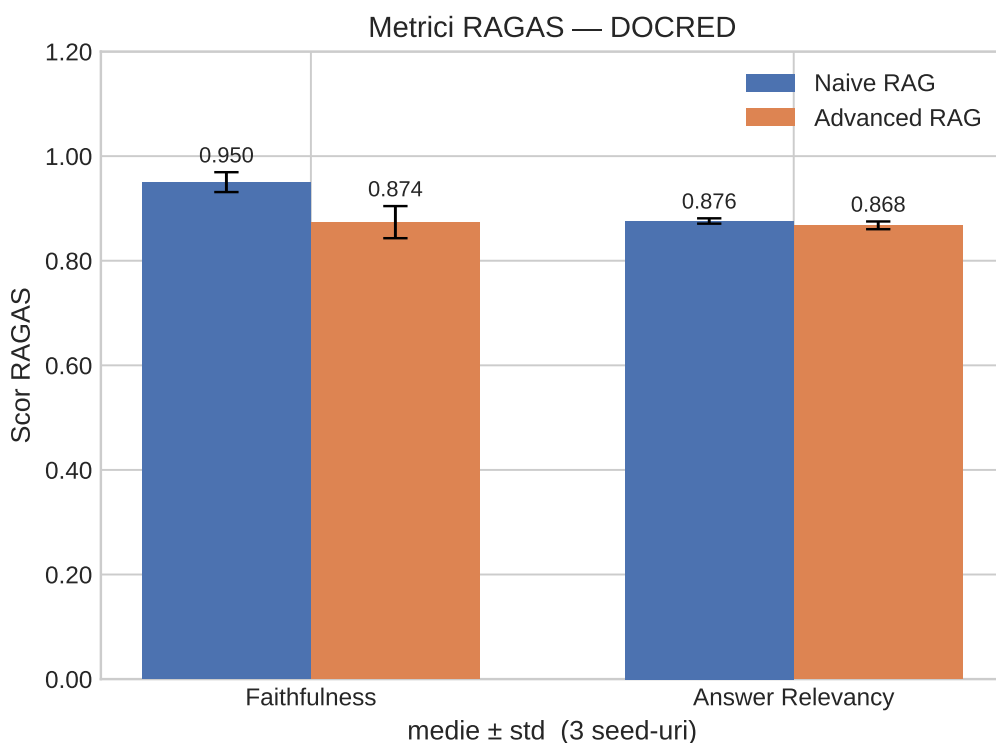


Figura 3.8: Metrici RAGAS pe DocRED, calculate pe subsetul propriu fiecărui sistem (refuzurile exclude). Advanced RAG pare mai puțin fidel decât Naive, dar comparația este înșelătoare: cele două sisteme sunt notate pe seturi diferite de întrebări, deoarece refuză întrebări diferite.

în cazul de față 40. Naive RAG evită toate aceste etape, ceea ce explică latența sa de doar 16 ms, formată aproape integral din codificarea întrebării și căutarea cosinus.

În contextul aplicațiilor RAG reale, o latență de recuperare de 770 ms rămâne acceptabilă, deoarece generarea unui răspuns cu un model de 8B parametri durează oricum mai multe secunde. Totuși, în scenarii unde se doresc timpi de răspuns sub o secundă, costul suplimentar al expansiunii interogărilor devine o constrângere reală și ar putea fi eliminat printr-o versiune fără expansiune, care combină recuperarea hibridă fără reformulare [5]. Această observație sugerează că expansiunea interogărilor ar trebui activată selectiv, doar pentru interogările unde aduce un câștig real de recall, nu pentru fiecare cerere.

3.3.2 Constrângerea de ancorare și refuzurile de a răspunde

Ambele fluxuri de procesare folosesc un prompt de sistem care constrânge modelul să răspundă exclusiv din contextul furnizat și să refuze explicit dacă informația lipsește. Exact promptul de refuz este: *“The provided context does not contain enough information to answer this question.”* Această decizie de proiectare este direct motivată de principiul fidelității contextuale descris în [22]: un sistem care preferă să refuze să răspundă, decât să inventeze, este mai fiabil decât unul care generează răspunsuri plauzibile dar incorecte.

Refuzurile nu pot fi evaluate cu Answer F1 sau Exact Match, deoarece nu conțin conținut factual comparabil cu referința. Advanced RAG generează mai puține refuzuri decât Naive RAG, pentru că recuperează mai frecvent fragmentul corect și îl plasează pe primele poziții, astfel că modelul primește mai rar un context din care nu poate răspunde.

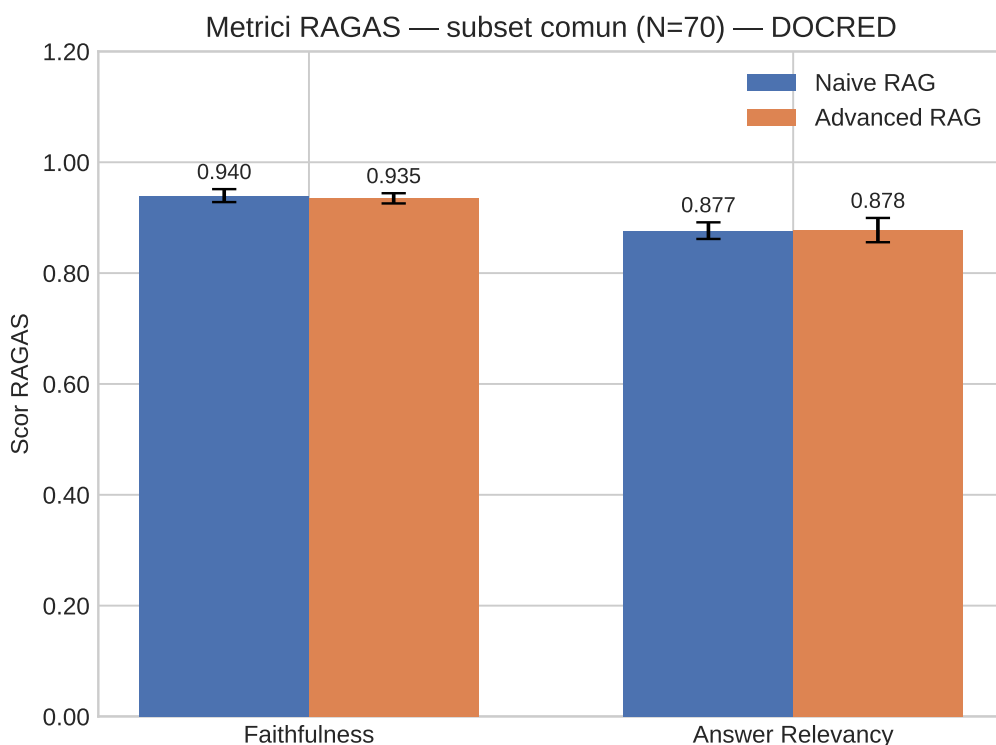


Figura 3.9: Metrici RAGAS pe subsetul comun ($N = 70$), unde ambele sisteme răspund la aceleași întrebări. Decalajul de fidelitate din Figura 3.8 dispăre: pe întrebări identice, cele două sisteme sunt la fel de fidele și la fel de relevante.

3.3.3 Limitările analizei

Rezultatele trebuie interpretate în lumina câtorva limitări ale configurației experimentale.

Prima ține de seturile de date și de modul în care a fost formulată sarcina. SQuAD este un set de date de tip extragere de răspunsuri, în care răspunsul este un fragment de text, nu o relație structurată; el validează arhitectura de recuperare, dar nu măsoară direct extragerea relațiilor. Pe DocRED, extragerea relațiilor a fost transformată în întrebări de completare a unui câmp lipsă (slot filling), în care modelul primește capul și tipul relației și trebuie să producă coada [10]. Această formulare nu testează clasificarea relațiilor sau extragerea completă a tuturor tripletelor dintr-un document, iar folosirea unui singur triplet per document nu valorifică bogăția adnotărilor DocRED, unde un document conține de regulă multe relații [4].

A doua ține de scala evaluării și de pregătirea datelor. Cele 600 de întrebări per sistem pe SQuAD și 300 pe DocRED oferă o bază statistică solidă, dar fiecare corpus provine dintr-o singură sursă, relativ omogenă ca stil. Pe corpusuri mai mari și mai diverse, comportamentul celor două sisteme ar putea diferi, în special în privința latenței și a calității recuperării. În plus, documentele DocRED sunt codificate integral, însă modelul de codificare trunchiază textul la maximum 256 de token-uri, ceea ce poate ascunde relații aflate în partea finală a documentelor lungi. Această limitare privește componenta densă a ambelor fluxuri de procesare, dar nu și BM25: indexul rar se construiește din textul brut integral al documentelor, fără trunchiere, deci poate potrivi termeni prezenți dincolo de limita ferestrei de context. Consecința este că Naive RAG, care se bazează exclusiv pe recuperarea densă, este integral afectat de trunchiere, în timp ce Advanced RAG beneficiază de recuperarea BM25 pentru a acoperi porțiunile trunchiate. Comparăția rămâne validă, deoarece ambele sisteme operează pe aceleași date indexate, dar valorile absolute de recall ale Advanced RAG pe DocRED reflectă și acest avantaj structural al recuperării hibride față de recuperarea densă pură, pe lângă mecanismul lexical deja descris [4].

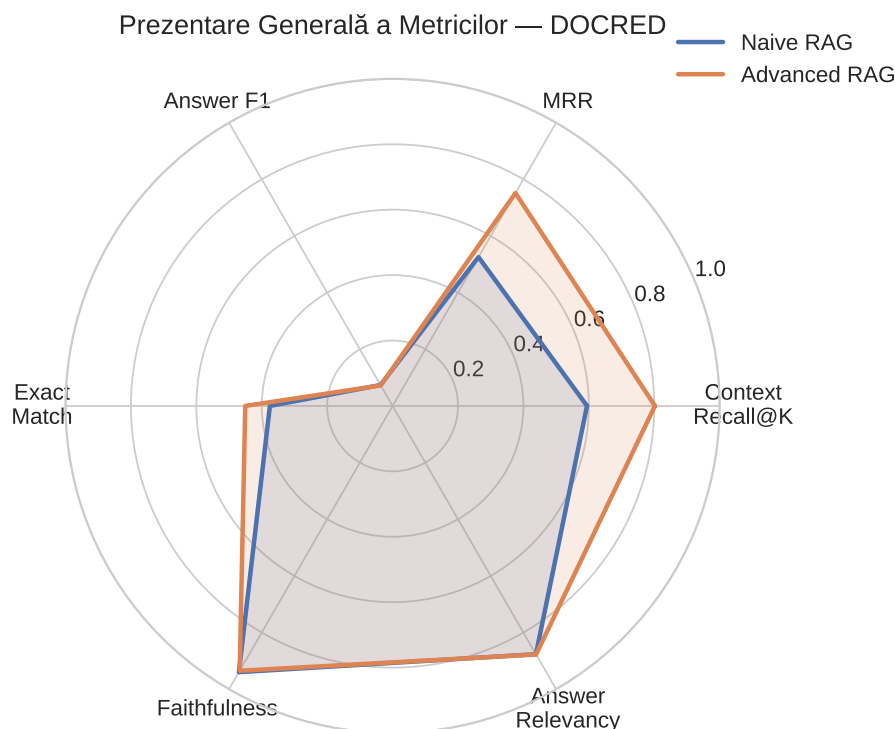


Figura 3.10: Vedere de ansamblu a metricilor de calitate pe DocRED. Fidelitatea și relevanța sunt calculate pe subsetul comun, pentru o comparație corectă. Advanced RAG conduce la recuperare și la Exact Match, iar la fidelitate și relevanță cele două sisteme sunt la egalitate. Latența nu este inclusă deoarece se află pe o scară diferită.

A treia ține de evaluarea calității răspunsului. Metricile de potrivire cu referința (Exact Match și Answer F1) nu surprind fidelitatea răspunsului față de contextul recuperat, motiv pentru care a fost adăugată evaluarea RAGAS. Aceasta are însă propriile limite: folosește tot un model lingvistic ca evaluator, din aceeași familie (Qwen3) cu cel care a generat răspunsurile, ceea ce poate introduce o părtinire, iar valorile de fidelitate trebuie interpretate ca relative între cele două sisteme, nu ca scoruri absolute [40]. De asemenea, întrebările fără răspuns folosite pentru testul de halucinație se bazează pe adnotările incomplete ale DocRED, astfel încât o relație presupusă inexistentă poate fi ocazional adevărată, adăugând zgomot acelei metrici [41].

3.4 Demonstratorul

Partea aplicativă a lucrării constă într-un demonstrator *open-source* care implementează cele două fluxuri de procesare și le expune printr-o interfață interactivă, alături de un modul de evaluare automată care a produs rezultatele din acest capitol. Demonstratorul este instrumentul studiului comparativ, nu scopul lui; această secțiune descrie interfața, modulul de evaluare și deciziile luate pe parcursul dezvoltării.

3.4.1 Interfața și fluxul utilizatorului

Interfața este construită cu Streamlit, în scriptul `main.py` (Anexa 2), și are forma unei conversații. Utilizatorul scrie o întrebare, iar răspunsurile celor două sisteme apar una lângă alta, pe două coloane, fiecare cu timpul de generare afișat deasupra. Această așezare face comparația imediată: pe aceeași întrebare se vede direct cum diferă răspunsurile și cât durează fiecare.

La pornire, interfața încarcă o singură dată modelele și indexul BM25 și le păstrează în

memorie printr-un mecanism de cache. Fără acest pas, fiecare interogare ar reîncărca modelele de pe disc, ceea ce ar face interacțiunea inutilizabil de lentă. Când utilizatorul trimite o întrebare, sistemul recuperează fragmentele pentru ambele fluxuri de procesare, apoi generează cele două răspunsuri în paralel, pe fire de execuție separate, ca să nu aștepte un sistem după celălalt. Răspunsurile sunt apoi adăugate în istoricul conversației, păstrat pe durata sesiunii.

Legătura dintre interfață și arhitectura RAG este directă. Coloana din stânga apelează fluxul de procesare Naive, cea din dreapta fluxul de procesare Advanced, iar ambele citesc din aceleași indexuri construite în faza de indexare. Interfața nu conține logică de recuperare proprie; ea doar orchestrează apelurile către cele două module și afișează rezultatele. Figurile 3.11 și 3.12 prezintă interfața în uz.

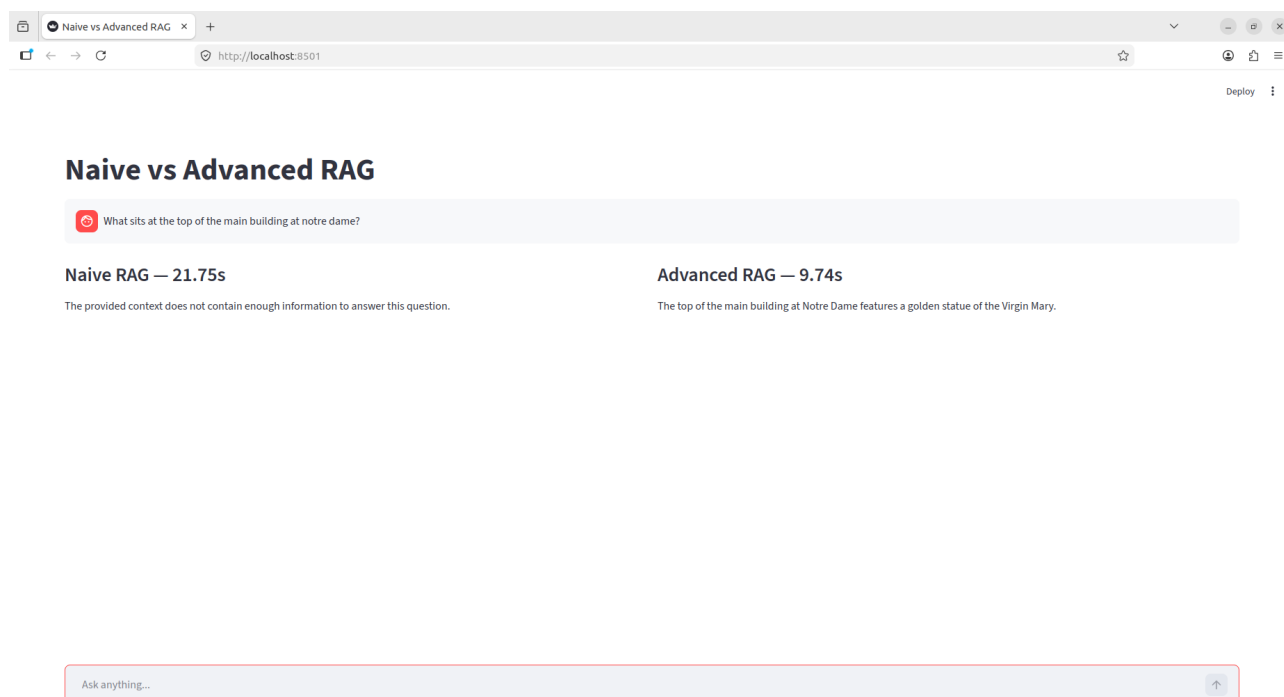


Figura 3.11: Interfața comparativă a demonstratorului. Cele două coloane corespund fluxurilor de procesare Naive și Advanced, care răspund simultan la aceeași întrebare, cu timpii de generare afișați deasupra fiecărui răspuns. În acest exemplu, Naive RAG nu găsește răspunsul, în timp ce Advanced RAG recuperează fragmentul corect.

3.4.2 Modulul de evaluare

Pe lângă cele două fluxuri de procesare și interfață, demonstratorul include un modul de evaluare automată, implementat în scriptul `benchmark.py` (Anexa 3). Rolul lui este să ruleze ambele sisteme pe același eșantion de întrebări și să producă metricile, testele statistice și figurile raportate în acest capitol, fără intervenție manuală.

Modulul funcționează în bucle imbricate: pentru fiecare seed se extrage un eșantion de întrebări din setul de date activ (SQuAD sau DocRED), iar pentru fiecare întrebare se rulează ambele fluxuri de procesare și se înregistrează fragmentele recuperate, timpul de procesare și, opțional, răspunsul generat. Evaluarea răspunsurilor este opțională tocmai pentru că generarea este pasul cel mai lent: metricile de recuperare se pot calcula rapid și separat, fără apeluri la modelul generativ.

O decizie de implementare care s-a dovedit utilă este salvarea de checkpoint: după fiecare seed, rezultatele parțiale se scriu pe disc printr-o înlocuire atomică de fișier, astfel încât o

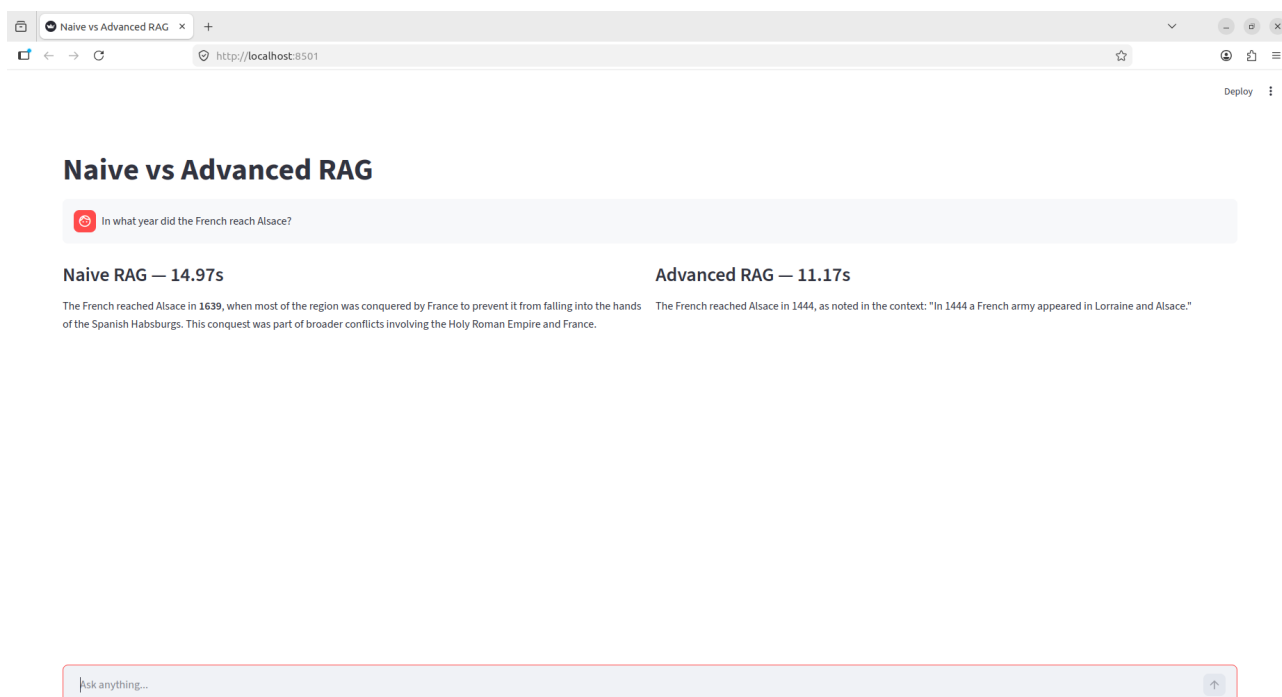


Figura 3.12: Exemplu de interogare cu termeni numerici. Naive RAG produce un răspuns numeric halucinat, deoarece similaritatea cosinus ratează valoarea exactă, în timp ce Advanced RAG găsește valoarea corectă prin recuperarea BM25.

întrerupere în timpul rulării (care poate dura ore cu generarea activată) să nu piardă seed-urile deja terminate. O altă priviște tratarea refuzurilor la evaluarea cu RAGAS: răspunsurile în care sistemul declară explicit că informația lipsește din context sunt excluse din evaluarea RAGAS, deoarece un refuz corect nu conține afirmații de verificat și ar fi altfel penalizat ca un răspuns nerelevant, ceea ce ar distorsiona comparația în defavoarea sistemului mai prudent.

La final, modulul agregă rezultatele pe toate seed-urile, calculează media și deviația standard pentru fiecare metrică, rulează testele de semnificație pe perechile de rezultate și generează automat figurile comparative în format PDF și PNG, gata de inclus în lucrare.

3.4.3 Procesul de dezvoltare

Implementarea a pornit cu varianta minimă funcțională: indexarea datelor SQuAD în ChromaDB și un flux de procesare Naive conectat la interfața Streamlit. Abia după ce varianta de bază funcționa de la cap la coadă au fost adăugate, una câte una, componentele Advanced RAG, ca să se poată vedea la fiecare pas ce aduce fiecare optimizare.

Prima extensie a fost recuperarea rară prin BM25 [19]. Inițial s-a folosit biblioteca `rank_bm25`, dar aceasta a fost înlocuită cu `bm25s`, mult mai rapidă prin tokenizare în loturi și indexare optimizată. Pentru că reconstruirea indexului la fiecare pornire era costisitoare, indexul se calculează o singură dată și se salvează pe disc.

Configurarea cross-encoder-ului a cerut câteva încercări. Au fost testate mai multe modele de reordonare, inclusiv o variantă mai mare, abandonată în favoarea lui `ms-marco-MiniLM-L-6-v2`, care oferă un compromis bun între precizie și viteză. Setul de candidați trimiși la reordonare a crescut de la 20 la 40 pe parcurs, după ce s-a observat că un set mai mic tăia uneori fragmentele relevante înainte ca *cross-encoder*-ul să le poată evalua.

Expansiunea interogărilor a trecut și ea prin ajustări. Modelul de reformulare a fost schimbat pe Qwen3-1.7B, iar recuperările pentru variantele de interogare au fost paralelizate, pentru

că rularea lor secvențială dubla timpul de răspuns. Toate aceste decizii sunt vizibile în istoricul de versiuni al proiectului, unde fiecare modificare corespunde unui pas în rafinarea fluxului de procesare.

În final, modulul de evaluare a fost construit cu suport pentru mai multe seed-uri și checkpoint după fiecare rulare, astfel încât o întrerupere să nu piardă rezultatele deja calculate. Testele de semnificație statistică (Wilcoxon și McNemar) au fost adăugate ulterior, pentru a verifica dacă diferențele observate sunt reale sau doar artefacte ale eșantionării aleatoare.

Concluzii

Rezultate obținute

Lucrarea a urmărit două direcții paralele: un studiu al literaturii de specialitate privind sistemele de tip Retrieval-Augmented Generation (RAG) și o comparație, realizată printr-un demonstrator dedicat, care măsoară îmbunătățirea adusă de o arhitectură RAG avansată față de una de bază pe două tipuri de sarcini: răspunsul la întrebări (QA) extractiv, folosit ca referință, și extragerea relațiilor între entități, sarcina-țintă a lucrării.

Din studiul de literatură (Capitolul 1) au reieșit câteva concluzii clare. Problema fundamentală pe care o adresează RAG rămâne aceeași ca la apariția termenului în 2020 [1]: modelele lingvistice mari au cunoștințe fixate la momentul antrenării și generează răspunsuri incorecte atunci când nu dețin informația solicitată. RAG rezolvă această limitare nu prin reantrenare, ci prin recuperarea documentelor relevante la momentul interogării și condiționarea generării pe aceste documente. Diversitatea abordărilor din literatură, de la Naive RAG la arhitecturi modulare și grafuri de cunoștințe [5, 18], arată că nu există o soluție universală: alegerea arhitecturii depinde de tipul sursei de date, de latența acceptabilă și de natura sarcinii.

Studiul a identificat că metodele de recuperare hibridă (densă + BM25 [19]) și reordonarea cu *cross-encoder* aduc cele mai consistente îmbunătățiri față de recuperarea densă pură, în special pe interogări cu termeni rari sau numerici. Metodele adaptive precum Self-RAG [22] și CRAG [32] adaugă un nivel de auto-evaluare a calității recuperării, care reduce halucinațiile cauzate de documente irelevante. Pentru extragerea relațiilor între entități, abordările bazate pe grafuri de cunoștințe [2] sunt cele mai potrivite structural, deoarece relațiile sunt reprezentate explicit și pot fi traversate direct.

Analiza metodelor a arătat și o tendință de evoluție a domeniului: de la fluxuri de procesare liniare fixe, spre sisteme care evaluează și corectează adaptiv calitatea recuperării, și mai departe spre arhitecturi care structurează explicit cunoștințele sub formă de graf. Fiecare pas pe acest drum aduce un câștig de capacitate, dar și un cost suplimentar de complexitate și de calcul. Niciuna dintre metode nu domină pe toate criteriile, iar alegerea potrivită rămâne condiționată de aplicația concretă. Această observație justifică abordarea adoptată în lucrare, de a porni de la o arhitectură simplă și de a măsura efectul fiecărei optimizări, în loc de a implementa direct cea mai complexă variantă.

Din evaluarea experimentală (Capitolul 3) rezultatele arată că Advanced RAG depășește Naive RAG pe toate metricile de calitate, cu diferențe semnificative statistic ($p \ll 0,05$ în toate testele). Context Recall@5 a crescut de la 83,5% la 95,5% (+12,0 puncte procentuale), MRR de la 0,719 la 0,904 (+0,185), iar Exact Match de la 64,2% la 74,5% (+10,3 puncte procentuale). Testul McNemar pentru Context Recall a arătat că există 74 de întrebări unde Advanced RAG recuperează fragmentul corect iar Naive RAG nu, față de doar 2 în situația inversă, un raport de 37:1 în favoarea arhitecturii avansate.

Costul acestor îmbunătățiri este latența: Advanced RAG necesită 770 ms per interogare față de 16 ms pentru Naive RAG, o diferență de aproximativ 50 de ori. Cea mai mare parte al acestui cost suplimentar provine din apelul LLM pentru expansiunea interogărilor. În aplicații unde latența de recuperare poate fi ascunsă parțial de latența de generare (care durează oricum câteva secunde pentru modele de 8B parametri), compromisul este acceptabil. Îmbunătățirile la Answer F1 au fost mai modeste (7,3% relativ), ceea ce este de așteptat: SQuAD este un set

de date extractiv, iar modelele generative modifică formularea răspunsului chiar și atunci când contextul corect este disponibil.

Cel de-al doilea task, extragerea relațiilor între entități, a fost evaluat pe DocRED [4], un set de date de extragere a relațiilor la nivel de document, în care fiecare relație a fost reformulată ca întrebare [10]. Pe acest set, avantajul recuperării avansate este chiar mai mare decât pe SQuAD: Context Recall a crescut de la 0,595 la 0,802 (+20,7 puncte procentuale), iar MRR de la 0,525 la 0,752, ambele diferențe fiind semnificative statistic, deși o parte din acest câștig absolut mai mare ține și de linia de bază Naive mai joasă pe DocRED, nu doar de natura întrebărilor. Pe întrebări construite special fără răspuns în corpus, ambele sisteme au refuzat corect în peste 90% din cazuri, ceea ce confirmă rezistența la halucinații adusă de constrângerea de ancorare [22]. Evaluarea fidelității cu RAGAS [40], calculată pe subșetul de întrebări la care ambele sisteme au răspuns, a arătat o fidelitate și o relevanță practic egale între cele două arhitecturi, ceea ce înseamnă că recuperarea avansată răspunde la mai multe întrebări fără a pierde din fidelitatea răspunsului.

Observațiile calitative din interfața comparativă au confirmat ceea ce arătau deja cifrele. Avantajul recuperării hibride s-a văzut cel mai clar pe întrebările cu termeni numerici și denumiri rare, unde recuperarea densă a variantei de bază aducea fragmente tematic apropiate, dar fără valoarea exactă cerută. Pe întrebările tematice generale, cele două sisteme s-au comportat similar, ceea ce arată că optimizările contează mai ales pe cazurile dificile.

Analiza pe seed-uri și testele de semnificație confirmă că aceste rezultate sunt stabile și reproductibile, nu artefacte ale unui eșantion particular. Advanced RAG depășește varianta de bază pe fiecare seed în parte, iar analiza erorilor arată că cele câteva cazuri în care Naive RAG răspunde mai bine se explică prin reformulări nereușite ale interogării sau prin limitele metricii Exact Match, nu printr-un avantaj real al recuperării simple. În ansamblu, studiul de literatură și partea experimentală conduc la aceeași concluzie: într-un sistem RAG, etapa de recuperare decide în cea mai mare măsură calitatea răspunsului.

Contribuții originale

Contribuția principală a lucrării este studiul comparativ realizat: o măsurare controlată a efectului optimizărilor de recuperare, în care un sistem Naive RAG și unul Advanced RAG sunt comparate pe același set de date și același model lingvistic, izolând contribuția etapelor de recuperare. Studiul se sprijină pe un demonstrator *open-source*, care rulează local prin Ollama și ChromaDB, fără dependență de servicii externe.

Din punct de vedere tehnic, Advanced RAG adaugă patru optimizări față de varianta de bază. Recuperarea hibridă combină recuperarea densă (all-MiniLM-L6-v2 + ChromaDB) cu recuperarea rară (BM25 [19] prin `bm25s`); listele rezultate sunt apoi combinate prin Reciprocal Rank Fusion [21]. Expansiunea interogărilor folosește Qwen3-1.7B pentru a genera două reformulări ale interogării originale, ceea ce reduce dependența de formularea exactă. Reordonarea cu *cross-encoder* (`ms-marco-MiniLM-L-6-v2`) re-scorează cele mai bune 40 de candidați și returnează primele 5. Ambele sisteme refuză explicit când contextul recuperat nu conține informația solicitată, în loc să genereze răspunsuri plauzibile dar incorecte [22].

O contribuție separată este modulul de evaluare automată, care rulează ambele sisteme pe aceleași întrebări, pe mai multe seed-uri, cu checkpoint salvat atomic după fiecare rulare, astfel încât o întrerupere să nu piardă rezultatele deja calculate. Modulul aplică testele de semnificație statistică (Wilcoxon și McNemar) pe perechile de rezultate, exclude refuzurile corecte de la metrica de relevanță pentru a nu penaliza sistemul mai prudent și generează automat figurile comparative folosite în lucrare. Modulul a fost extins pentru a evalua ambele sisteme și

pe DocRED, cu un test de rezistență la halucinații pe întrebări fără răspuns și cu integrarea metricilor RAGAS de fidelitate și relevanță. Interfața Streamlit completează evaluarea automată cu testare interactivă: ambele sisteme răspund simultan la aceeași întrebare, iar răspunsurile, fragmentele recuperate și timpii de procesare apar unul lângă altul.

Taxonomia din Capitolul 1 a fost adaptată pentru extragerea relațiilor între entități, nu pentru domeniul RAG în ansamblu, ceea ce limitează acoperirea dar face clasificarea mai directă pentru înțelegerea alegerii arhitecturale din studiul comparativ.

Dincolo de cifrele de performanță, demonstratorul are valoare ca instrument de comparație: pentru că ambele sisteme rulează pe aceeași bază și pot fi interogate simultan, diferențele de comportament devin vizibile direct, nu doar prin metrici agregate. Interfața permite observarea felului în care recuperarea hibridă recuperează termeni pe care recuperarea densă îi ratează, sau a felului în care constrângerea de ancorare conduce la un refuz explicit în locul unei halucinații. Această transparentă face din demonstrator un punct de plecare util atât pentru extragerea relațiilor între entități, cât și pentru înțelegerea practică a compromisurilor descrise teoretic în studiul de literatură.

Perspective de dezvoltare ulterioară

Evaluarea pe extragerea relațiilor a fost realizată pe DocRED [4], însă într-o formulare simplificată: fiecare relație a fost transformată într-o întrebare de completare a entității-coadă (slot filling) [10], cu un singur triplet per document. Pasul următor natural este extragerea completă a tuturor tripletelor dintr-un document și clasificarea tipului de relație, nu doar regăsirea entității-coadă, împreună cu evaluarea pe seturi complementare precum T-REx [11], care aliniază text Wikipedia cu triplete Wikidata. Aceste extinderi ar valorifica mai deplin adnotările bogate ale seturilor de extragere a relațiilor.

Evaluarea RAGAS [40], cu metrici de fidelitate (*faithfulness*) și relevanță a răspunsului (answer relevancy), a fost realizată pe DocRED și a arătat că cele două arhitecturi sunt la fel de fidele contextului atunci când sunt comparate pe aceleași întrebări. O limită a acestei evaluări este că modelul judecător provine din aceeași familie (Qwen3) cu cel care generează răspunsurile, ceea ce poate introduce o părtinire. Reluarea evaluării cu un model judecător independent, eventual mai mare, ar oferi o estimare mai obiectivă a fidelității.

Pe latura arhitecturală, GraphRAG [2] și HippoRAG [17] indexează explicit relațiile dintre entități și le utilizează la recuperare, ceea ce le face candidați direcți pentru extragerea relațiilor față de fluxurile de procesare actuale bazate pe fragmente de text. O comparație extinsă cu aceste metode ar clarifica în ce măsură structura relațională explicită aduce beneficii față de recuperarea hibridă pe text nestructurat.

Latenta Advanced RAG (770 ms per interogare) provine în cea mai mare parte din apelul LLM pentru expansiunea interogărilor. O variantă fără expansiune, care păstrează recuperarea hibridă și cross-encoder-ul, ar reduce latenta la câteva zeci de milisecunde cu o pierdere relativ mică la recall. Această variantă ar fi utilă în scenarii cu constrângeri stricte de timp de răspuns.

Evaluarea actuală folosește Qwen3-8B ca model generativ, fără *fine-tuning*. Modele mai mari sau antrenate specific pe extragerea relațiilor pot produce îmbunătățiri la Answer F1 independent de arhitectura de recuperare. Separarea contribuției modelului generativ de contribuția fluxului de procesare de recuperare rămâne o direcție deschisă pentru cercetare ulterioară.

O direcție complementară privește extinderea evaluării dincolo de limba engleză. SQuAD și majoritatea seturilor de date folosite sunt în engleză, iar comportamentul sistemului pe texte în alte limbi, inclusiv în română, rămâne neexplorat. Modelul de codificare și cel generativ folosite au capacități multilingve, dar calitatea recuperării și a extragerii relațiilor pe corpusuri în limba română ar trebui măsurată separat, deoarece resursele și performanța modelelor diferă

semnificativ între limbi. O astfel de evaluare ar fi relevantă pentru aplicarea sistemului pe documente locale, unde nevoia de extragere a relațiilor este la fel de prezentă.

În fine, trecerea de la demonstrator la un sistem utilizabil în practică ar cere câteva completări. Corpusul ar trebui extins de la paragrafele SQuAD la documente reale ale aplicației vizate, cu o etapă de fragmentare adaptată formatului acestora. Indexul ar trebui actualizat incremental pe măsură ce apar documente noi, fără reindexarea întregii colecții. Pentru mai mulți utilizatori simultani, expansiunea interogărilor și reordonarea ar trebui servite din instanțe dedicate, deoarece partajarea unei singure plăci grafice între generare și recuperare devine un punct de strângere. Aceste completări nu schimbă arhitectura validată în lucrare, însă fără ele sistemul nu poate fi pus în fața unor utilizatori reali.

Pe scurt, lucrarea confirmă pe un caz concret o observație recurentă din literatură: în sistemele RAG, calitatea recuperării este factorul determinant pentru calitatea răspunsului final. Optimizările aduse de Advanced RAG, deși nu sunt complexe, produc îmbunătățiri consistente și semnificative statistic, la un cost de latență acceptabil pentru cele mai multe aplicații. Această concluzie justifică direcția aleasă și a fost confirmată pe extragerea relațiilor între entități, scopul final al cercetării, prin evaluarea pe DocRED; formulările mai complete ale acestei sarcini rămân o extindere naturală. Rezultatul se sprijină pe o metodologie strictă de comparație: aceeași bază de date, același model generativ, aceleași întrebări și teste statistice pereche. Fără aceste precauții, diferența dintre cele două arhitecturi ar fi putut fi atribuită la fel de bine întâmplării sau unor factori colaterali.

Bibliografie

- [1] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-Augmented Generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 9459–9474, 2020.
- [2] Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, and Jonathan Larson. From local to global: A graph RAG approach to query-focused summarization. *arXiv preprint arXiv:2404.16130*, 2024. doi: 10.48550/arXiv.2404.16130.
- [3] Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. SQuAD: 100,000+ questions for machine comprehension of text. In Jian Su, Kevin Duh, and Xavier Carreras, editors, *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, pages 2383–2392, Austin, Texas, November 2016. Association for Computational Linguistics. doi: 10.18653/v1/D16-1264.
- [4] Yuan Yao, Deming Ye, Peng Li, Xu Han, Yankai Lin, Zhenghao Liu, Zhiyuan Liu, Lixin Huang, Jie Zhou, and Maosong Sun. DocRED: A large-scale document-level relation extraction dataset. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 764–777, 2019. doi: 10.18653/v1/P19-1074.
- [5] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, Meng Wang, and Haofen Wang. Retrieval-Augmented Generation for Large Language Models: A survey, 2024.
- [6] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. Deep learning. *Nature*, 521(7553): 436–444, 2015. doi: 10.1038/nature14539.
- [7] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, Cambridge, MA, 2016.
- [8] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017. doi: 10.48550/arXiv.1706.03762.
- [9] Marti A. Hearst. Automatic acquisition of hyponyms from large text corpora. In *Proceedings of the 14th International Conference on Computational Linguistics (COLING)*, pages 539–545, 1992. doi: 10.3115/992133.992154.
- [10] Omer Levy, Minjoon Seo, Eunsol Choi, and Luke Zettlemoyer. Zero-shot relation extraction via reading comprehension. In *Proceedings of the 21st Conference on Computational Natural Language Learning (CoNLL)*, pages 333–342, 2017. doi: 10.18653/v1/K17-1034.
- [11] Hady Elsahar, Pavlos Vougiouklis, Arslan Remaci, Christophe Gravier, Jonathon Hare, Frederique Laforest, and Elena Simperl. T-REx: A large scale alignment of natural language with knowledge base triples. In *Proceedings of the Eleventh International Conference on Language Resources and Evaluation (LREC)*, pages 3448–3452, 2018.

- [12] Claire Gardent, Anastasia Shimorina, Shashi Narayan, and Laura Perez-Beltrachini. Creating training corpora for NLG micro-planners. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 179–188, 2017. doi: 10.18653/v1/P17-1017.
- [13] Iris Hendrickx, Su Nam Kim, Zornitsa Kozareva, Preslav Nakov, Diarmuid Ó Séaghdha, Sebastian Padó, Marco Pennacchiotti, Lorenza Romano, and Stan Szpakowicz. SemEval-2010 task 8: Multi-way classification of semantic relations between pairs of nominals. In *Proceedings of the 5th International Workshop on Semantic Evaluation (SemEval)*, pages 33–38, 2010.
- [14] Yuhao Zhang, Victor Zhong, Danqi Chen, Gabor Angeli, and Christopher D. Manning. Position-aware attention and supervised data improve slot filling. In *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 35–45, 2017. doi: 10.18653/v1/D17-1004.
- [15] Mike Mintz, Steven Bills, Rion Snow, and Daniel Jurafsky. Distant supervision for relation extraction without labeled data. In *Proceedings of the Joint Conference of the 47th Annual Meeting of the ACL and the 4th International Joint Conference on Natural Language Processing (ACL-IJCNLP)*, pages 1003–1011, 2009. doi: 10.3115/1690219.1690287.
- [16] Michele Banko, Michael J. Cafarella, Stephen Soderland, Matthew Broadhead, and Oren Etzioni. Open information extraction from the web. In *Proceedings of the 20th International Joint Conference on Artificial Intelligence (IJCAI)*, pages 2670–2676, 2007.
- [17] Bernal Jiménez Gutiérrez, Yiheng Shu, Yu Gu, Michihiro Yasunaga, and Yu Su. Hippo-RAG: Neurobiologically inspired long-term memory for large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [18] Boci Peng, Yun Zhu, Yongchao Liu, Xiaohe Bo, Haizhou Shi, Chuntao Hong, Yan Zhang, and Siliang Tang. Graph Retrieval-Augmented Generation: A survey. *arXiv preprint arXiv:2408.08921*, 2024. doi: 10.48550/arXiv.2408.08921.
- [19] Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009. doi: 10.1561/1500000019.
- [20] Vladimir Karpukhin, Barlas Oğuz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 6769–6781, 2020. doi: 10.18653/v1/2020.emnlp-main.550.
- [21] Gordon V. Cormack, Charles L. A. Clarke, and Stefan Buettcher. Reciprocal rank fusion outperforms condorcet and individual rank learning methods. In *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 758–759, 2009. doi: 10.1145/1571941.1572114.
- [22] Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. Self-RAG: Learning to retrieve, generate, and critique through self-reflection. In *The Twelfth International Conference on Learning Representations*, 2023.
- [23] Yu A. Malkov and Dmitry A. Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42(4):824–836, 2020. doi: 10.1109/TPAMI.2018.2889473.

- [24] Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3):535–547, 2021. doi: 10.1109/TBDATA.2019.2921572.
- [25] Jianguo Wang, Xiaomeng Yi, Rentong Guo, Hai Jin, Peng Xu, Shengjun Li, Xiangyu Wang, Xiangzhou Guo, Chengming Li, Xiaohai Xu, Yuxing Yu, Yu Yuan, Yinghao Zou, Jiquan Long, Yudong Cai, Zhenxiang Li, Zhifeng Zhang, Yihua Mo, Jun Gu, Ruiyi Jiang, Yi Wei, and Charles Xie. Milvus: A purpose-built vector data management system. In *Proceedings of the 2021 International Conference on Management of Data (SIGMOD)*, pages 2614–2627, 2021. doi: 10.1145/3448016.3457550.
- [26] Bowen Jin, Jinsung Yoon, Jiawei Han, and Serkan O Arik. Long-context LLMs meet RAG: Overcoming challenges for long inputs in RAG. *arXiv preprint arXiv:2410.05983*, 2024. doi: 10.48550/arXiv.2410.05983.
- [27] Omar Khattab and Matei Zaharia. Colbert: Efficient and effective passage search via contextualized late interaction over bert. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 39–48, 2020. doi: 10.1145/3397271.3401075.
- [28] Luyu Gao, Xueguang Ma, Jimmy Lin, and Jamie Callan. Precise zero-shot dense retrieval without relevance labels. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 1762–1777, 2023. doi: 10.18653/v1/2023.acl-long.99.
- [29] Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, and Ming-Wei Chang. REALM: Retrieval-Augmented language model pre-training. In *Proceedings of the International Conference on Machine Learning (ICML)*, pages 3929–3938, 2020.
- [30] Gautier Izacard and Edouard Grave. Leveraging passage retrieval with generative models for open domain question answering. In *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics (EACL)*, pages 874–880, 2021. doi: 10.18653/v1/2021.eacl-main.74.
- [31] Sebastian Borgeaud, Arthur Mensch, Jordan Hoffmann, et al. Improving language models by retrieving from trillions of tokens. In *Proceedings of the International Conference on Machine Learning (ICML)*, pages 2206–2240, 2022.
- [32] Shi-Qi Yan, Jia-Chen Gu, Yun Zhu, and Zhen-Hua Ling. Corrective retrieval augmented generation. *arXiv preprint arXiv:2401.15884*, 2024. doi: 10.48550/arXiv.2401.15884.
- [33] Soyeong Jeong, Jinheon Baek, Sukmin Cho, Sung Ju Hwang, and Jong C. Park. Adaptive-RAG: Learning to adapt retrieval-augmented large language models through question complexity. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, pages 7036–7050, 2024. doi: 10.18653/v1/2024.naacl-long.389.
- [34] Zhengbao Jiang, Frank F. Xu, Luyu Gao, Zhiqing Sun, Qian Liu, Jane Dwivedi-Yu, Yiming Yang, Jamie Callan, and Graham Neubig. Active retrieval augmented generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 7969–7992, 2023. doi: 10.18653/v1/2023.emnlp-main.495.
- [35] Parth Sarthi, Salman Abdullah, Aditi Tuli, Shubh Khanna, Anna Goldie, and Christopher D. Manning. RAPTOR: Recursive abstractive processing for tree-organized retrieval. In *International Conference on Learning Representations (ICLR)*, 2024.

- [36] Lang Mei, Siyu Mo, Zhihan Yang, and Chong Chen. A survey of multimodal retrieval-augmented generation. *arXiv preprint arXiv:2504.08748*, 2025. doi: 10.48550/arXiv.2504.08748.
- [37] Peng Xia, Kangyu Zhu, Haoran Li, Tianze Wang, Weijia Shi, Sheng Wang, Linjun Zhang, James Zou, and Huaxiu Yao. MMed-RAG: Versatile multimodal RAG system for medical vision language models. *arXiv preprint arXiv:2410.13085*, 2024. doi: 10.48550/arXiv.2410.13085.
- [38] Haoran Hao, Jiaming Han, Changsheng Li, Yu-Feng Li, and Xiangyu Yue. RAP: Retrieval-augmented personalization for multimodal large language models. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 14538–14548, 2025. doi: 10.1109/CVPR52734.2025.01355.
- [39] Jun Chen, Dannong Xu, Junjie Fei, Chun-Mei Feng, and Mohamed Elhoseiny. Document haystacks: Vision-language reasoning over piles of 1000+ documents. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 24817–24826, 2025. doi: 10.1109/CVPR52734.2025.02311.
- [40] Shahul Es, Jithin James, Luis Espinosa-Anke, and Steven Schockaert. RAGAS: Automated evaluation of Retrieval Augmented Generation. *arXiv preprint arXiv:2309.15217*, 2023. doi: 10.48550/arXiv.2309.15217.
- [41] Qingyu Tan, Lu Xu, Lidong Bing, Hwee Tou Ng, and Sharifah Mahani Aljunied. Revisiting DocRED – addressing the false negative problem in relation extraction. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 8472–8487, 2022. doi: 10.18653/v1/2022.emnlp-main.580.
- [42] Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, et al. Natural Questions: A benchmark for question answering research. *Transactions of the Association for Computational Linguistics (TACL)*, 7:453–466, 2019. doi: 10.1162/tacl.a.00276.
- [43] Mandar Joshi, Eunsol Choi, Daniel S. Weld, and Luke Zettlemoyer. TriviaQA: A large scale distantly supervised challenge dataset for reading comprehension. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 1601–1611, 2017. doi: 10.18653/v1/P17-1147.
- [44] Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. HotpotQA: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 2369–2380, 2018. doi: 10.18653/v1/D18-1259.
- [45] Xiao Yang, Kai Sun, Hao Xin, Yushi Sun, Nikita Bhalla, Xiangsen Chen, Sajal Choudhary, Rongze Daniel Gui, Ziran Will Jiang, Ziyu Jiang, et al. CRAG – comprehensive RAG benchmark. In *Advances in Neural Information Processing Systems (NeurIPS) – Datasets and Benchmarks Track*, 2024.

Anexa 1

Cod sursă - ingest.py

Scriptul de ingestie încarcă documentele dintr-un dataset înregistrat, le codifică vectorial cu modelul all-MiniLM-L6-v2 și le stochează în colecția ChromaDB corespunzătoare.

```
1 #!/usr/bin/env python3
2 """
3 Ingest a dataset's corpus into ChromaDB.
4
5 Both pipeline variants (naive and advanced) read from the same per-dataset
6 collection; the difference lies in retrieval architecture, not in data.
7
8 Run once per dataset:
9     python ingest.py                # squad (default)
10    python ingest.py --dataset docred
11 """
12
13 from __future__ import annotations
14
15 import argparse
16
17 import chromadb
18 from sentence_transformers import SentenceTransformer
19
20 from datasets_registry import CHROMA_PATH, active, available, set_active
21
22 EMBEDDING_MODEL = "all-MiniLM-L6-v2"
23 BATCH_SIZE = 256
24
25
26 def ingest(dataset: str) -> None:
27     """Embed and store the full corpus of *dataset* into ChromaDB.
28
29     Skips ingestion if the target collection already contains documents.
30     Embeddings are computed in batches of 'BATCH_SIZE' to bound memory usage.
31
32     :param dataset: registered dataset key (e.g. "squad", "docred")
33     """
34     set_active(dataset)
35     spec = active()
36
37     print(f"Loading '{dataset}' corpus...")
38     contexts = spec.load_corpus()
39     print(f"Found {len(contexts)} unique documents.")
40
41     print("Loading embedding model...")
42     embedder = SentenceTransformer(EMBEDDING_MODEL)
43
44     print("Connecting to ChromaDB...")
45     CHROMA_PATH.mkdir(parents=True, exist_ok=True)
46     client = chromadb.PersistentClient(path=str(CHROMA_PATH))
47     collection = client.get_or_create_collection(
48         spec.collection_name,
49         metadata={"hnsw:space": "cosine"},
50     )
51
52     if collection.count() > 0:
53         print(f"Collection '{spec.collection_name}' already contains "
54               f"{collection.count()} documents. Skipping.")
55         return
56
57     print("Embedding and ingesting...")
58     for i in range(0, len(contexts), BATCH_SIZE):
59         batch = contexts[i : i + BATCH_SIZE]
60         texts = [c["text"] for c in batch]
```

```

61     embeddings = embedder.encode(texts, show_progress_bar=False).tolist()
62
63     collection.add(
64         ids=[f"ctx_{i + j}" for j in range(len(batch))],
65         embeddings=embeddings,
66         documents=texts,
67         metadatas=[{"title": c["title"]} for c in batch],
68     )
69     print(f"  {i + len(batch)}/{len(contexts)}")
70
71     print(f"Done. {collection.count()} documents stored in collection "
72           f"'{spec.collection_name}'.")
73
74
75 if __name__ == "__main__":
76     parser = argparse.ArgumentParser(description=__doc__)
77     parser.add_argument("--dataset", choices=available(), default="squad",
78                         help="Which dataset's corpus to ingest (default: squad)")
79     args = parser.parse_args()
80     ingest(args.dataset)

```

Listing 1.1: ingest.py - ingestia corpus-ului în ChromaDB

Anexa 2

Cod sursă - main.py

Interfața web Streamlit afișează răspunsurile celor două pipeline-uri (Naive RAG și Advanced RAG) în paralel, măsurând latența fiecăruia.

```
1 import time
2 from concurrent.futures import ThreadPoolExecutor
3
4 import streamlit as st
5
6 import datasets_registry
7 from advanced import pipeline as adv
8 from advanced.retrieval import _get_bm25, _get_embedder as _adv_embedder, _get_reranker
9 from naive import pipeline as naive
10
11
12 @st.cache_resource
13 def _warmup(dataset: str) -> None:
14     """Load all lazy singletons for the selected dataset into memory.
15
16     Keyed by *dataset* so Streamlit rebuilds singletons when the user
17     switches datasets in the sidebar.
18
19     :param dataset: dataset key used as the Streamlit cache key
20     """
21     datasets_registry.set_active(dataset)
22     naive._get_embedder()
23     _adv_embedder()
24     _get_reranker()
25     _get_bm25()
26
27
28 st.set_page_config(layout="wide", page_title="Naive vs Advanced RAG")
29 st.title("Naive vs Advanced RAG")
30
31 dataset = st.sidebar.selectbox("Dataset", datasets_registry.available())
32 datasets_registry.set_active(dataset)
33 _warmup(dataset)
34
35 if "messages" not in st.session_state:
36     st.session_state.messages = []
37
38
39 def get_response(pipeline, prompt: str, chunks: list[dict]) -> tuple[str, float]:
40     """Stream a full response from *pipeline* and measure wall-clock latency.
41
42     :param pipeline: naive or advanced pipeline module (must expose 'stream')
43     :param prompt: user question
44     :param chunks: retrieved context chunks passed to the pipeline
45     :returns: tuple of (generated answer text, elapsed seconds)
46     """
47     start = time.perf_counter()
48     result = "".join(pipeline.stream(prompt, chunks))
49     elapsed = time.perf_counter() - start
50     return result, elapsed
51
52
53 for msg in st.session_state.messages:
54     with st.chat_message("user"):
55         st.markdown(msg["prompt"])
56
57     col1, col2 = st.columns(2)
58     col1.subheader(f"Naive RAG - {msg['naive_time']:.2f}s")
59     col1.markdown(msg["naive"])
60     col2.subheader(f"Advanced RAG - {msg['adv_time']:.2f}s")
```

```

61     col2.markdown(msg["advanced"])
62
63 if prompt := st.chat_input("Ask anything..."):
64     with st.chat_message("user"):
65         st.markdown(prompt)
66
67     naive_chunks = naive.retrieve(prompt)
68     adv_chunks = adv.retrieve(prompt)
69
70     with ThreadPoolExecutor(max_workers=2) as ex:
71         f_naive = ex.submit(get_response, naive, prompt, naive_chunks)
72         f_adv = ex.submit(get_response, adv, prompt, adv_chunks)
73         naive_resp, naive_time = f_naive.result()
74         adv_resp, adv_time = f_adv.result()
75
76     col1, col2 = st.columns(2)
77     col1.subheader(f"Naive RAG - {naive_time:.2f}s")
78     col1.markdown(naive_resp)
79     col2.subheader(f"Advanced RAG - {adv_time:.2f}s")
80     col2.markdown(adv_resp)
81
82     st.session_state.messages.append({
83         "prompt": prompt,
84         "naive": naive_resp,
85         "naive_time": naive_time,
86         "advanced": adv_resp,
87         "adv_time": adv_time,
88     })

```

Listing 2.1: main.py - interfața comparativă Streamlit

Anexa 3

Cod sursă - benchmark.py

Scriptul de evaluare rulează ambele pipeline-uri pe eşantioane extrase din dataset-ul selectat și calculează metricile de regăsire (*Context Recall@K*, MRR, latență), calitatea răspunsurilor (F1, Exact Match), metricile RAGAS (Faithfulness, Answer Relevancy) și testele de semnificație statistică (Wilcoxon, McNemar).

Sunt omise funcțiile auxiliare de afișare și generare figuri (`_plot_results`, `_print_results` etc.), păstrându-se exclusiv logica de evaluare și calculul metricilor.

```
1 #!/usr/bin/env python3
2 """
3 Benchmark: Naive RAG vs Advanced RAG on a sample from the SQuAD training set.
4
5 Retrieval metrics (always computed):
6     context_recall - fraction of questions where a gold answer span appears in
7                     a top-K chunk from the correct article
8     mrr            - Mean Reciprocal Rank of the first such chunk
9     avg_retrieval_ms - mean query-processing latency in milliseconds. NOTE: for the
10                      Advanced pipeline this includes the multi-query expansion LLM
11                      call, not just vector/BM25 retrieval.
12
13 Answer quality metrics (opt-in, requires LLM calls):
14     answer_f1      - SQuAD token-F1 against reference answer spans
15     exact_match    - SQuAD exact match against reference answer spans
16
17 RAGAS metrics (opt-in, implies --with-generation, very slow):
18     faithfulness   - how well the answer is grounded in the retrieved context
19     answer_relevancy - how relevant the answer is to the question
20
21 Results are reported as mean +/- std across all seeds.
22
23 Usage:
24     python benchmark.py
25     python benchmark.py --samples 100 --num-seeds 4
26     python benchmark.py --with-generation --samples 50
27     python benchmark.py --with-ragas --samples 50 --output results.json
28 """
29
30 from __future__ import annotations
31
32 import argparse
33 import json
34 import math
35 import random
36 import string
37 import time
38 from collections import Counter
39
40 import datasets_registry
41 from advanced import pipeline as adv
42 from naive import pipeline as naive
43
44 DEFAULT_SAMPLES = 200
45 DEFAULT_SEED = 42
46 DEFAULT_NUM_SEEDS = 4
47
48 REFUSAL_MARKER = "does not contain enough information"
49
50
51 def main() -> None:
52     """Parse CLI arguments and run the full benchmark pipeline."""
53     parser = argparse.ArgumentParser(
54         description=__doc__,
```

```

55     formatter_class=argparse.RawDescriptionHelpFormatter,
56 )
57 parser.add_argument("--samples", type=int, default=DEFAULT_SAMPLES,
58                     help=f"Questions per seed (default: {DEFAULT_SAMPLES})")
59 parser.add_argument("--seed", type=int, default=DEFAULT_SEED,
60                     help=f"Base random seed (default: {DEFAULT_SEED})")
61 parser.add_argument("--num-seeds", type=int, default=DEFAULT_NUM_SEEDS,
62                     help=f"Number of seeds to run (default: {DEFAULT_NUM_SEEDS})")
63 parser.add_argument("--with-generation", action="store_true",
64                     help="Also evaluate answer quality (slow - one LLM call per question
per pipeline)")
65 parser.add_argument("--with-ragas", action="store_true",
66                     help="Run RAGAS faithfulness and answer_relevancy (implies --with-
generation)")
67 parser.add_argument("--dataset", choices=datasets_registry.available(),
68                     default="squad",
69                     help="Which dataset to benchmark (default: squad)")
70 parser.add_argument("--neg-frac", type=float, default=0.0,
71                     help="Fraction of questions made UNANSWERABLE to measure hallucination
resistance")
72 parser.add_argument("--output", metavar="FILE",
73                     help="Save full results to a JSON file")
74 args = parser.parse_args()
75
76 if args.with_ragas:
77     args.with_generation = True
78
79 datasets_registry.set_active(args.dataset)
80 spec = datasets_registry.active()
81
82 seeds = [args.seed + i * 1000 for i in range(args.num_seeds)]
83 rows = spec.load_eval_rows()
84
85 naive.retrieve("warm-up query")
86 adv.retrieve("warm-up query")
87
88 pipelines = [("naive", naive), ("advanced", adv)]
89 per_seed_summaries: list[dict] = []
90 all_records: list[dict] = []
91
92 for seed_idx, seed in enumerate(seeds):
93     rng = random.Random(seed)
94     make_neg = spec.make_negative_record if args.with_generation else None
95     # Oversample 2x so we can absorb None returns from make_record/make_neg.
96     pool = rng.sample(rows, min(args.samples * 2, len(rows)))
97     sample: list[dict] = []
98     for row in pool:
99         use_neg = make_neg is not None and args.neg_frac > 0 and rng.random() < args.
neg_frac
100         record = make_neg(row) if use_neg else spec.make_record(row)
101         if record is not None:
102             record["seed"] = seed
103             sample.append(record)
104         if len(sample) >= args.samples:
105             break
106
107 records: list[dict] = []
108 ragas_rows: dict[str, list[dict]] = {"naive": [], "advanced": []}
109
110 for i, record in enumerate(sample):
111     question: str = record["question"]
112     gold_answers: list[str] = record["gold_answers"]
113     gold_title: str = record["gold_title"]
114
115     for name, pipeline in pipelines:
116         t0 = time.perf_counter()
117         chunks = pipeline.retrieve(question)
118         retrieval_ms = (time.perf_counter() - t0) * 1000
119
120         rank = _context_rank(chunks, gold_answers, gold_title)
121
122         entry: dict = {
123             "retrieval_ms": round(retrieval_ms, 2),
124             "recall": rank is not None,
125             "mrr": round(1.0 / rank, 4) if rank is not None else 0.0,
126             "rank": rank,

```



```

127         }
128
129         if args.with_generation:
130             try:
131                 answer = "".join(pipeline.stream(question, chunks))
132             except Exception as exc:
133                 answer = f"[ERROR: {exc}]"
134             entry["answer"] = answer
135             entry["abstained"] = REFUSAL_MARKER in answer.lower()
136
137             if record.get("answerable", True):
138                 entry["f1"] = round(
139                     max((_token_f1(answer, ref) for ref in gold_answers), default=0.0)
140                 , 4
141             )
142             entry["em"] = any(_exact_match(answer, ref) for ref in gold_answers)
143
144             if args.with_ragas:
145                 ragas_rows[name].append({
146                     "question": question,
147                     "answer": answer,
148                     "contexts": [c["text"] for c in chunks],
149                 })
150
151             record[name] = entry
152
153             records.append(record)
154
155             all_records.extend(records)
156
157             ragas_scores: dict = {}
158             if args.with_ragas:
159                 ragas_scores = _run_ragas(ragas_rows)
160
161             per_seed_summaries.append(_build_summary(records, args.with_generation, ragas_scores))
162
163             sig_tests = _significance_tests(all_records, args.with_generation)
164
165             if args.output:
166                 with open(args.output, "w") as fh:
167                     json.dump({
168                         "dataset": args.dataset,
169                         "per_seed_summaries": per_seed_summaries,
170                         "significance": sig_tests,
171                         "seeds": seeds,
172                         "records": all_records,
173                     }, fh, indent=2)
174
175 def _run_ragas(rows_by_pipeline: dict, llm_model: str = "qwen3:8b",
176               ragas_workers: int = 4) -> dict:
177     """Evaluate faithfulness and answer_relevancy with RAGAS for each pipeline.
178
179     Grounded-refusal answers (containing 'REFUSAL_MARKER') are excluded
180     from scoring because penalising a correct abstention distorts the metric.
181
182     'num_ctx=4096' is intentional: RAGAS prompts are well under 4 K tokens,
183     so the model's 40 K default wastes KV-cache and reduces parallelism.
184     """
185     try:
186         from ragas import EvaluationDataset, SingleTurnSample, evaluate
187         from ragas.llms import LangchainLLMWrapper
188         from ragas.embeddings import LangchainEmbeddingsWrapper
189         from ragas.metrics import Faithfulness, ResponseRelevancy
190         from langchain_ollama import ChatOllama
191         from langchain_community.embeddings import HuggingFaceEmbeddings
192     except ImportError as exc:
193         print(f"\nRAGAS dependencies missing: {exc}")
194         return {}
195
196     llm = LangchainLLMWrapper(ChatOllama(model=llm_model, num_ctx=4096))
197     embeddings = LangchainEmbeddingsWrapper(
198         HuggingFaceEmbeddings(model_name="all-MiniLM-L6-v2")
199     )
200
201     results: dict = {}

```

```

202     for name, rows in rows_by_pipeline.items():
203         if not rows:
204             continue
205
206         usable = [r for r in rows
207                   if REFUSAL_MARKER not in r["answer"].lower()
208                   and not r["answer"].startswith("[ERROR:")]
209         if not usable:
210             continue
211
212         samples = [
213             SingleTurnSample(
214                 user_input=r["question"],
215                 response=r["answer"],
216                 retrieved_contexts=r["contexts"],
217             )
218             for r in usable
219         ]
220         dataset = EvaluationDataset(samples=samples)
221         from ragas import RunConfig
222         result = evaluate(
223             dataset=dataset,
224             metrics=[Faithfulness(), ResponseRelevancy()],
225             llm=llm,
226             embeddings=embeddings,
227             run_config=RunConfig(max_workers=ragas_workers, timeout=600),
228         )
229         df = result.to_pandas()
230         faith_vals = [v for v in df["faithfulness"] if not math.isnan(v)]
231         relev_vals = [v for v in df["answer_relevancy"] if not math.isnan(v)]
232         results[name] = {
233             "faithfulness": round(_avg(faith_vals), 4),
234             "answer_relevancy": round(_avg(relev_vals), 4),
235         }
236
237     return results
238
239
240 def _context_rank(chunks: list[dict], gold_answers: list[str],
241                  gold_title: str | None = None) -> int | None:
242     """Return the 1-based rank of the first chunk containing a gold answer span.
243
244     When *gold_title* is given, a chunk only counts if it also comes from the
245     correct article. This prevents short answer spans (e.g. "May", "US") from
246     spuriously matching unrelated passages and inflating recall.
247     """
248     golds = [a.lower() for a in gold_answers]
249     title = gold_title.strip().lower() if gold_title else None
250     for i, chunk in enumerate(chunks):
251         if title is not None and chunk.get("source", "").strip().lower() != title:
252             continue
253         text = chunk["text"].lower()
254         if any(ans in text for ans in golds):
255             return i + 1
256     return None
257
258
259 def _normalize(text: str) -> str:
260     """Lowercase, strip punctuation, and collapse whitespace."""
261     text = text.lower().translate(str.maketrans("", "", string.punctuation))
262     return " ".join(text.split())
263
264
265 def _token_f1(prediction: str, reference: str) -> float:
266     """Compute SQuAD-style token-F1 between prediction and reference."""
267     pred_tokens = _normalize(prediction).split()
268     ref_tokens = _normalize(reference).split()
269     common = Counter(pred_tokens) & Counter(ref_tokens)
270     num_common = sum(common.values())
271     if num_common == 0 or not pred_tokens or not ref_tokens:
272         return 0.0
273     precision = num_common / len(pred_tokens)
274     recall = num_common / len(ref_tokens)
275     return 2 * precision * recall / (precision + recall)
276
277

```

```

278 def _exact_match(prediction: str, reference: str) -> bool:
279     """Return True if the normalised reference is a substring of normalised prediction."""
280     return _normalize(reference) in _normalize(prediction)
281
282
283 def _avg(values: list) -> float:
284     """Return the arithmetic mean, or 0.0 for an empty list."""
285     return sum(values) / len(values) if values else 0.0
286
287
288 def _build_summary(records: list[dict], with_generation: bool, ragas_scores: dict) -> dict:
289     """Build a per-seed metric summary from evaluated records.
290
291     Retrieval and answer-quality metrics are computed over answerable questions
292     only; the hallucination metric is computed over the unanswerable ones.
293     """
294     answerable = [r for r in records if r.get("answerable", True)]
295     negatives = [r for r in records if not r.get("answerable", True)]
296
297     summary = {}
298     for name in ("naive", "advanced"):
299         entries = [r[name] for r in records]
300         ans = [r[name] for r in answerable]
301         s: dict = {
302             "n": len(entries),
303             "context_recall": round(_avg([e["recall"] for e in ans]), 4),
304             "mrr": round(_avg([e["mrr"] for e in ans]), 4),
305             "avg_retrieval_ms": round(_avg([e["retrieval_ms"] for e in entries]), 2),
306         }
307         if with_generation:
308             s["answer_f1"] = round(_avg([e.get("f1", 0.0) for e in ans]), 4)
309             s["exact_match"] = round(_avg([float(e.get("em", False)) for e in ans]), 4)
310         if negatives:
311             neg = [r[name] for r in negatives]
312             s["hallucination_rate"] = round(
313                 _avg([0.0 if e.get("abstained") else 1.0 for e in neg]), 4
314             )
315         if name in ragas_scores:
316             s["faithfulness"] = ragas_scores[name]["faithfulness"]
317             s["answer_relevancy"] = ragas_scores[name]["answer_relevancy"]
318         summary[name] = s
319     return summary
320
321
322 def _significance_tests(records: list[dict], with_generation: bool) -> dict:
323     """Run paired Advanced-vs-Naive significance tests over pooled per-question results.
324
325     Continuous metrics (MRR, answer_f1) use the Wilcoxon signed-rank test.
326     Binary metrics (recall, exact_match) use McNemar's test (exact binomial).
327     All tests run over answerable questions only; the hallucination abstention
328     test runs over the unanswerable subset.
329     """
330     try:
331         from scipy.stats import binomtest, wilcoxon
332     except ImportError:
333         return {}
334
335     answerable = [r for r in records if r.get("answerable", True)]
336     negatives = [r for r in records if not r.get("answerable", True)]
337
338     tests: dict = {}
339
340     continuous = [("MRR", "mrr")]
341     if with_generation:
342         continuous.append(("Answer F1", "f1"))
343     for label, key in continuous:
344         naive_vals = [r["naive"][key] for r in answerable]
345         adv_vals = [r["advanced"][key] for r in answerable]
346         if not answerable or all(av == nv for av, nv in zip(adv_vals, naive_vals)):
347             p = float("nan")
348         else:
349             try:
350                 _, p = wilcoxon(adv_vals, naive_vals)
351             except ValueError:
352                 p = float("nan")
353         tests[label] = {"test": "Wilcoxon", "p": p, "n": len(answerable)}

```

```

354
355 binary = [("Context Recall@K", "recall")]
356 if with_generation:
357     binary.append(("Exact Match", "em"))
358 for label, key in binary:
359     adv_only = sum(1 for r in answerable if r["advanced"][key] and not r["naive"][key])
360     naive_only = sum(1 for r in answerable if r["naive"][key] and not r["advanced"][key])
361     discordant = adv_only + naive_only
362     p = 1.0 if discordant == 0 else binomtest(min(adv_only, naive_only), discordant, 0.5).
pvalue
363     tests[label] = {
364         "test": "McNemar", "p": p, "n": len(answerable),
365         "adv_better": adv_only, "naive_better": naive_only,
366     }
367
368 if negatives:
369     adv_only = sum(1 for r in negatives if r["advanced"].get("abstained") and not r["naive
370     naive_only = sum(1 for r in negatives if r["naive"].get("abstained") and not r["
advanced"].get("abstained"))
371     discordant = adv_only + naive_only
372     p = 1.0 if discordant == 0 else binomtest(min(adv_only, naive_only), discordant, 0.5).
pvalue
373     tests["Abstention (neg)"] = {
374         "test": "McNemar", "p": p, "n": len(negatives),
375         "adv_better": adv_only, "naive_better": naive_only,
376     }
377
378 return tests
379
380
381 if __name__ == "__main__":
382     main()

```

Listing 3.1: benchmark.py - pipeline-ul de evaluare (extras)